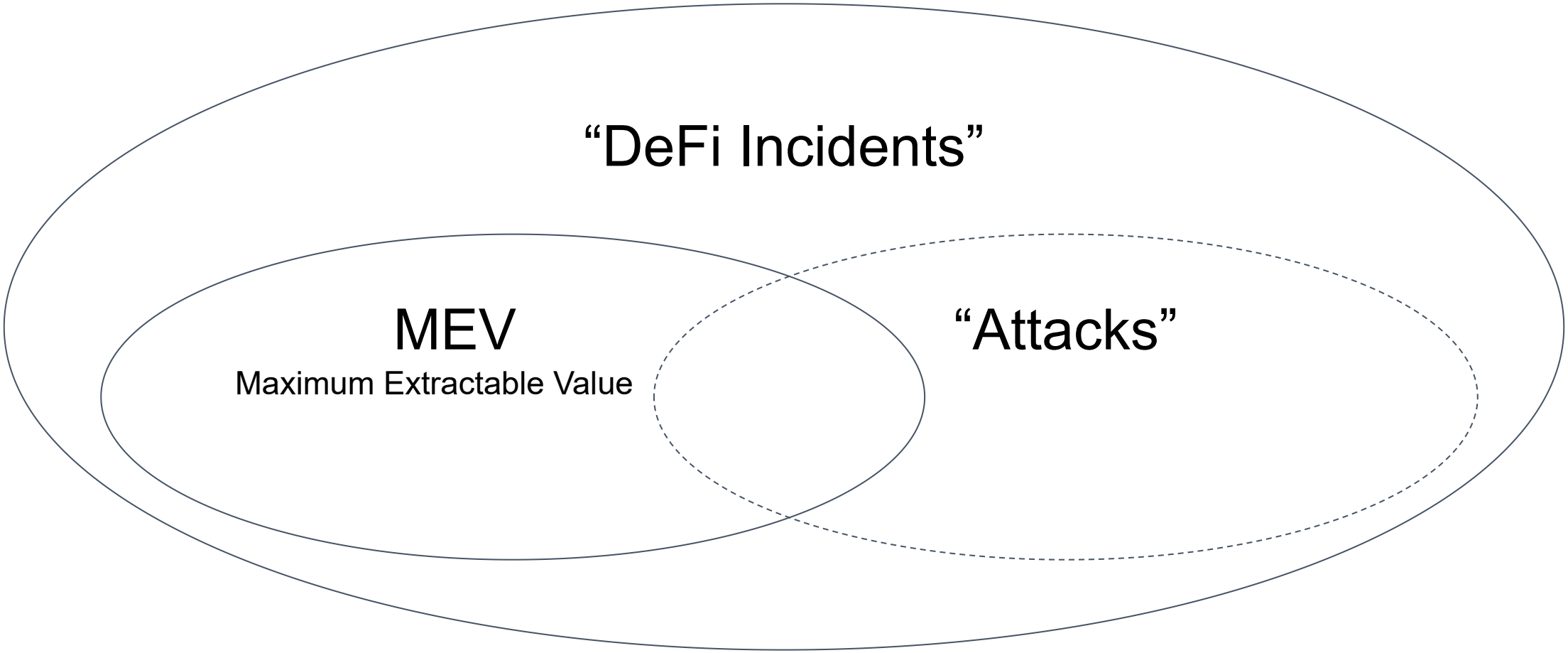
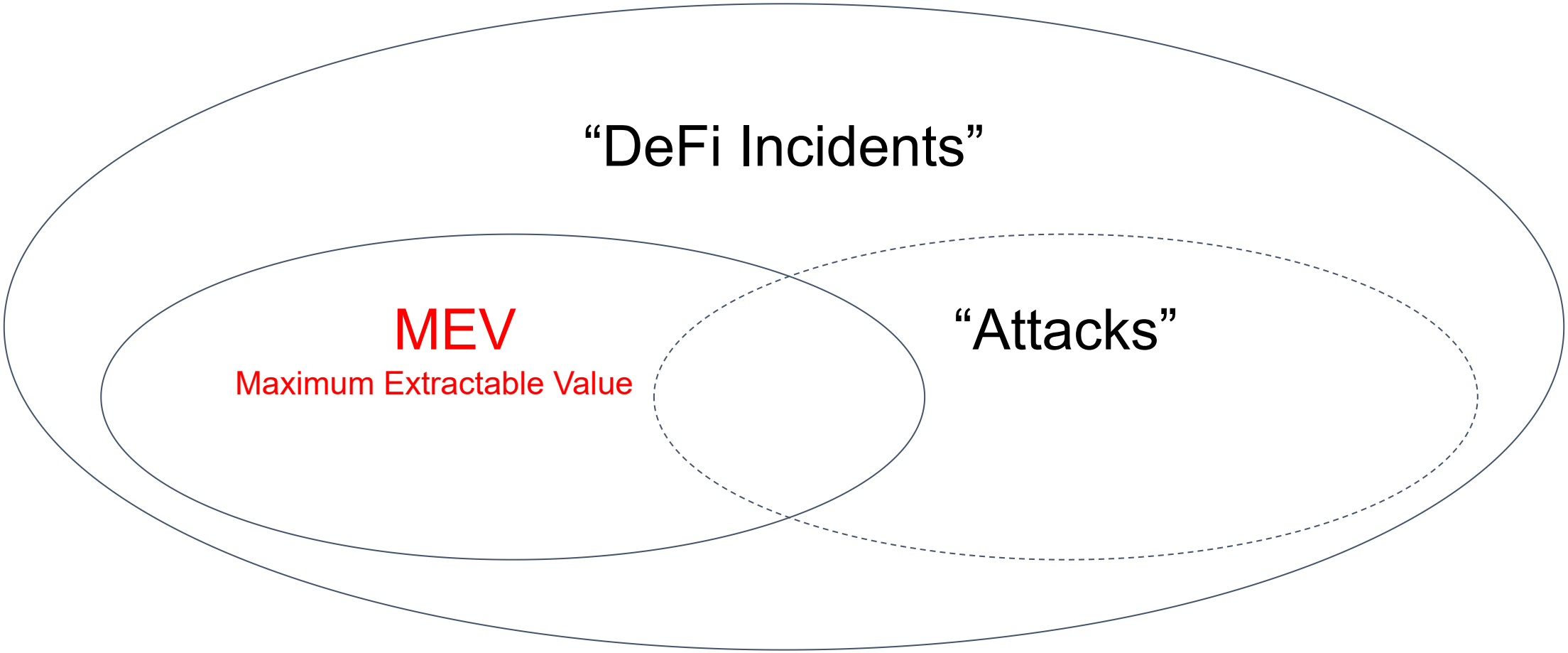
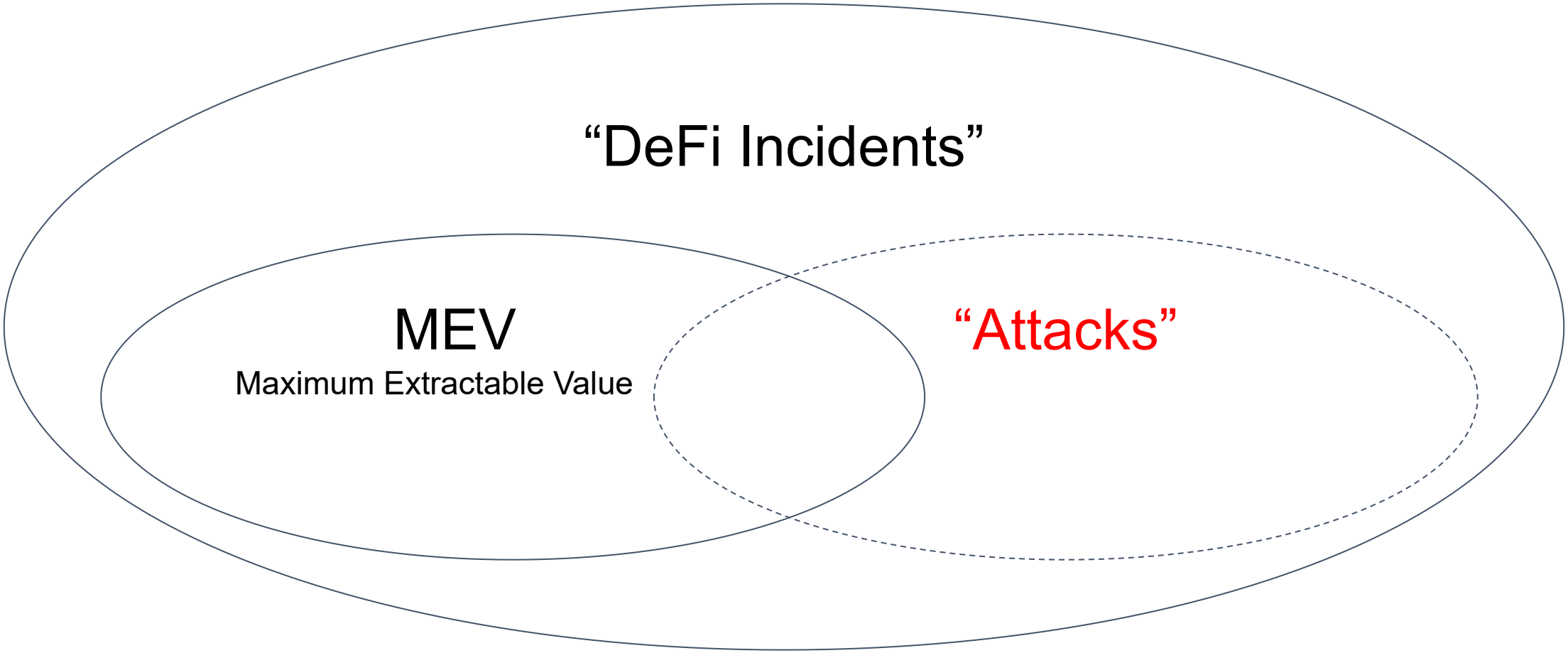
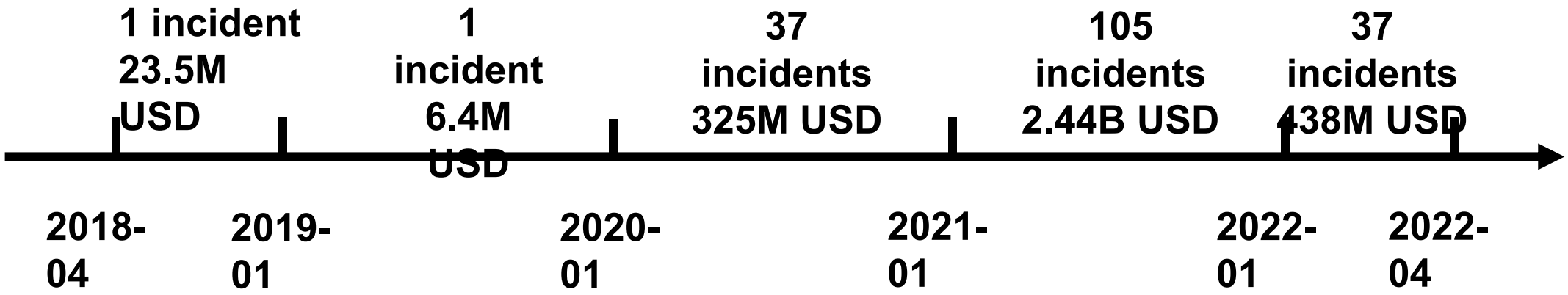
Lecture 08 Security Challenges in Blockchain


“DeFi Incidents”
MEV
Maximum Extractable Value
“Attacks”


“DeFi Incidents”
MEV
Maximum Extractable Value
“Attacks”


“DeFi Incidents”
MEV
Maximum Extractable Value
“Attacks”

DeFi Incidents on Ethereum & BSC



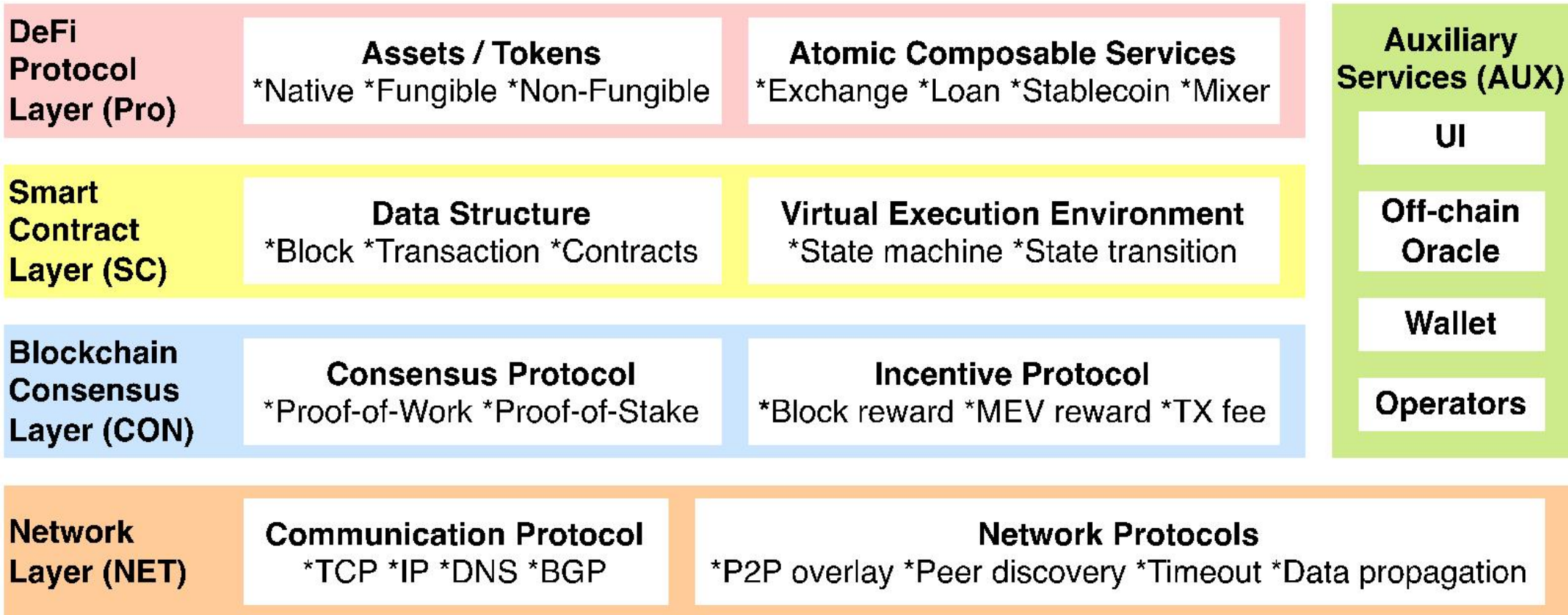


At least 181 DeFi “attacks”



3.24B USD losses

System Model



Attack Outline

Smart Contract Layer

1. Reentrancy attack
2. Delegatecall attack

DeFi Protocol Layer

1. On-chain oracle manipulation

Attack Outline

Smart Contract Layer

1. **Reentrancy attack**
2. Delegatecall attack

DeFi Protocol Layer

1. On-chain oracle manipulation

Smart Contract Layer: Reentrancy Attack

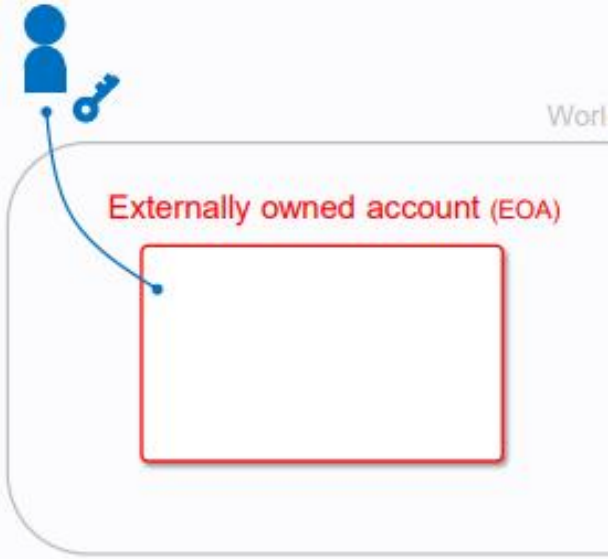
What Is Reentrancy Attacks

Reentrancy attack is an exploitation of smart contract logic where an attacker induces improper execution or state changes by recursively calling a contract's function, or calling another function before the first completes

- often to withdraw funds multiple times and deplete the contract's assets.

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack

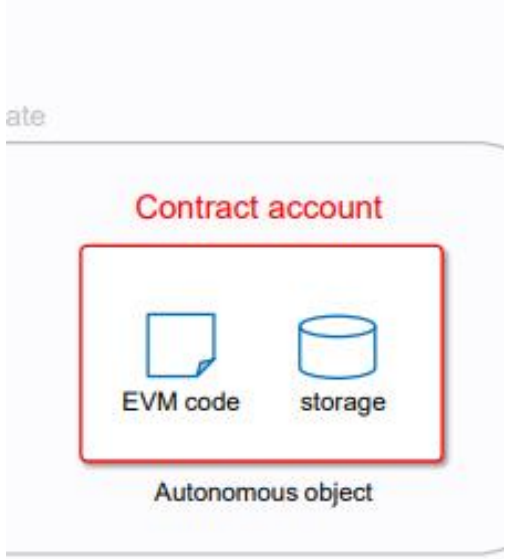


1. **Initiates a transaction that calls the *withdrawbalance* function**





User Need 10

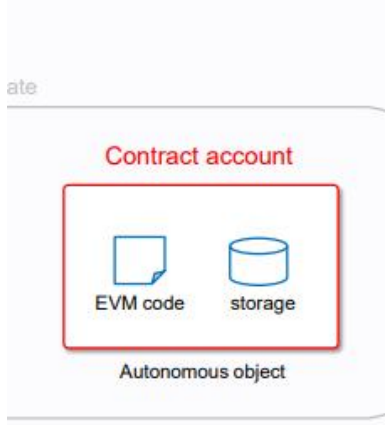




Contract Reserve: 20

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



2. Executes the *withdrawbalance* function



Contract Reserve: 20 – 10



User Balance: 10



User Balance: 10-10=0



User Balance: 0

1.1) Verify User Identity

- Ensure that the party calling the contract is authorized to make a withdrawal.

1.2) Check Balance

- Confirm that the user has an adequate balance available for withdrawal.

1.3) Perform Transfer

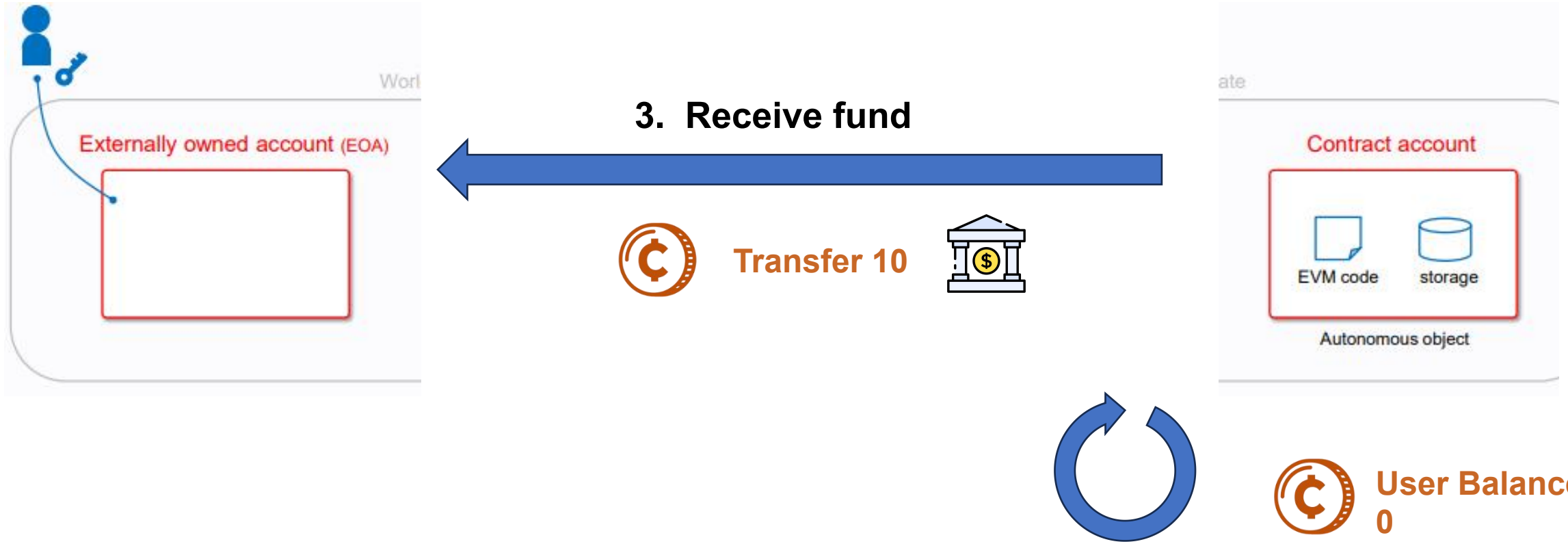
- Transfer the specified amount of funds from the contract to the user's address.

1.4) Update State

- Update the user's balance state within the contract to reflect the new balance after

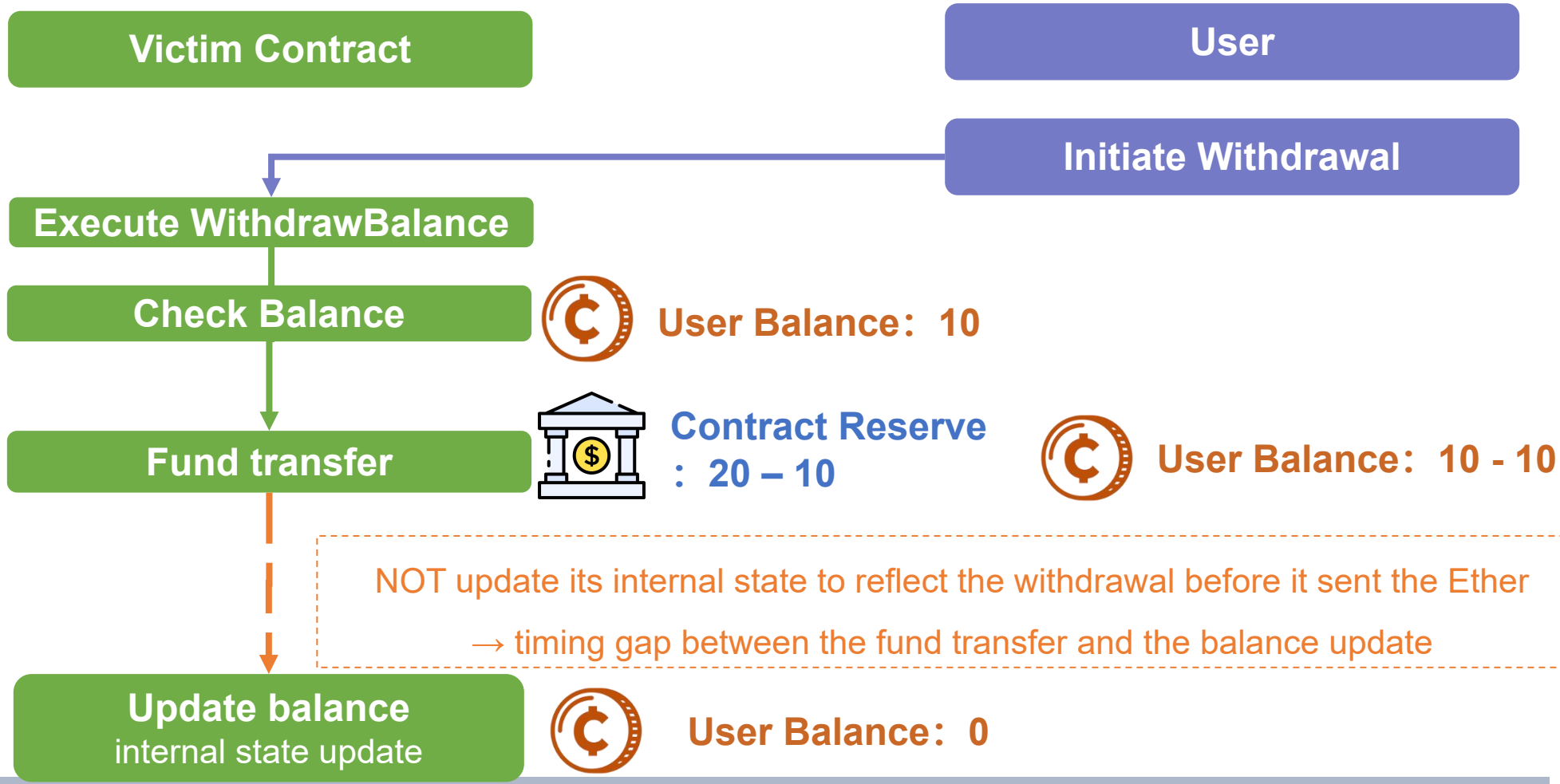
Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



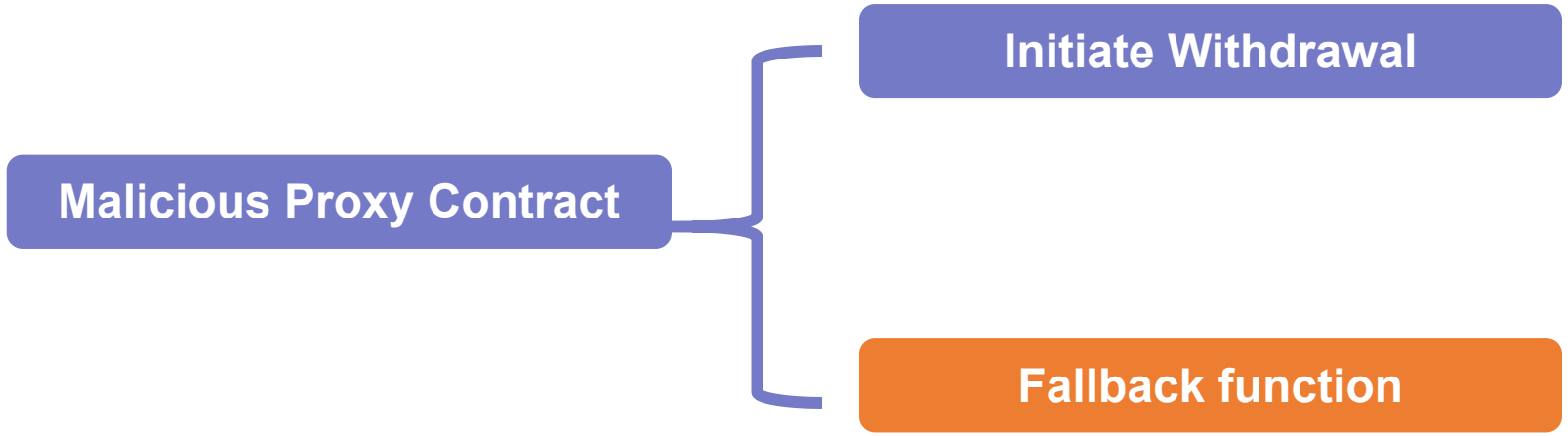
Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



According to the rules of Ethereum

This fallback function will be executed automatically

when the contract receives ether

Without any other function calls.

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve: 20

Victim Contract

Malicious Proxy Contract

Initiate the First transaction

Initiate Withdrawal

Execute WithdrawBalance



User Balance: 10

Check Balance

Fund transfer

Fallback function

Update balance
internal state update

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve: 20-10=10

Victim Contract

Malicious Proxy Contract

Initiate the First transaction

Initiate Withdrawal

Execute WithdrawBalance



User Balance: 10

Check Balance

Fund transfer

Fallback function

Update balance
internal state update



User Balance: 10 – 10=0 (waiting for update)

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve: 10

Victim Contract

Malicious Proxy Contract

Initiate the First transaction

Initiate Withdrawal

Execute WithdrawBalance



User Balance: 10

Check Balance

Fund transfer

Fallback function

Update balance
internal state update



User Balance: 10 – 10=0 (waiting for update)

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve: 10



User Balance: 10

Victim Contract

Malicious Proxy Contract

Initiate the Second transaction

Initiate Withdrawal

Execute WithdrawBalance

Check Balance

Fund transfer

Fallback function

Update balance
internal state update

Smart Contract Layer: Reentrancy Attack

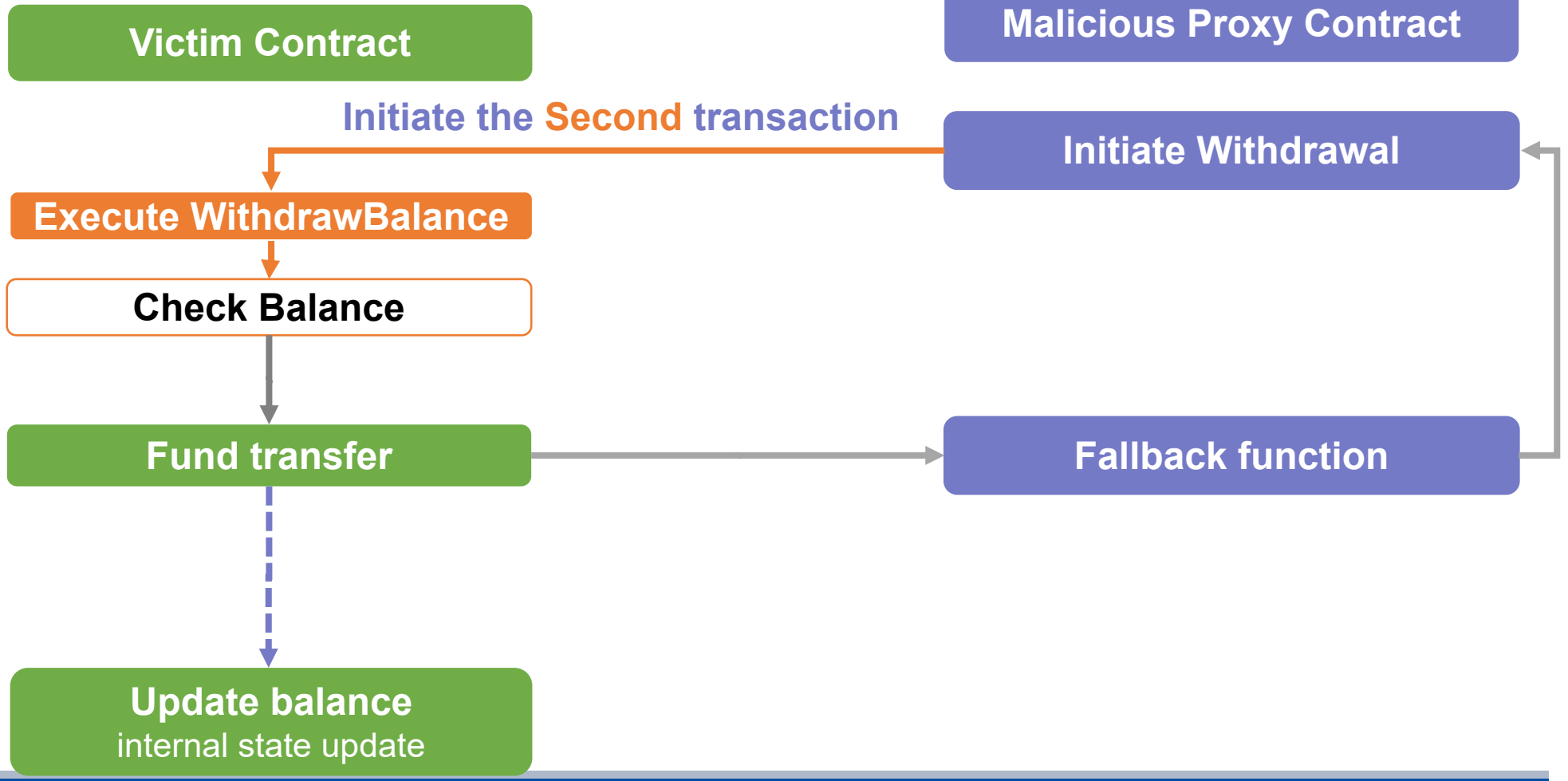
Example Flow of a Reentrancy Attack



Contract Reserve: 10



User Balance: 10



Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve: 10-10 =0

Victim Contract

Malicious Proxy Contract

Initiate the Second transaction

Initiate Withdrawal

Execute WithdrawBalance



User Balance: 10

Check Balance

Fund transfer

Fallback function

Update balance
internal state update



User Balance: 0 (Still not updated yet)

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve: 0

Victim Contract

Malicious Proxy Contract

Initiate the Second transaction

Initiate Withdrawal

Execute WithdrawBalance



User Balance: 10

Check Balance

Fund transfer

Fallback function

Update balance
internal state update



User Balance: 0 (Still not updated yet)

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve: 0



User Balance: 10

Victim Contract

Malicious Proxy Contract

Initiate the Third transaction

Initiate Withdrawal

Execute WithdrawBalance

Check Balance

Fund transfer

Fallback function

Update balance
internal state update



User Balance: 0 (Still not updated yet)

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve: 0

Victim Contract

Malicious Proxy Contract

Initiate the Third transaction

Initiate Withdrawal

Execute WithdrawBalance



User Balance: 10

Check Balance

Fund transfer

Fallback function

Update balance
internal state update



User Balance: 0 (Still not updated yet)

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve:
0 – 10 (failed)



User Balance:
10

Victim Contract

Malicious Proxy Contract

Initiate the Third transaction

Initiate Withdrawal

Execute WithdrawBalance

Check Balance

Fund transfer

Fallback function

Update balance
internal state update



User Balance: 0 (Still not updated yet)

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Contract Reserve:
0 – 10 (failed)

Victim Contract

Malicious Proxy Contract

Initiate the Third transaction

Initiate Withdrawal

Execute WithdrawBalance



User Balance:
10

Check Balance

Fund transfer

Fallback function

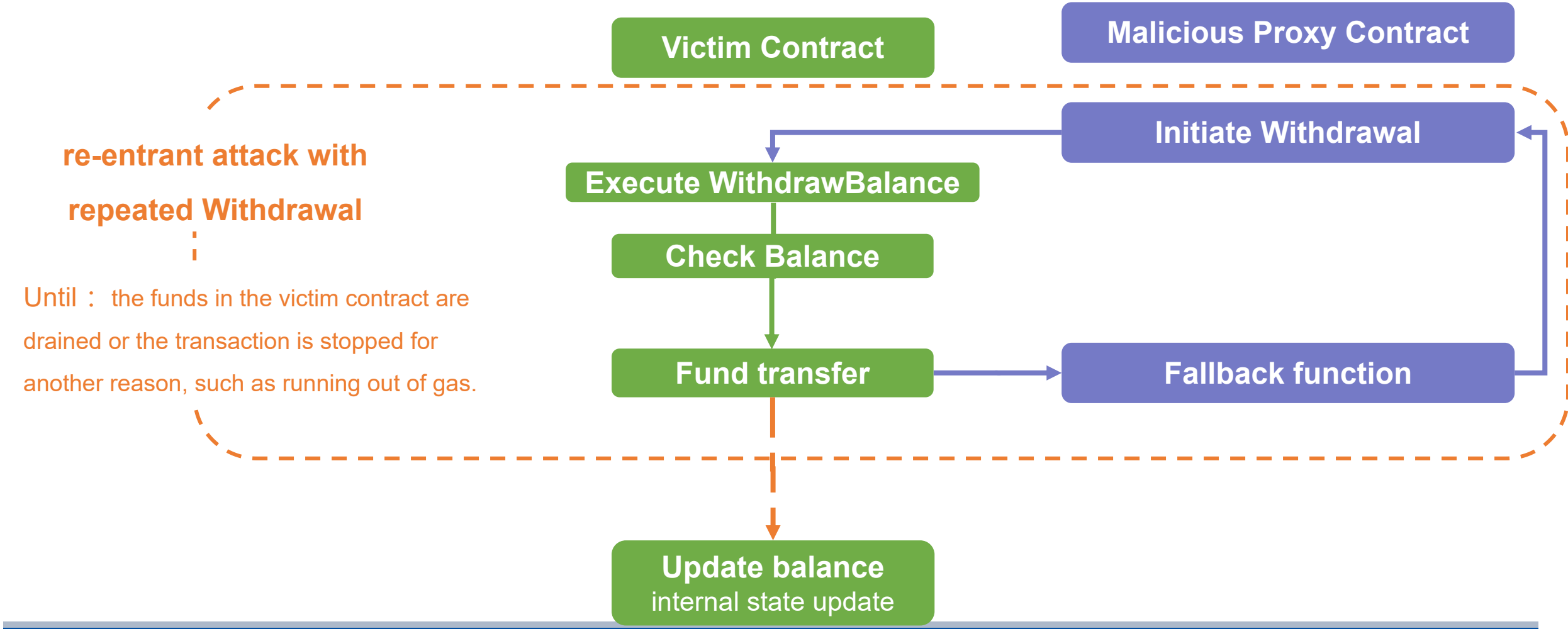
Update balance
internal state update



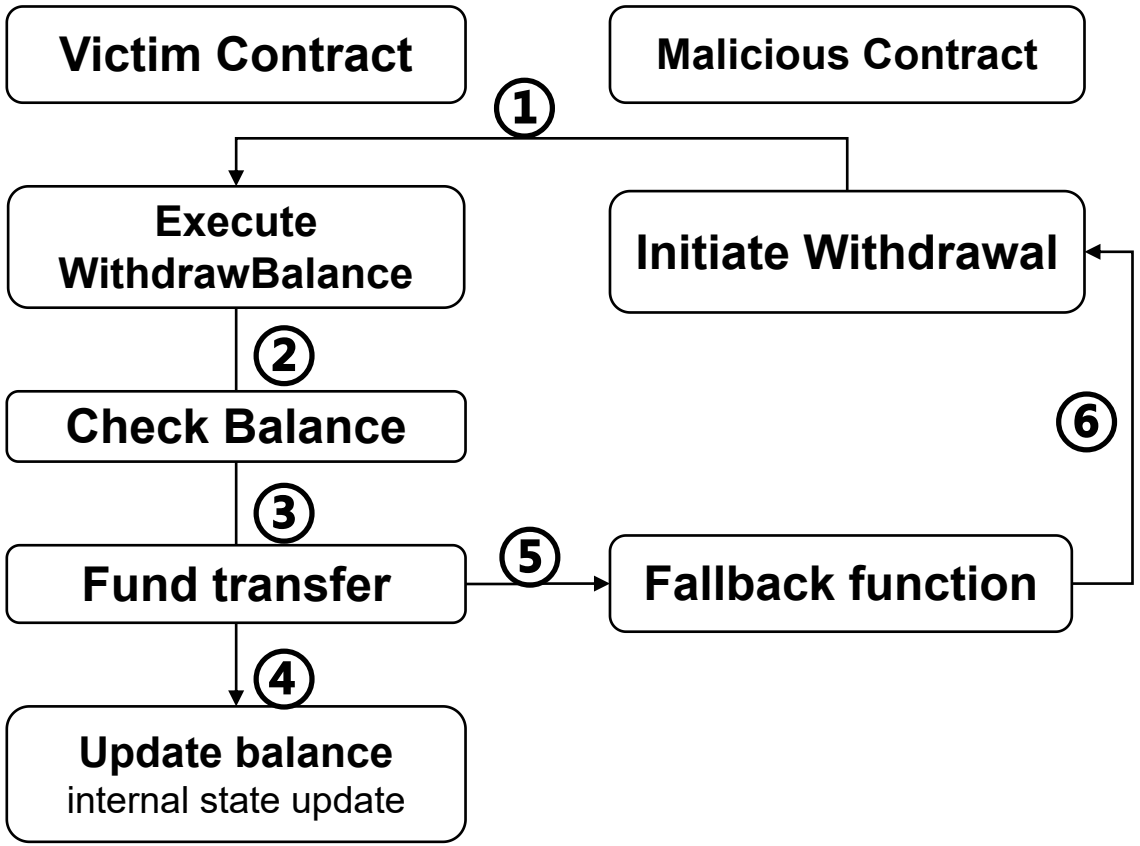
User Balance: 0 (Still not updated yet)

Smart Contract Layer: Reentrancy Attack

Example Flow of a Reentrancy Attack



Smart Contract Layer: Reentrancy Attack



Attack Outline

Smart Contract Layer

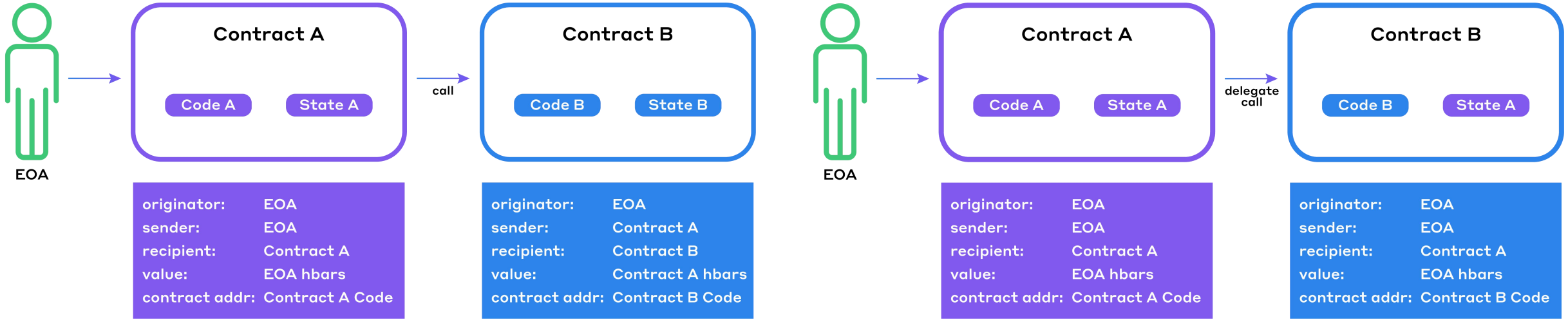
1. Reentrancy attack
2. **Delegatecall attack**

DeFi Protocol Layer

1. On-chain oracle manipulation

Smart Contract Layer: Delegatecall Attack

- **Smart contracts have two methods for making external calls:**

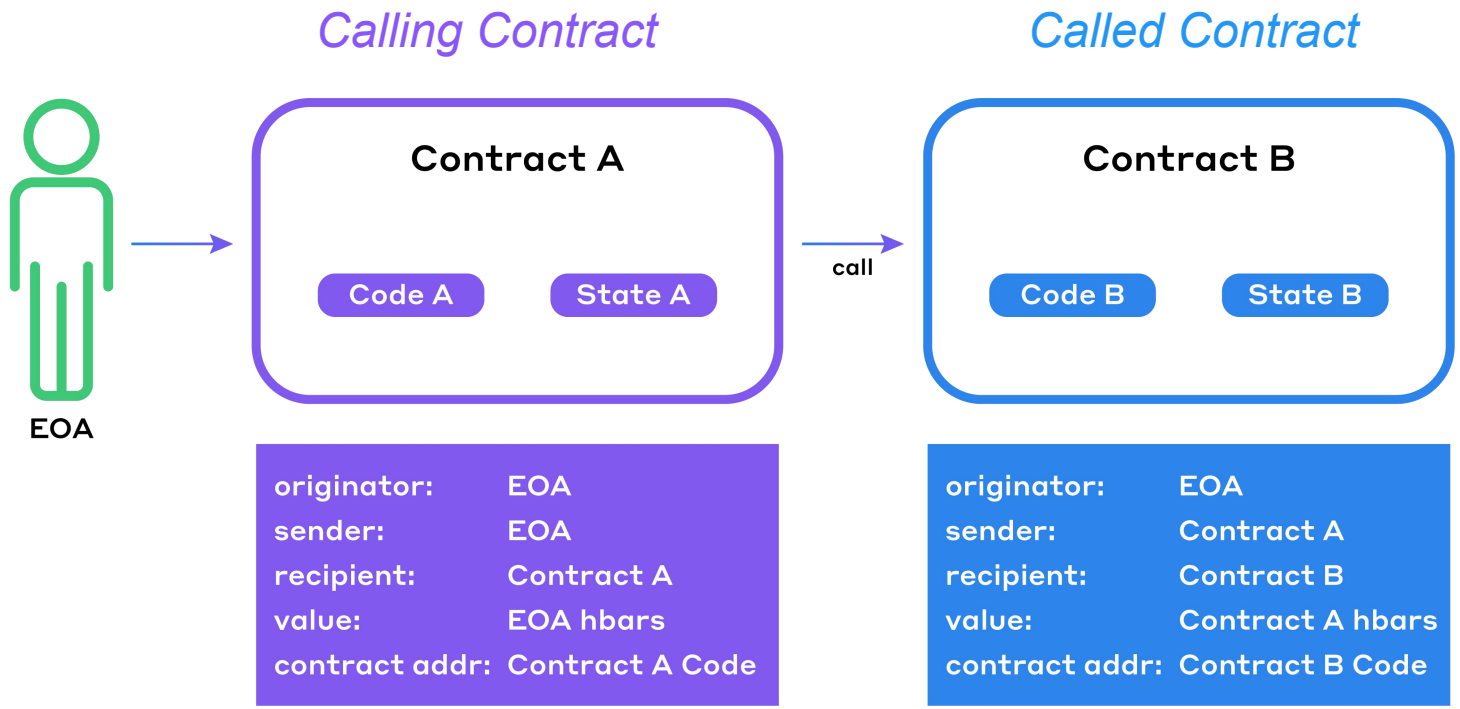


Call **Delegatecall**

Smart Contract Layer: Delegatecall Attack

Call

- **Execution Context:** Executes in the context of the called contract..
- **Effect on Called Contract:** Can change the state of the called contract.
- **Effect on Calling Contract:** Does not alter the calling contract's state or storage.
- **Use Case:** Suitable for general interactions and Ether transfers between contracts.



Smart Contract Layer: Delegatecall Attack

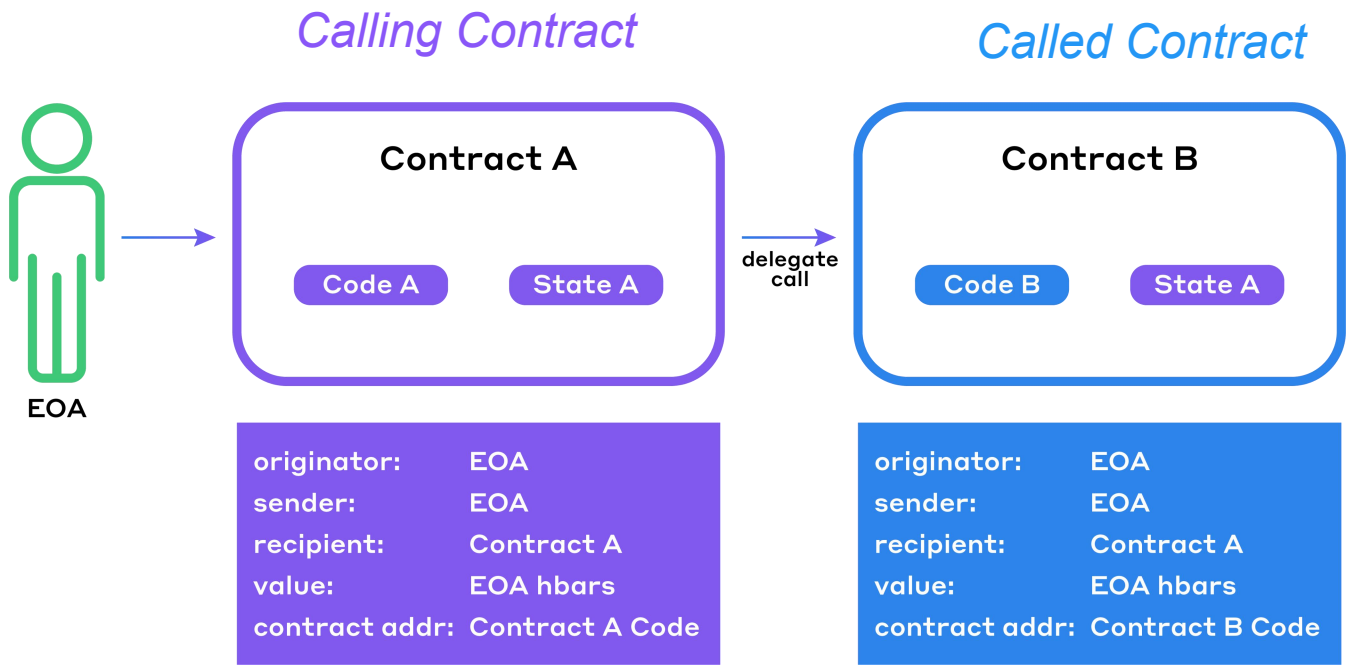
Delegate Call

Execution Context: Executes in the context of the calling contract, using its storage.

Effect on Called Contract: Code of the called contract runs as if it were part of the caller.

Effect on Calling Contract: Can modify the calling contract's state and storage.

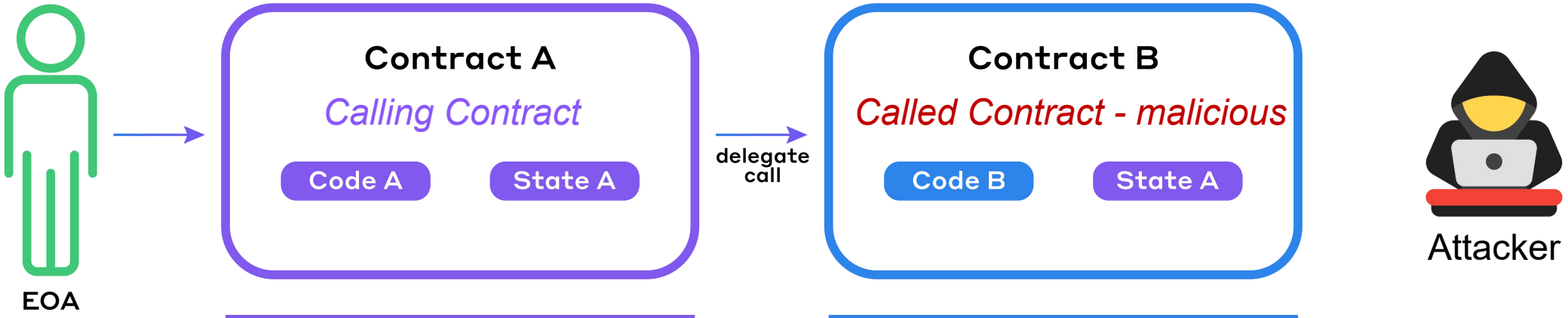
Use Case: Ideal for leveraging library code and upgrading contract logic.



Smart Contract Layer: Delegatecall Attack

As Delegate Call Can modify the calling contract's state and storage →

- A ***delegatecall attack*** is a security exploit in which an attacker execute unintended actions from called contract in a vulnerable calling smart contract, potentially leading to the alteration of the calling contract's state, theft of funds, or loss of contract control.



Smart Contract Layer: Delegatecall Attack

Example Flow of a Delegatecall Attack

```
1   contract Wallet{
2       function() payable { //fallback function
3       if (msg.value > 0)
4           Deposit(msg.sender, msg.value);
5       else if (msg.data.length > 0)
6           _walletLibrary.delegatecall(msg.data);
7       }
8   }
9   contract WalletLibrary {
10  function initWallet(address[] _owners, uint _required, uint
    _daylimit) {
11      initDaylimit(_daylimit);
12      initMultiowned(_owners, _required);
13      }
14  }
```

Calling Contract : *Wallet*

- Acts as a user's personal wallet on the Ethereum blockchain.

Called Contract : *WalletLibrary*

- Provides reusable logic for wallet functionalities to multiple wallet instances
- .Core Function*: initWallet*

Smart Contract Layer: Delegatecall Attack

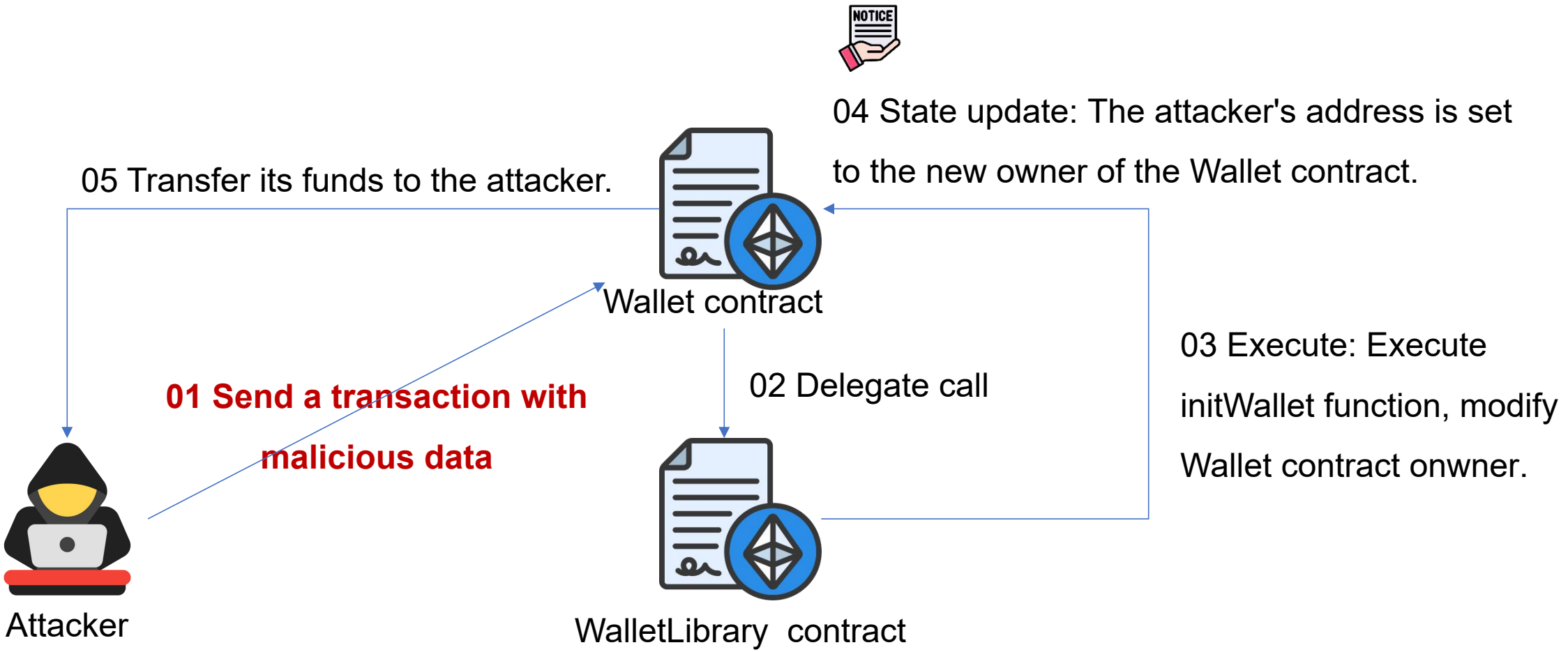
Example Flow of a Delegatecall Attack

```
1   contract Wallet{
2       function() payable { //fallback function
3       if (msg.value > 0)
4           Deposit(msg.sender, msg.value);
5       else if (msg.data.length > 0)
6           _walletLibrary.delegatecall(msg.data);
7       }
8   }
9   contract WalletLibrary {
10  function initWallet(address[] _owners, uint _required, uint
    _daylimit) {
11      initDaylimit(_daylimit);
12      initMultiowned(_owners, _required);
13      }
14  }
```

Role of *delegatecall*

- **Function Execution:** Allows *Wallet* contract to execute code and use functions from *WalletLibrary* contract.
- **State Management:** When *Wallet* executes *delegatecall*, it runs fundction *initWallet* of *WalletLibrary's* code in its own storage context, can Initializes wallet settings
- **Upgradeability:** Facilitates the upgrade of contract logic without altering the state or redeploying the *Wallet*.

Smart Contract Layer: Delegatecall Attack



Smart Contract Layer: Delegatecall Attack

Example Flow of a Delegatecall Attack

```
1  contract Wallet{
2      function() payable { //fallback function
3      if (msg.value > 0)
4          Deposit(msg.sender, msg.value);
5      else if (msg.data.length > 0)
6          _walletLibrary.delegatecall(msg.data);
7      }
```



Wallet contract

a) **Transaction Receipt:** The Ethereum network receives the transaction intended for the Wallet contract

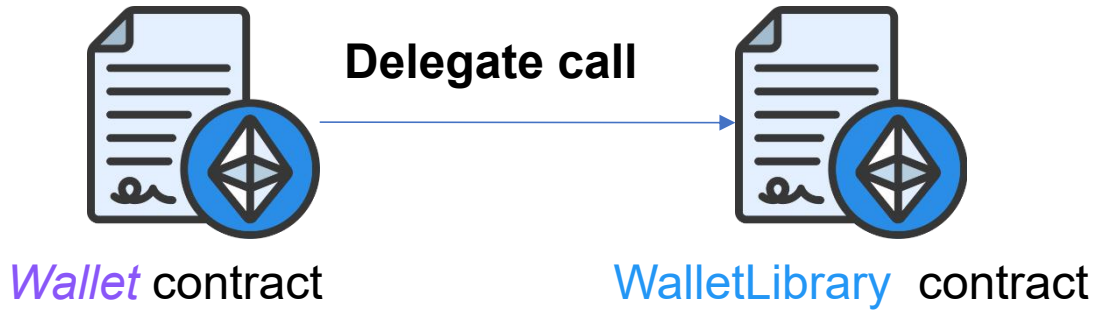
b) **Fallback Function Trigger:** Since the transaction data does not match any of the function signatures in the Wallet contract, the fallback function is triggered.

- In Ethereum, when a contract receives a transaction that doesn't match any function or if it was sent with just Ether and extra data, the fallback function is executed.

Smart Contract Layer: Delegatecall Attack

Example Flow of a Delegatecall Attack

```
1  contract Wallet{
2      function() payable { //fallback function
3      if (msg.value > 0)
4          Deposit(msg.sender, msg.value);
5      else if (msg.data.length > 0)
6          _walletLibrary.delegatecall(msg.data);
7      }
```

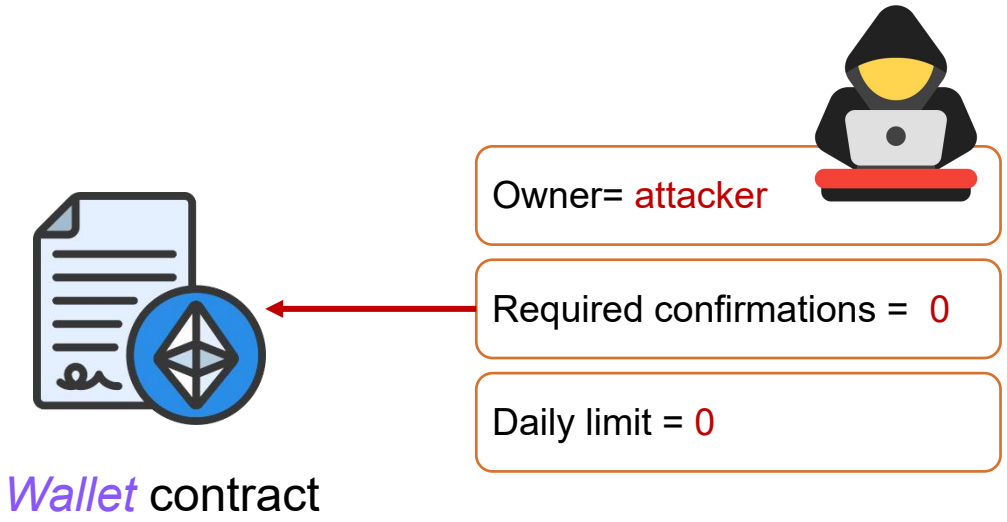


c) Delegatecall Execution: The fallback function, as written, has a condition that checks for the presence of data (msg.data.length > 0). Since the malicious msg.data is present, the fallback function executes the line:*_walletLibrary.delegatecall(msg.data);*

Smart Contract Layer: Delegatecall Attack

Example Flow of a Delegatecall Attack

```
9   contract WalletLibrary {
10  function initWallet(address[] _owners, uint _required, uint
    _daylimit) {
11      initDaylimit(_daylimit);
12      initMultiowned(_owners, _required);
13  }
```



d) Code Execution in Context: The EVM then executes the *initWallet* using the *Wallet* contract's storage. This means any changes that *initWallet* makes are applied to the *Wallet* contract's storage variables.

e) State Alteration: If the initWallet function in the WalletLibrary includes code to set ownership, then the attacker's address will be set as the owner in the Wallet contract's storage. This is because delegatecall runs the initWallet code as if it were part of the Wallet contract itself.

Smart Contract Layer: Delegatecall Attack

Example Flow of a Delegatecall Attack



Attack Outline

Smart Contract Layer

1. Reentrancy attack
2. Delegatecall attack

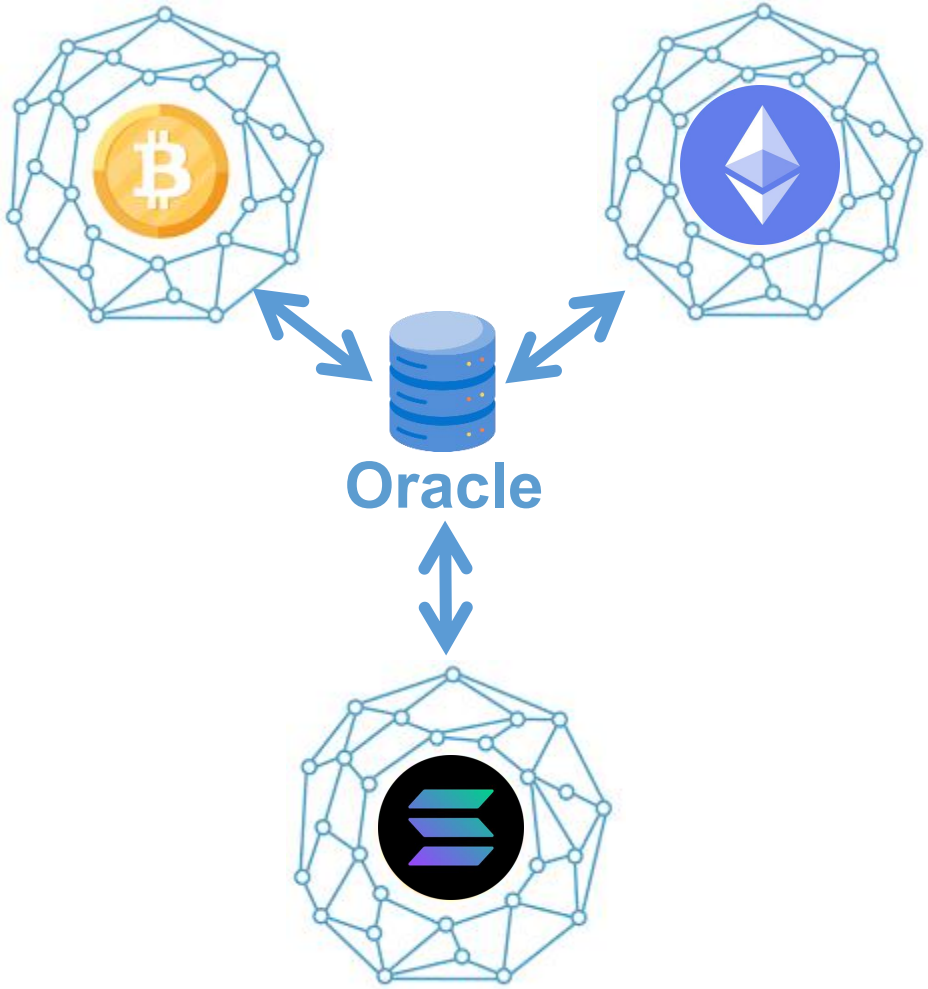
DeFi Protocol Layer

1. **On-chain oracle manipulation**

Defi Protocol Layer: On-chain Oracle Manipulation

What is On-chain Oracle?

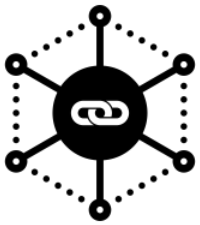
- Oracles are third-party services that feed external data to smart contracts on the blockchain.
- **Necessity for Oracles:**
  - **Isolated Networks:** Blockchains are isolated ecosystems, inherently unable to access external off-chain data.
  - **Market Data Requirements:** DeFi and other financial blockchain applications require real-time market data from various sources.



Defi Protocol Layer: On-chain Oracle Manipulation

What is On-chain Oracle Manipulation?

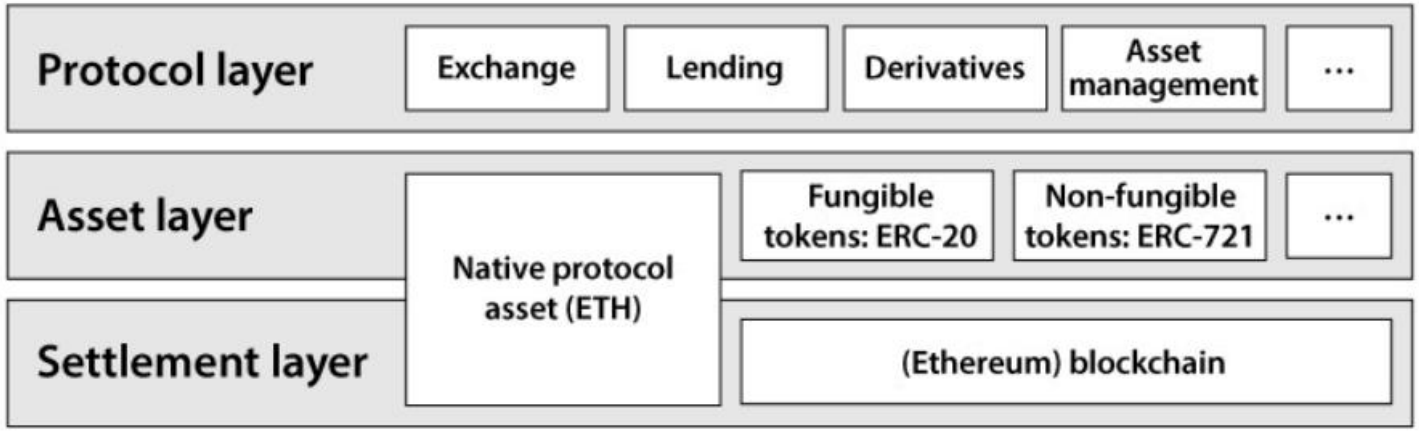
- On-chain Oracle Manipulation refers to the deliberate act of influencing or altering the data provided by on-chain oracles in a blockchain network.



Oracle



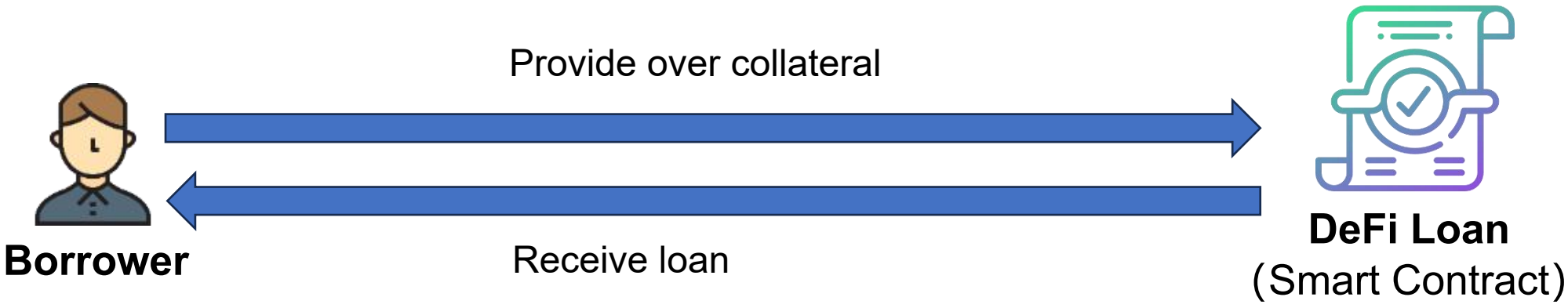




Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

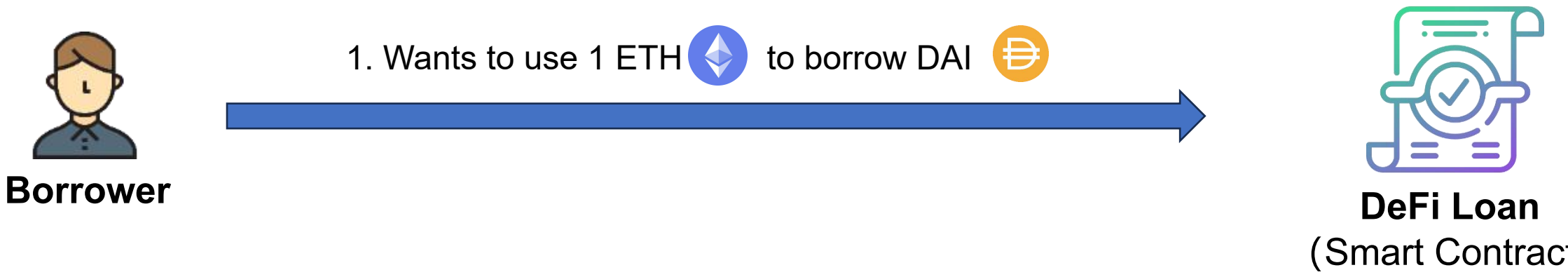
- On-chain Oracle Manipulation refers to the deliberate act of influencing or altering the data provided by on-chain oracles in a blockchain network.



Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- Normal Case: Alice wants to use 1 ETH to borrow DAI



Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- Normal Case: Alice wants to use 1 ETH to borrow DAI

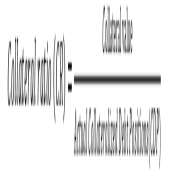
2. Calculate loan amount according to Collateral ratio (CR)



DeFi Loan
(Smart Contract)

$$Collateral\ ratio\ (CR) = \frac{\text{Collateral value}}{Actual\ Collateralized\ Debt\ Positions(CDP)}$$

$$= \frac{current\ market\ value\ of\ the\ ETH}{the\ actual\ amount\ of\ DAI\ lent}$$



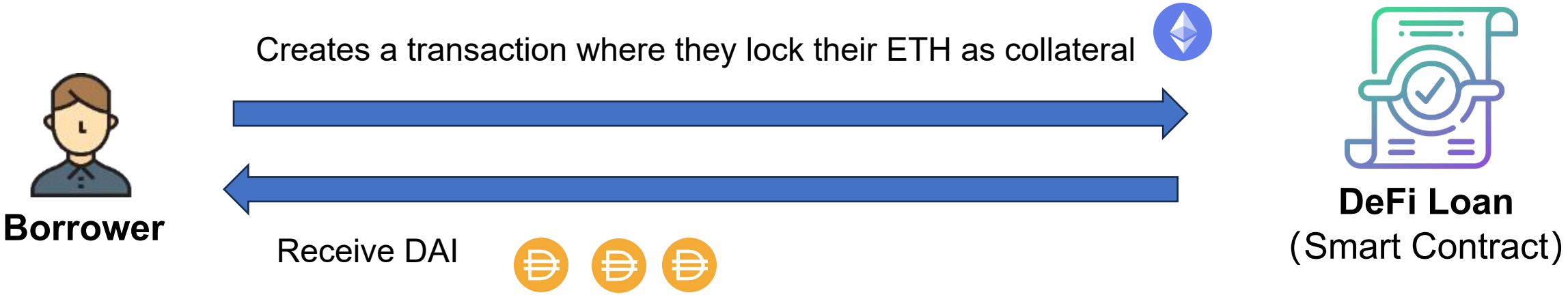
Determined by the price feed from oracles

Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- Normal Case: Alice wants to use 1 ETH to borrow DAI

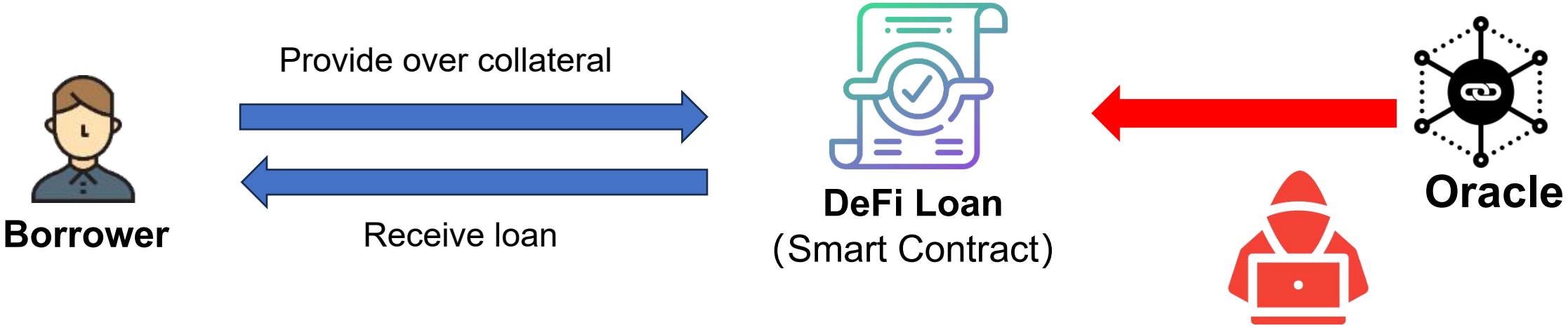
3. Loan Issuance



Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

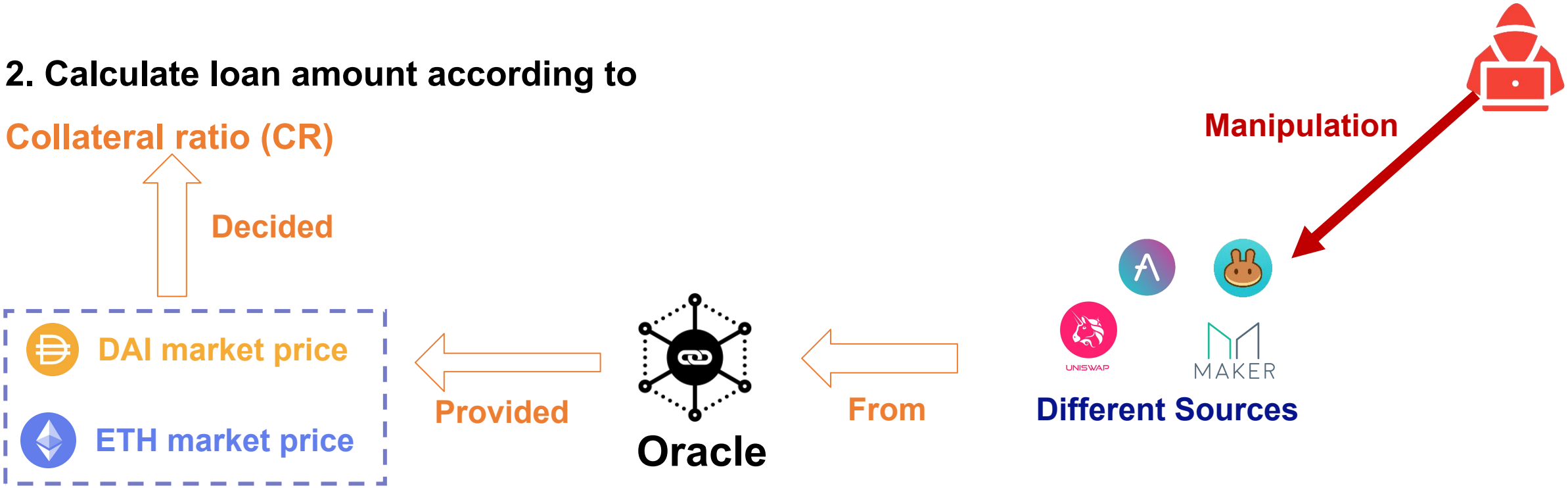
- On-chain Oracle Manipulation： Due to the attacker‘s control, the oracle obtained and provided the wrong ETH market price from the DEX to the lending platform, resulting in wrong CR



Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- On-chain Oracle Manipulation： Due to the attacker‘s control, the oracle obtained and provided the wrong ETH market price from the DEX to the lending platform, resulting in wrong CR



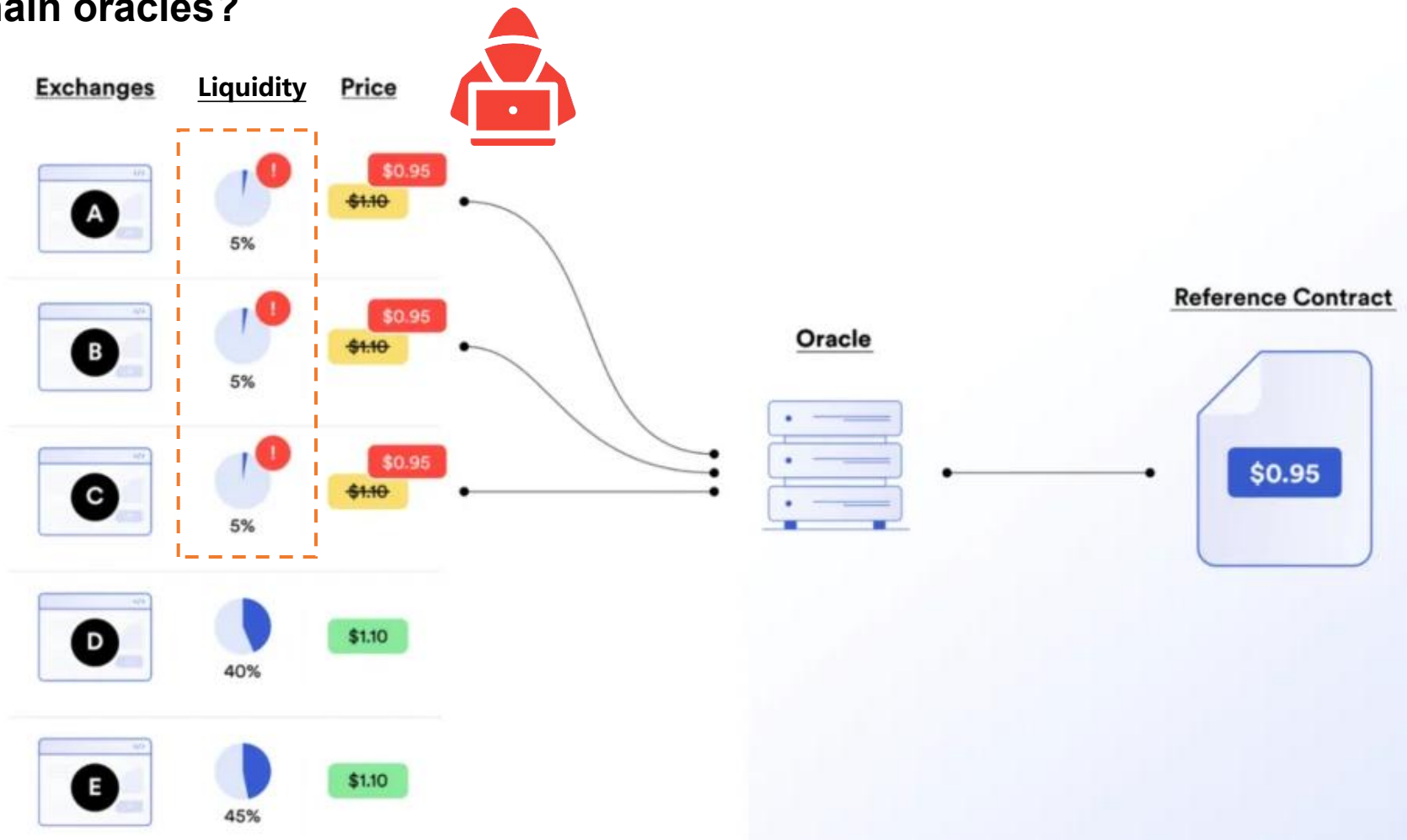
Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- **How attackers manipulate on-chain oracles?**

1) Targeting Vulnerable DEXs

- Attackers target DEXs with lower liquidity for manipulation.
- Lower liquidity means higher susceptibility to price changes.
- Example: DEX A,B, and C are chosen.



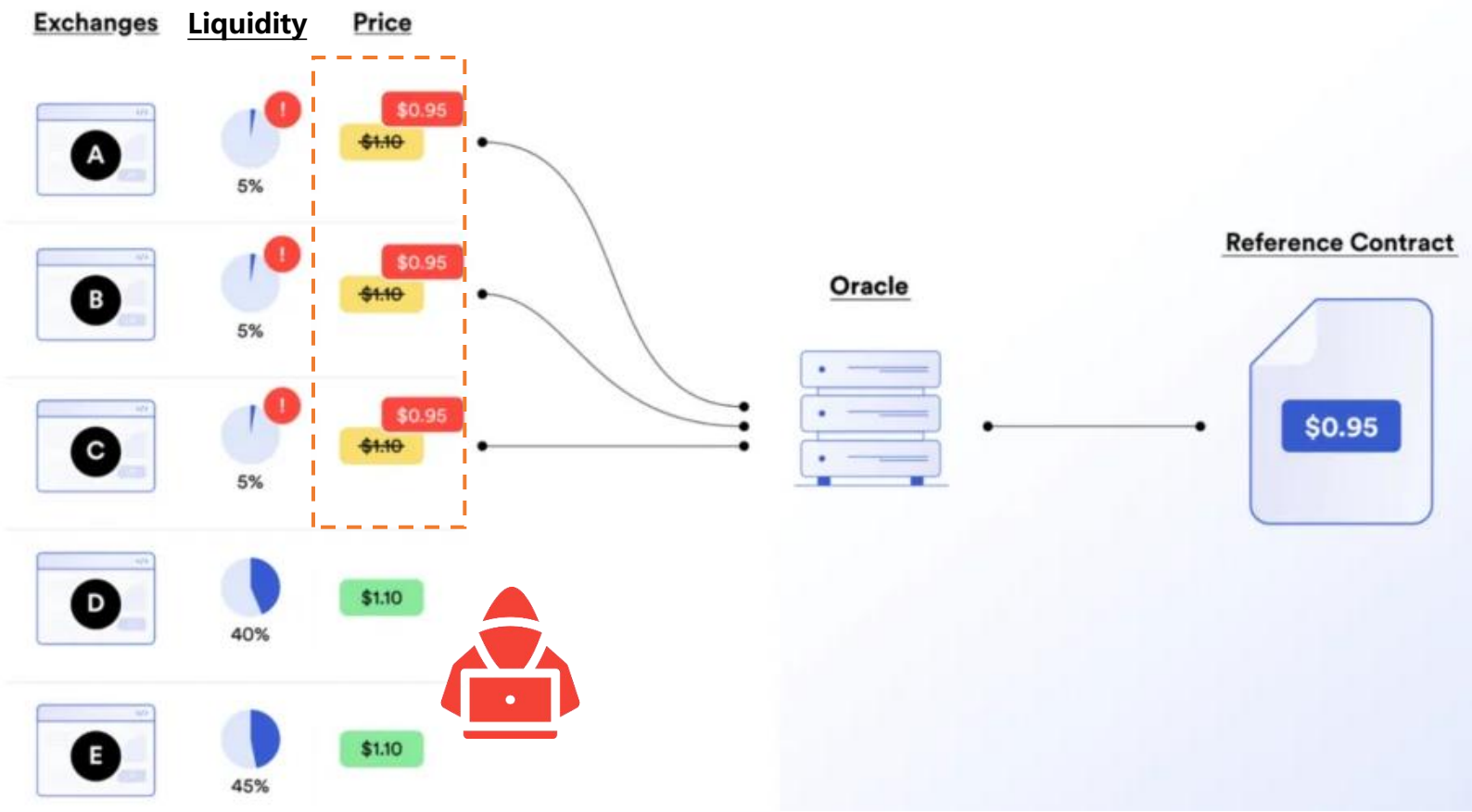
Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- **How attackers manipulate on-chain oracles?**

2) The Attacker Perform Large Trades

- Low liquidity means fewer buy and sell orders at each price point.
- A large order can exhaust available orders, pushing the price up or down.
- Example: A substantial buy order for DAI with ETH can quickly consume all sell orders for DAI at lower prices, forcing the price upward.——Constant Product (CP) AMM



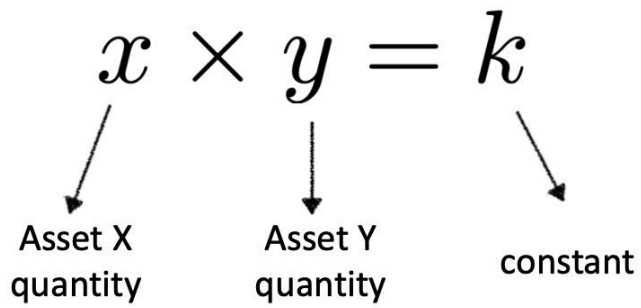
Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- **How attackers manipulate on-chain oracles?**

2) The Attacker perform large trades

- Example: A substantial buy order for DAI with ETH can quickly consume all sell orders for DAI at lower prices, forcing the price upward.——Constant Product (CP) AMM



CPAMM Properties:

- Instant liquidity, irrespective of the trade size.
- Purchase of asset X increases price of X and decreases the price of Y.
- Ratio of asset X and Y sets the price.

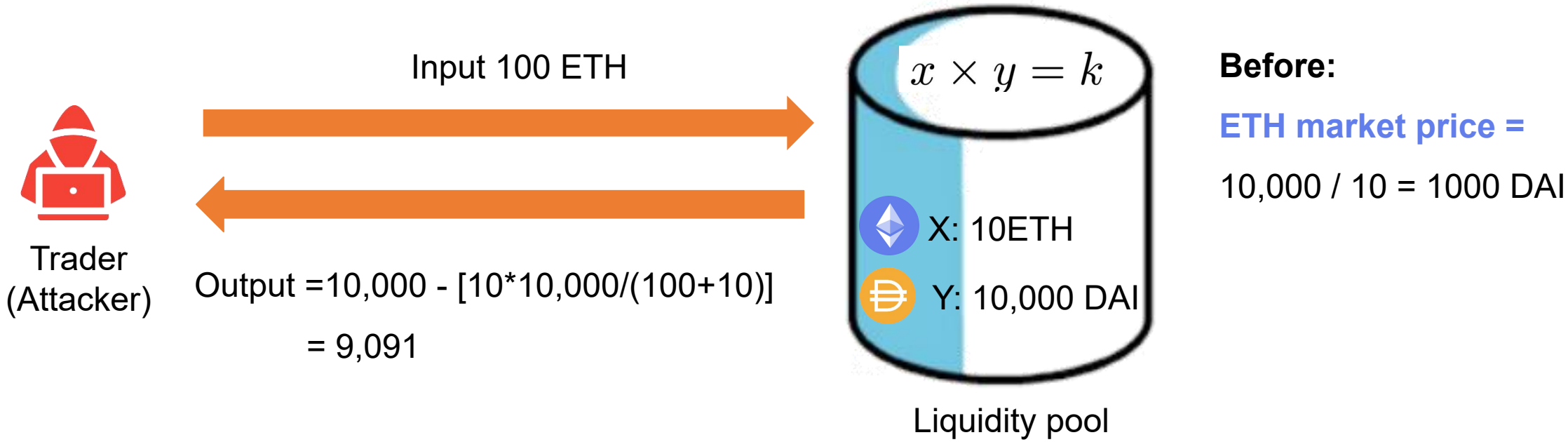
Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- **How attackers manipulate on-chain oracles?**

2) The Attacker perform large trades

- Example: A substantial buy order for DAI with ETH can quickly consume all sell orders for DAI at lower prices, forcing the price upward.——Constant Product (CP) AMM



Before:

ETH market price =

10,000 / 10 = 1000 DAI

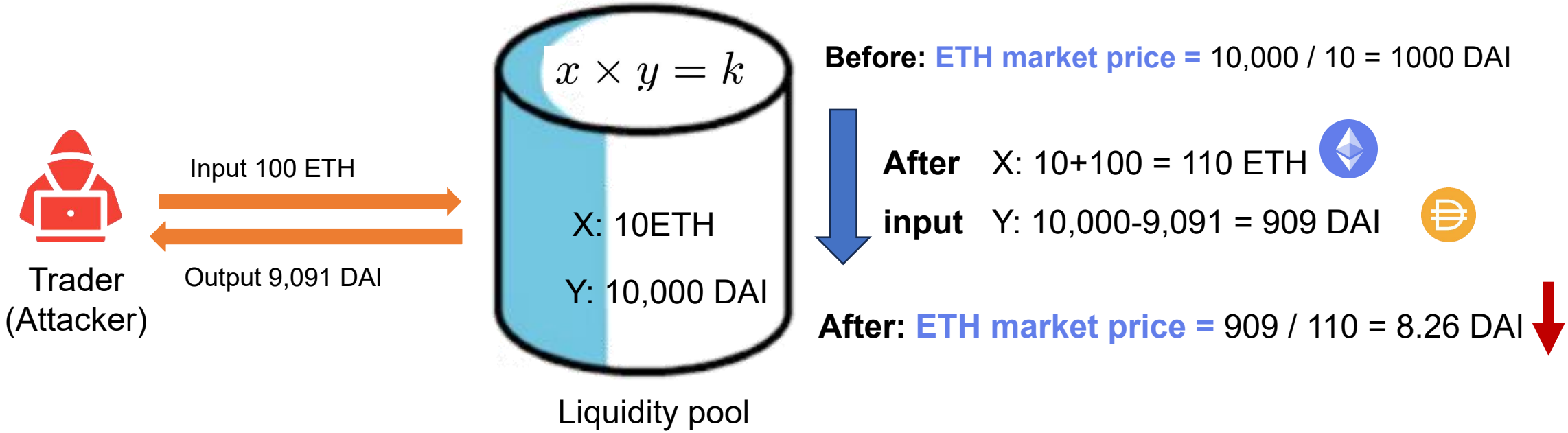
Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- **How attackers manipulate on-chain oracles?**

2) The Attacker perform large trades

- Example: A substantial buy order for DAI with ETH can quickly consume all sell orders for DAI at lower prices, forcing the price upward.——Constant Product (CP) AMM



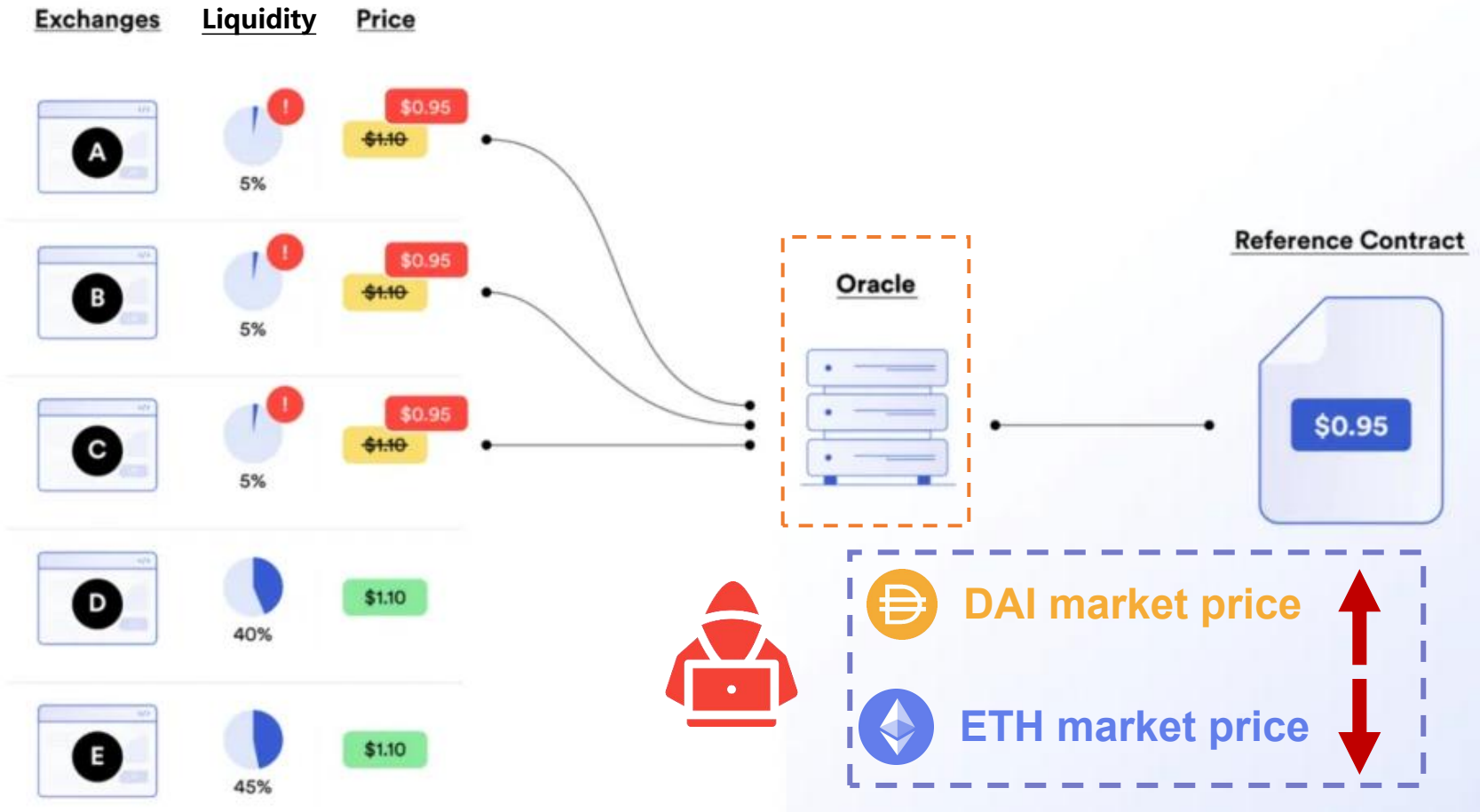
Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- **How attackers manipulate on-chain oracles?**

3) Impact of Trades on Oracle Data

- Large trades skew the oracle's data.
- Example: Oracle now reports a lower ETH price due to the attacker's actions.



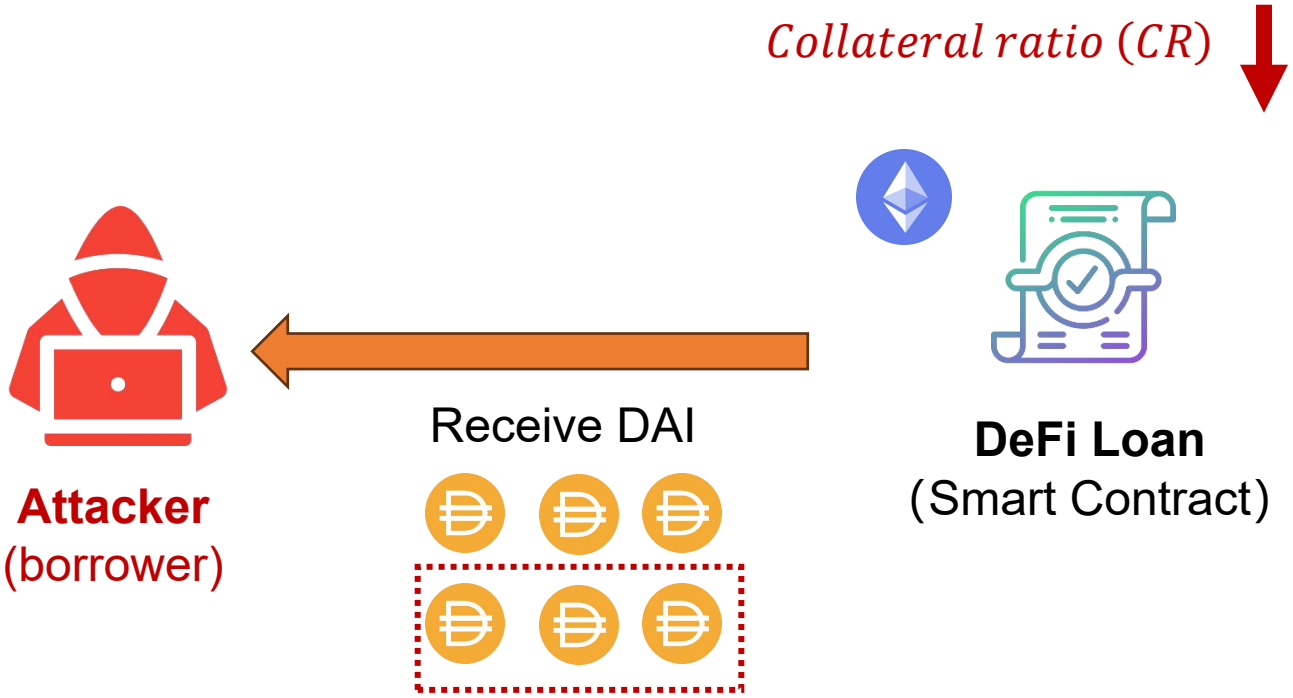
Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

- **How attackers manipulate on-chain oracles?**

4) Manipulated Prices and Contract Exposure

- Manipulated oracle data affects reference contracts, leading to inaccurate ETH pricing.
- Mispriced reference contracts can issue excessive DAI against ETH collateral.
- Attackers exploit this by borrowing more DAI than the actual value of the collateral.



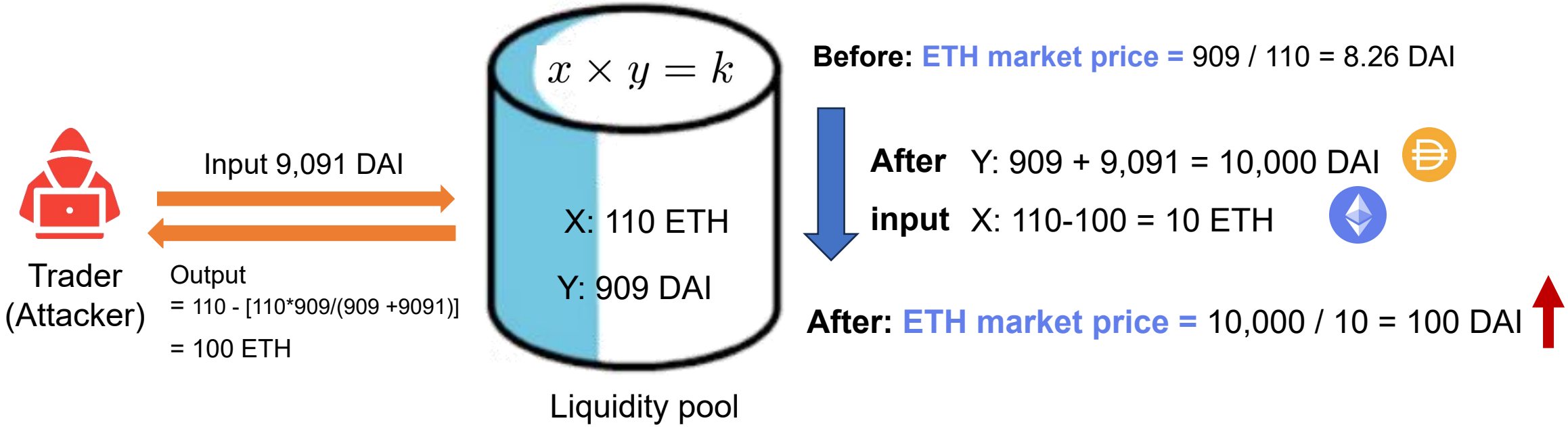
Defi Protocol Layer: On-chain Oracle Manipulation

Example Flow of a On-chain Oracle Manipulation

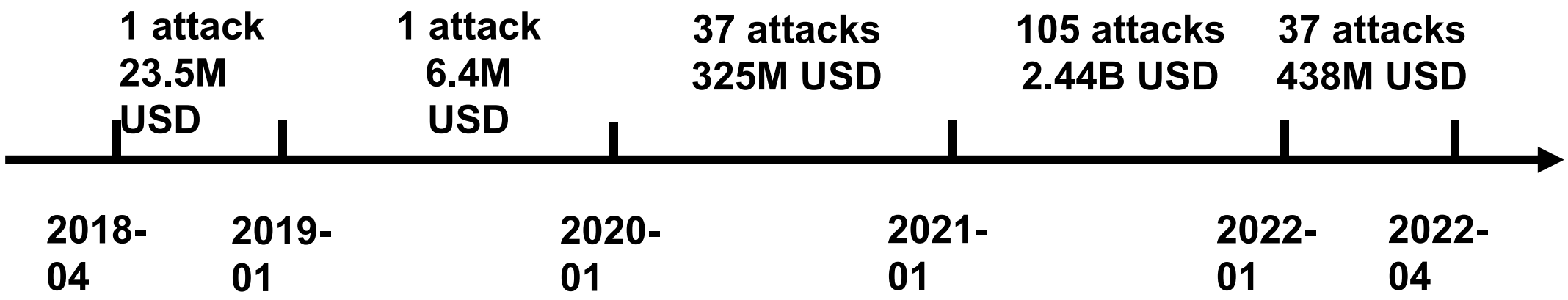
- **How attackers manipulate on-chain oracles?**

5) Attacker Rebalances Liquidity Pool

- Example: The attacker utilizes will reinvest money in the liquidity pool, in order to restore normal market prices and prevent potential losses due to the excessively low ETH price.——Constant Product (CP) AMM



DeFi Attacks on Ethereum & BSC





At least 181 DeFi “attacks”



3.24B USD losses

RQ1: How Can We Systematically Measure Defi Attacks?

RQ2: How Can We Learn From Past Attacks to Strengthen Defi Security?

Research Questions



RQ1: How Can We Systematically Measure Defi Attacks?

1. **Protocol Type Analysis**
2. **Analysis of “Emergency Pause”**

Attacks vs. Protocol Types

	Yield	Bridge	Lending	DEX	Stablecoin	DAO	Payment	Derivatives	Insurance	Others
Is the monetary loss related to the type of the DeFi protocol?										
Loss (in M USD)	868	860	485	450	286	200	72	32	14	713
Pct. of Total Loss	22%	22%	13%	12%	7%	5%	2%	1%	0%	18%
Is the number of the security incidents related to the type of the DeFi protocol?										
Num. of Incidents	50	10	22	28	7	7	7	6	3	49
Pct. of Incidents	27%	5%	12%	15%	4%	4%	4%	3%	2%	27%
TVL (in B USD)	9.2	11.4	18.2	27.7	-	-	0.5	2.2	0.6	-
Is the vulnerability type related to the type of the DeFi protocol?										
SC layer related	48%	60%	50%	39%	43%	0%	0%	50%	33%	43%
AUX layer related	20%	30%	18%	29%	0%	43%	71%	50%	33%	47%
PRO layer related	52%	10%	59%	39%	86%	29%	43%	17%	33%	24%
NET layer related	0%	0%	5%	4%	0%	14%	0%	0%	0%	2%

- Loss (in million USD) and frequency of DeFi incidents grouped by application type

Attacks vs. Protocol Types

	Yield	Bridge	Lending	DEX	Stablecoin	DAO	Payment	Derivatives	Insurance	Others
Is the monetary loss related to the type of the DeFi protocol?										
Loss (in M USD)	868	860	485	450	286	200	72	32	14	713
Pct. of Total Loss	22%	22%	13%	12%	7%	5%	2%	1%	0%	18%
Is the number of the security incidents related to the type of the DeFi protocol?										
Num. of Incidents	50	10	22	28	7	7	7	6	3	49
Pct. of Incidents	27%	5%	12%	15%	4%	4%	4%	3%	2%	27%
TVL (in B USD)	9.2	11.4	18.2	27.7	-	-	0.5	2.2	0.6	-
Is the vulnerability type related to the type of the DeFi protocol?										
SC layer related	48%	60%	50%	39%	43%	0%	0%	50%	33%	43%
AUX layer related	20%	30%	18%	29%	0%	43%	71%	50%	33%	47%
PRO layer related	52%	10%	59%	39%	86%	29%	43%	17%	33%	24%
NET layer related	0%	0%	5%	4%	0%	14%	0%	0%	0%	2%

- Loss (in million USD) and frequency of DeFi incidents grouped by application type

The Time to Pause A Protocol..

Duration after the incident starts	$\leq 1h$	$\leq 6h$	$\leq 12h$	$\leq 24h$	$\leq 48h$
Number of protocols	1	24	11	7	8
Percentage (out of 87 protocols)	2.%	47%	22%	14%	16%

- **87/181** Protocols have an emergency pause mechanisms.
- **25/87** Protocols initiated emergency pause within the first 6 hrs of the attack.

The Time to Pause A Protocol..

Duration after the incident starts	$\leq 1h$	$\leq 6h$	$\leq 12h$	$\leq 24h$	$\leq 48h$
Number of protocols	1	24	11	7	8
Percentage (out of 87 protocols)	2.%	47%	22%	14%	16%

- **87/181** protocols have an emergency pause mechanisms.
- **25/87** protocols initiated emergency pause within the first 6 hrs of the attack.



Can We Develop Intrusion Detection Tools to Automatically Trigger Emergency Pauses?

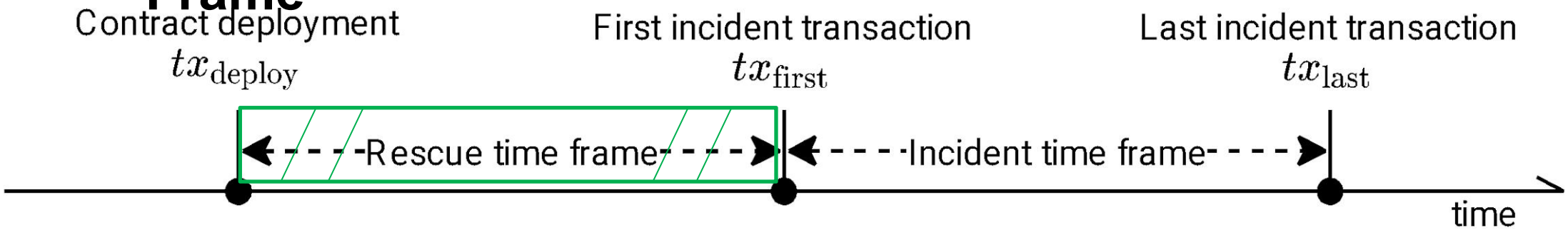
Research Questions



RQ2: What Can We Learn From Past Attacks to Strengthen Defi Security?

1. **Rescue Time Analysis**
2. **Bytecode Similarity**
3. **How Many Attacks Go Through A Relay?**
4. **Where Does The Money Go?**

Rescue and Incident Time Frame



1. Deploy an adversarial SC before attack

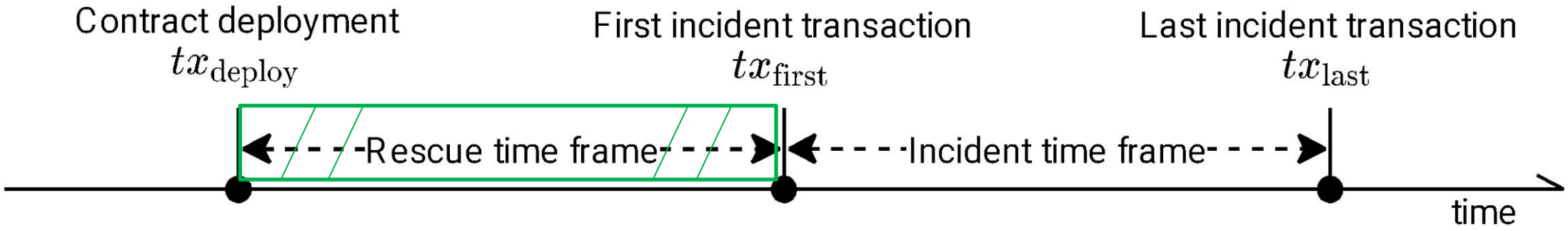
or

2. Batching contract deployment + attack execution

103 (56%) attacks are NOT executed atomically

→ rescue time frame for defenders

Rescue and Incident Time Frame



1. Deploy an adversarial SC before attack

or

2. Batching contract deployment + attack execution

103 (56%) attacks are NOT executed atomically → rescue time frame for defenders

Adversaries *can* be front-run during the rescue time frame.

Bytecode Similarity Analysis

- **100%** similarity among 38 victim contracts
  - **80%** similarity among 85 victim contracts
- **100%** similarity among 29 adversarial contracts
  - **80%** similarity among 73 adversarial contracts

Bytecode Similarity Analysis

- **100%** similarity among 38 victim contracts
  - **80%** similarity among 85 victim contracts
- **100%** similarity among 29 adversarial contracts
  - **80%** similarity among 73 adversarial contracts



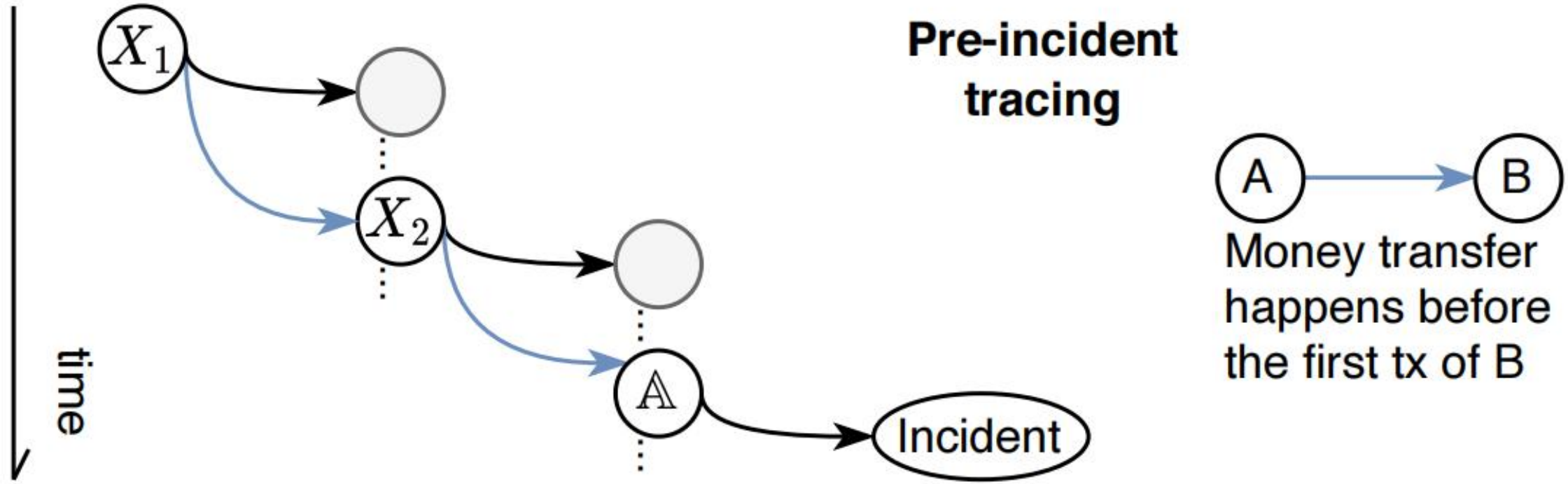
Adversarial and vulnerable contracts are detectable.

Attacks and Relays

- 18/181 incidents are executed through FaaS using **Flashbots API** (622M USD)
- 6/18 incidents appear to be accelerated by, e.g., arbitrage trader

List of incidents executed via Flashbots	
20210713_DeFiPie	20220114_FloatProtocol
20210718_ArrayFinance	20220118_Multichain
20210720_SanshuInu	20220210_BuildFinance
20210830_CreamFinance	20220217_RigoBlock
20211102_VesperFinance	20220320_Lifinance
20211110_Curve	20220327_RevestFinance
20211121_BadgerDAO	20220402_InverseFinance
20211127_dydx	20220430_FeiProtocol
20211211_SorbetFinance	20220430_SaddleFinance

Money Tracing



Fund Source

Pattern	Total	$h = 1$	$h = 2$	$h \geq 3$
Pre-incident (76 incidents in total, excluding U_2-non-monetary adversaries)				
Centralized Exchange $\xrightarrow{h}$ $\mathbb{A}$	128(49.0%)	40(15.3%)	23(8.8%)	65(24.9%)
Tornado.Cash $\xrightarrow{h}$ $\mathbb{A}$	94(36.0%)	67(25.7%)	19(7.3%)	8(3.1%)
Typhoon.Network $\xrightarrow{h}$ $\mathbb{A}$	9(3.4%)	6(2.3%)	2(0.8%)	1(0.4%)
Mining Pool $\xrightarrow{h}$ $\mathbb{A}$	7(2.7%)	-	1(0.4%)	6(2.3%)
Cross-chain Bridge $\xrightarrow{h}$ $\mathbb{A}$	5(1.9%)	3(1.1%)	2(0.8%)	0(0.0%)
Unknown	18(6.9%)			

Privacy

Wikileaks Says …

Bitcoin

Bitcoin is a secure and anonymous digital currency. Bitcoins cannot be easily tracked back to you, and are safer and faster alternative to other donation methods. You can send BTC to the following address:

Wired Says …

Bitcoin

Bitcoin won't hide you from the NSA's prying eyes

By **KADHIM SHUBBER**
11 Jun 2013

Why Privacy?



Alice is paid in BTC

0.05 BTC every week

Should everyone know her salary?



Gaming console company …

Pays a VR headset consultant …

Should competitors know?

Why Privacy?

But crime, drugs, terrorism, etc?

Cannot target only worthy uses of privacy!

Society is full of privacy compromises.

Anonymity



Interacting with no name at all

Pseudonymity



Interacting under a name, just not yours

Bitcoin Addresses are Pseudonymous

- Hash of public key: Not your name, But still a pseudonym

Pseudonymous is not Private

- You order a latte at a coffee shop and pay in Bitcoin
- The barista has a link between you and that address and from that address to prior addresses…

Linkability

Every Bitcoin transaction is public

- Repeated interactions can be linked to each other
- And maybe directly to you

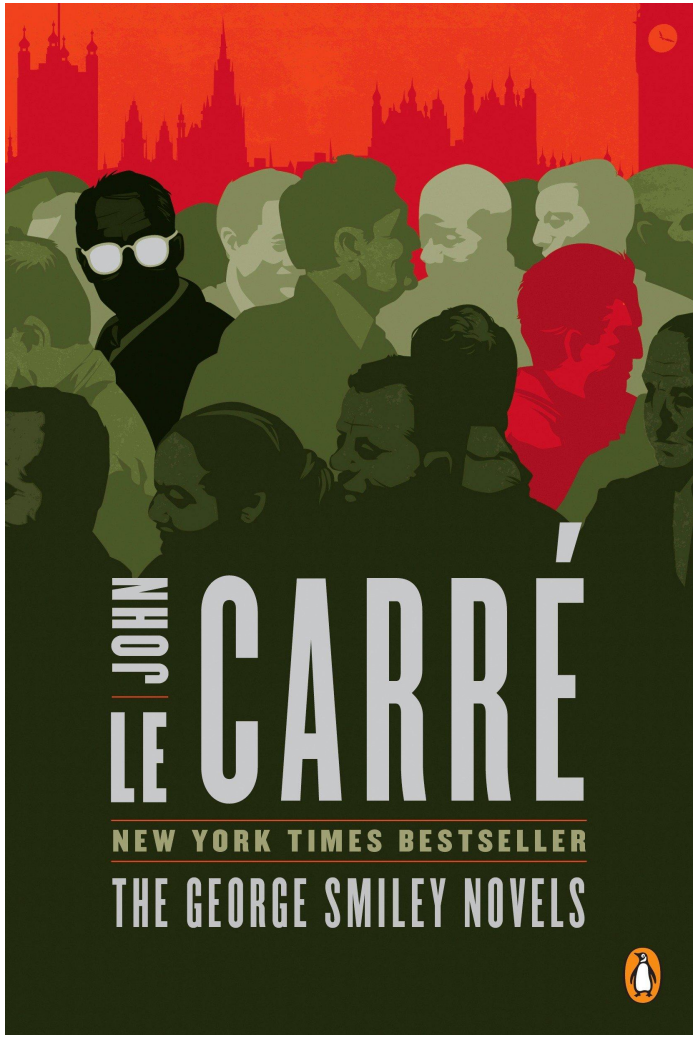
Even if you use lots of different addresses,

- They can of often be linked

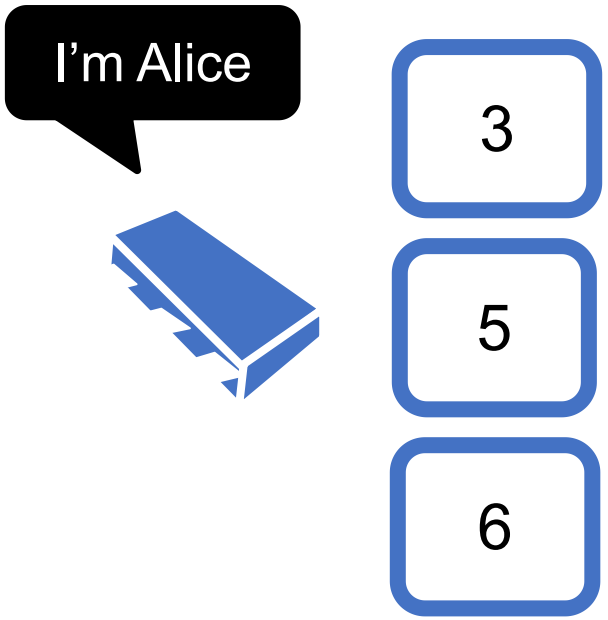
Side Channel Attacks

In a LeCaré novel, Smiley notices that encrypted transmissions intercepted from anonymous Soviet mole paused while a colleague was on vacation

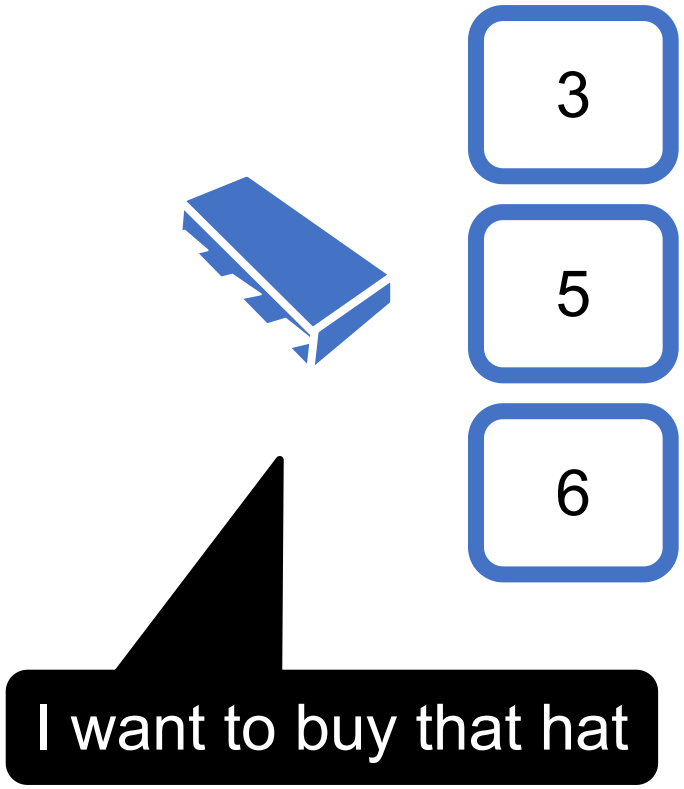
Timing, Amounts, Patterns can link addresses



Let’s De-Anonymize Bitcoin

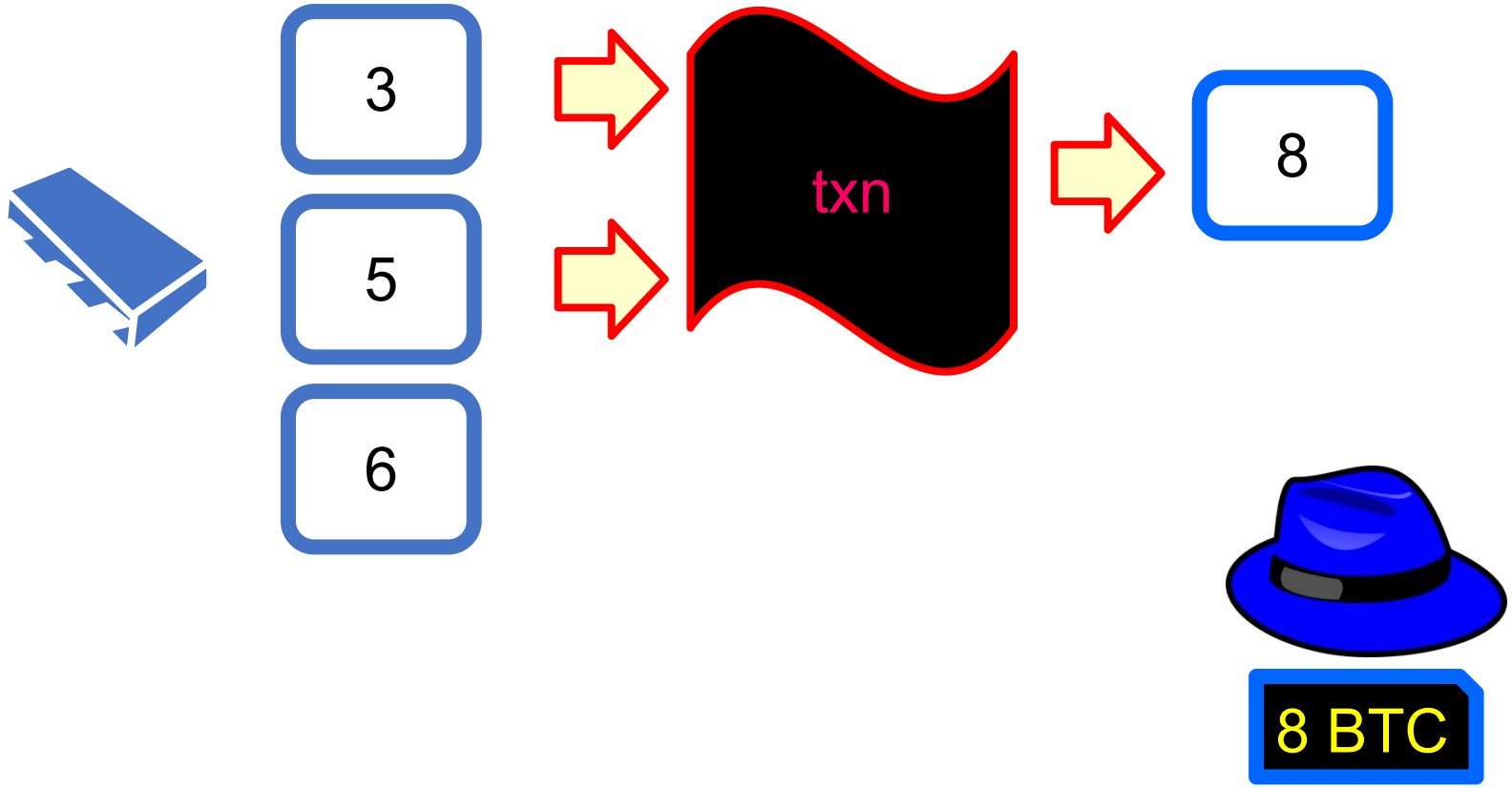


Let’s De-Anonymize Bitcoin

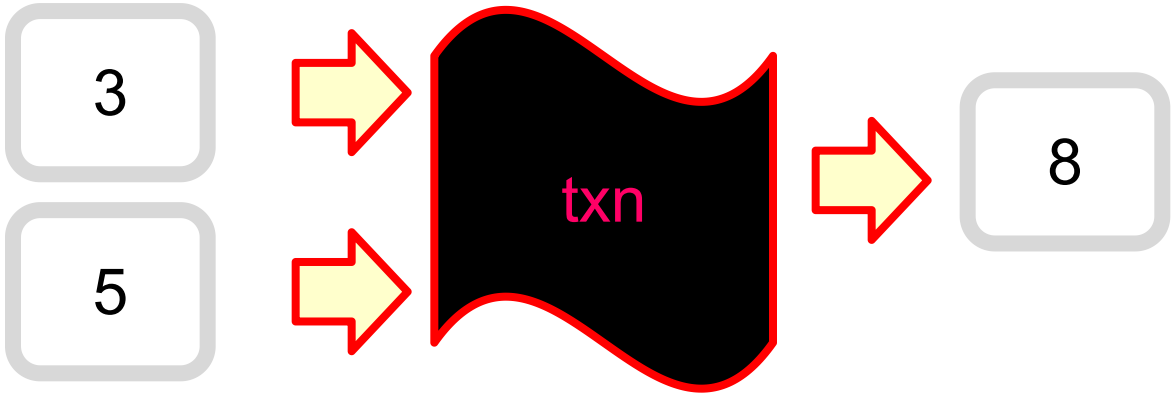




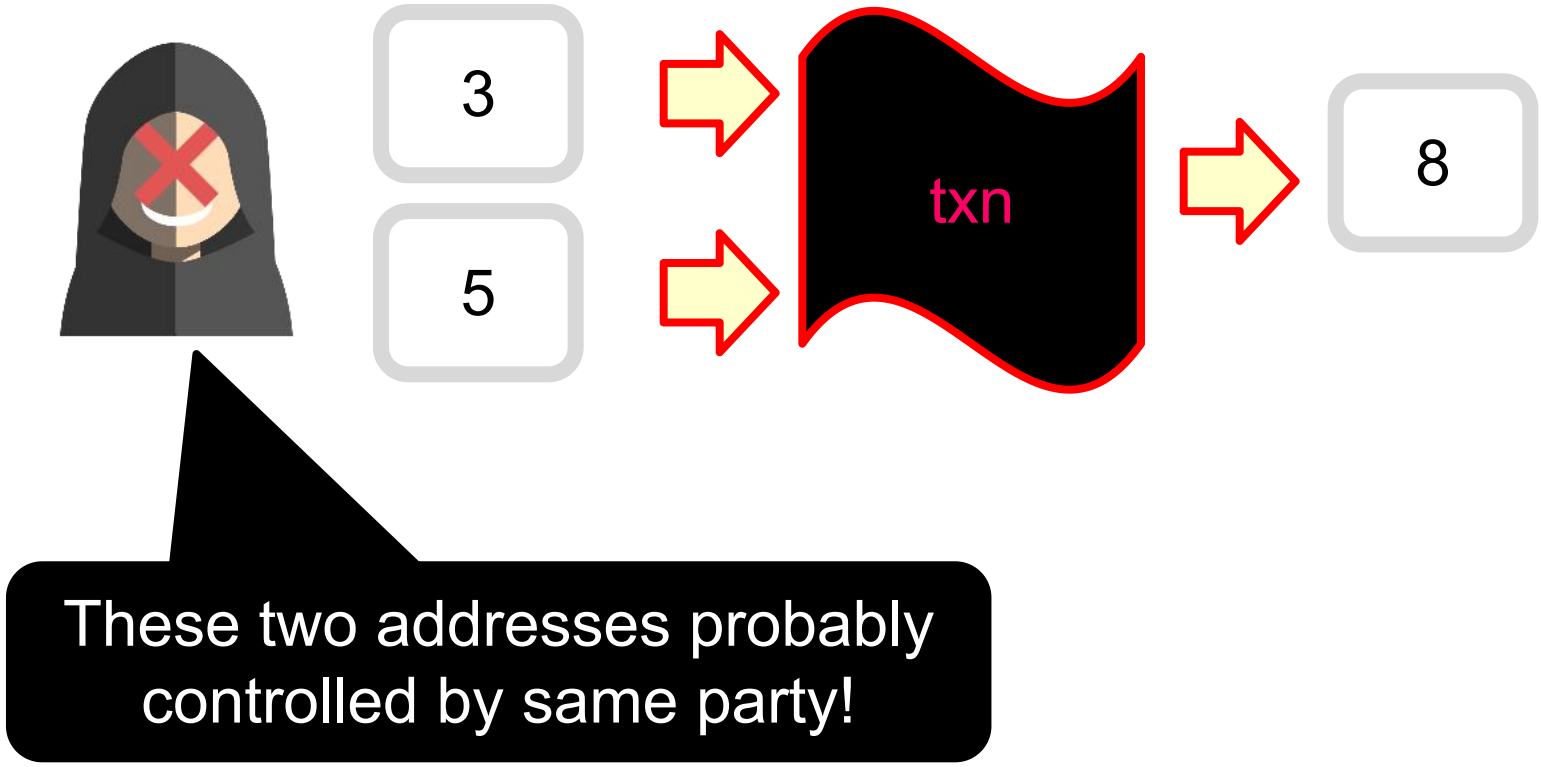
Let's De-Anonymize Bitcoin



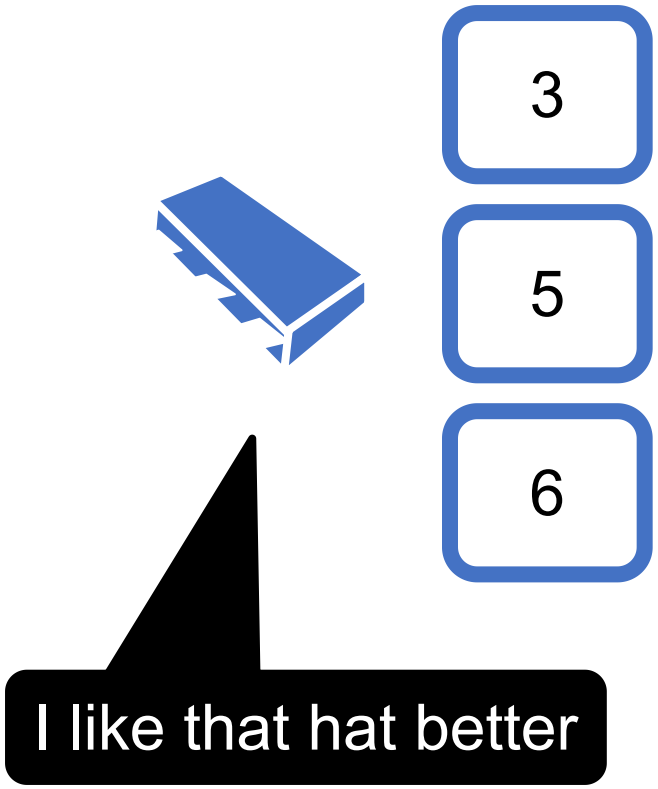
Adversary Sees



Adversary Sees

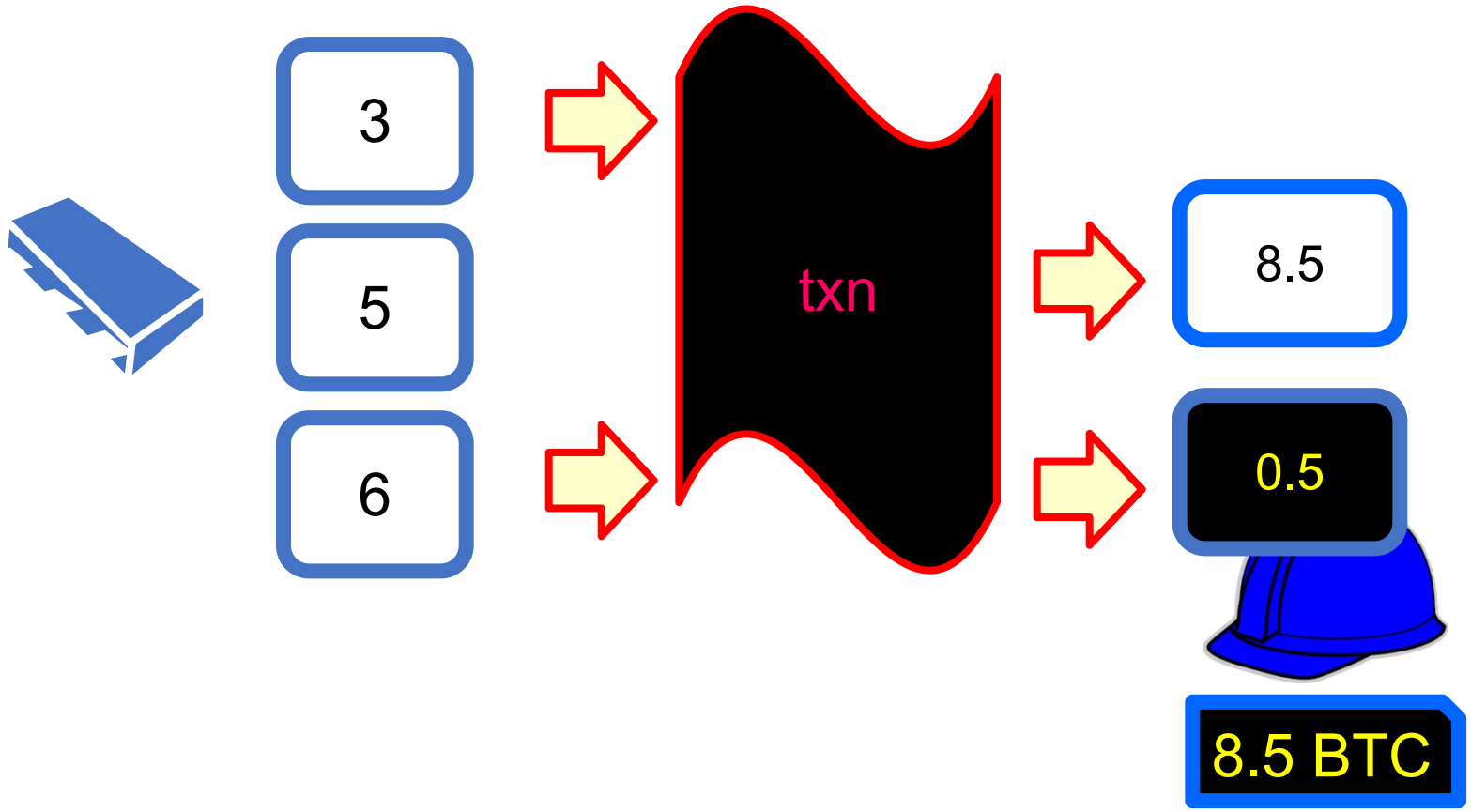


Let’s De-Anonymize Bitcoin

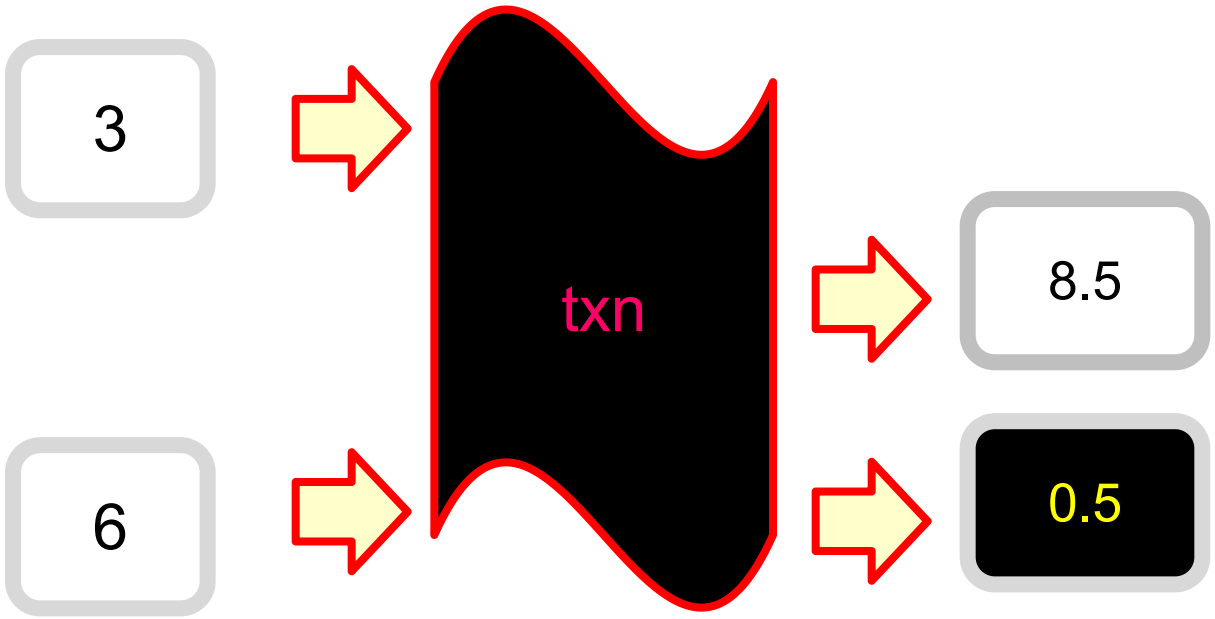




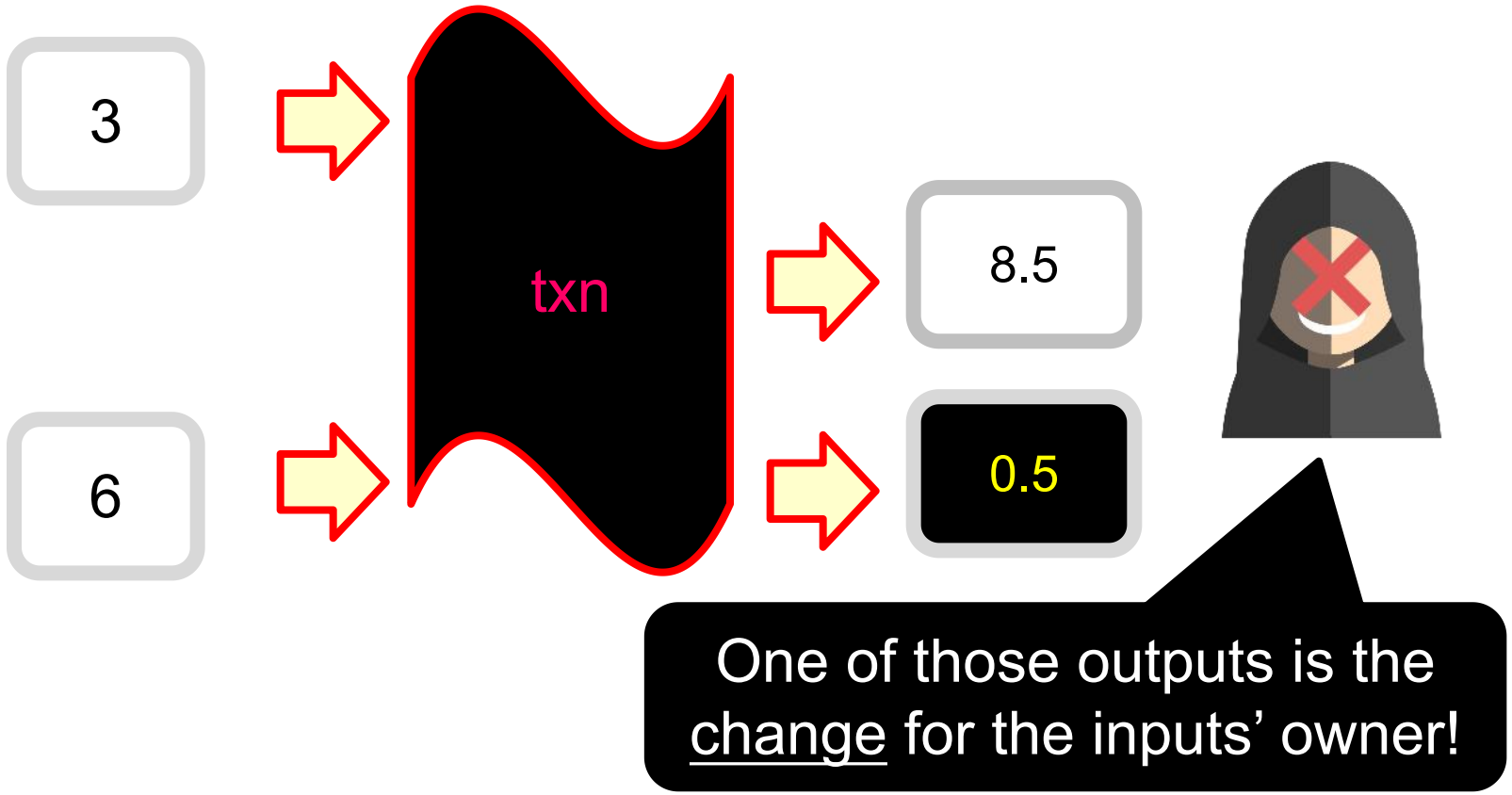
Let's De-Anonymize Bitcoin



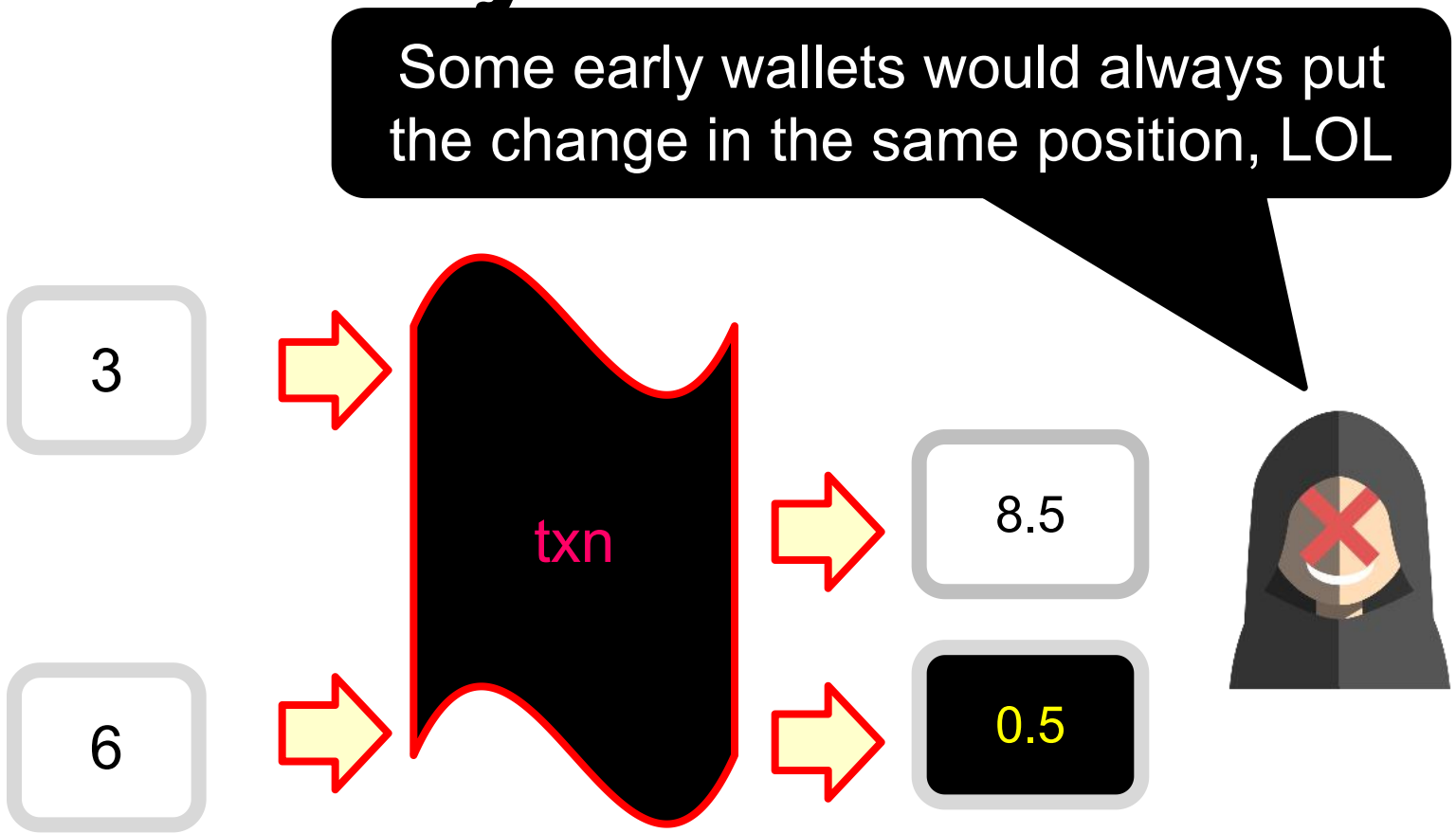
Adversary Sees



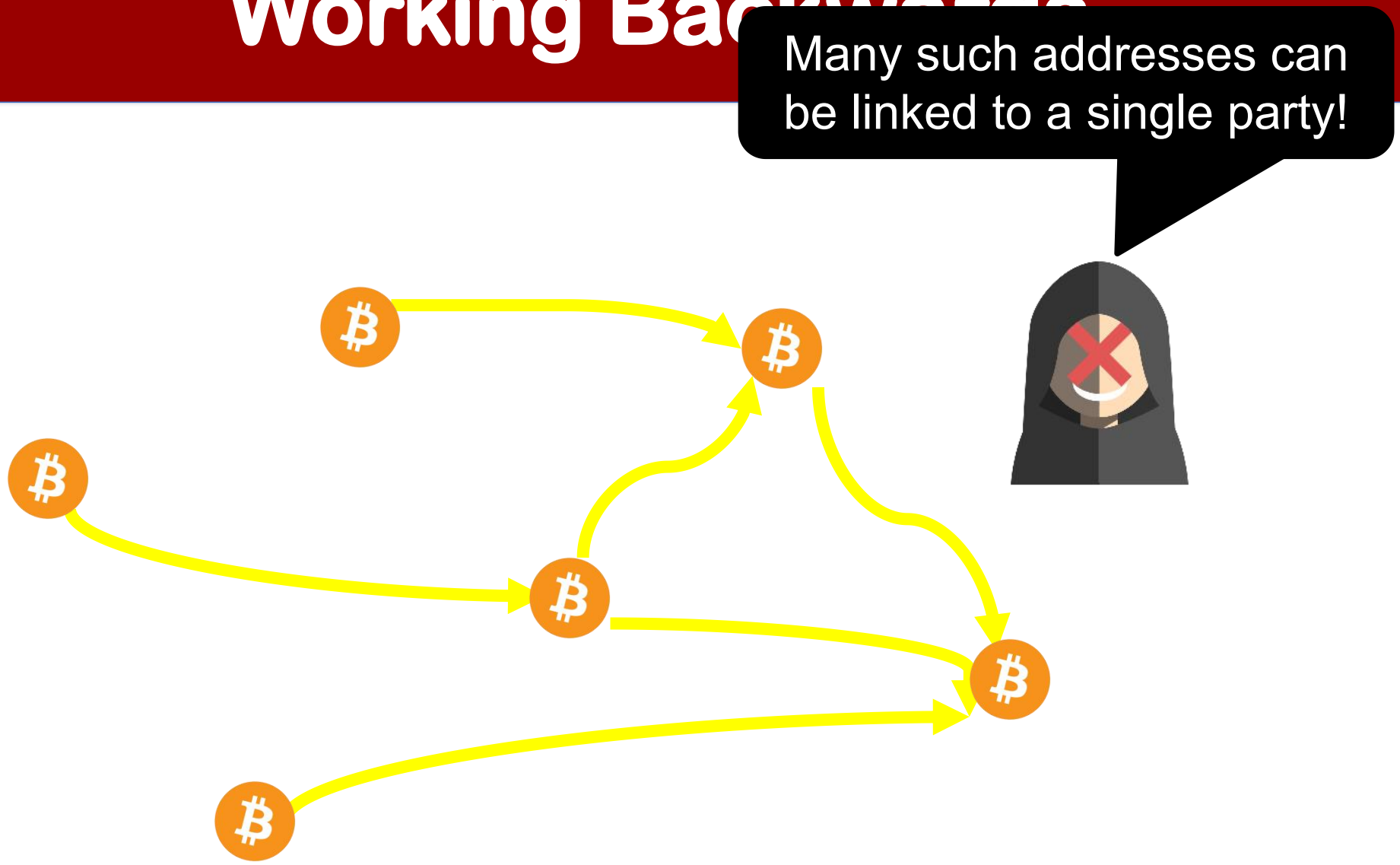
Adversary Sees



Adversary Sees



Working Backwards



Research Work in Linkability

A Fistful of Bitcoins: Characterizing Payments Among Men with No Names

Sarah Meiklejohn Marjori Pomarole Grant Jordan
Kirill Levchenko Damon McCoy† Geoffrey M. Voelker Stefan Savage

University of California, San Diego George Mason University†

ABSTRACT

Bitcoin is a purely online virtual currency, unbacked by either physical commodities or sovereign obligation; instead, it relies on a combination of cryptographic protection and a peer-to-peer protocol for witnessing settlements. Consequently, Bitcoin has the unintuitive property that while the ownership of money is implicitly anonymous, its flow is globally visible. In this paper we explore this unique characteristic further, using heuristic clustering to group Bitcoin wallets based on evidence of shared authority, and then using re-identification attacks (i.e., empirical purchasing of goods and services) to classify the operators of those clusters. From this analysis, we characterize longitudinal changes in the Bitcoin market, the stresses these changes are placing on the system, and the challenges for those seeking to use Bitcoin for criminal or fraudulent purposes at scale.

By
deplo
tem t
paym
identify the payer or the payee: a transaction is a cryptographically-signed transfer of funds from one public key to another. Moreover, like cash, Bitcoin transactions are irreversible (in particular, there is no *chargeback* risk as with credit cards). However, unlike cash, Bitcoin requires third party mediation: a global peer-to-peer network of participants validates and certifies all transactions; such decentralized accounting requires each network participant to maintain the entire transaction history of the system, cu
over 3GB of compressed data. Bitcoin identit
anonymous: while not explicitly tied to real-
organizations, all transactions are completely t
This unusual combination of features has gi

Uses heuristics to link bitcoin transactions

Industrial scale

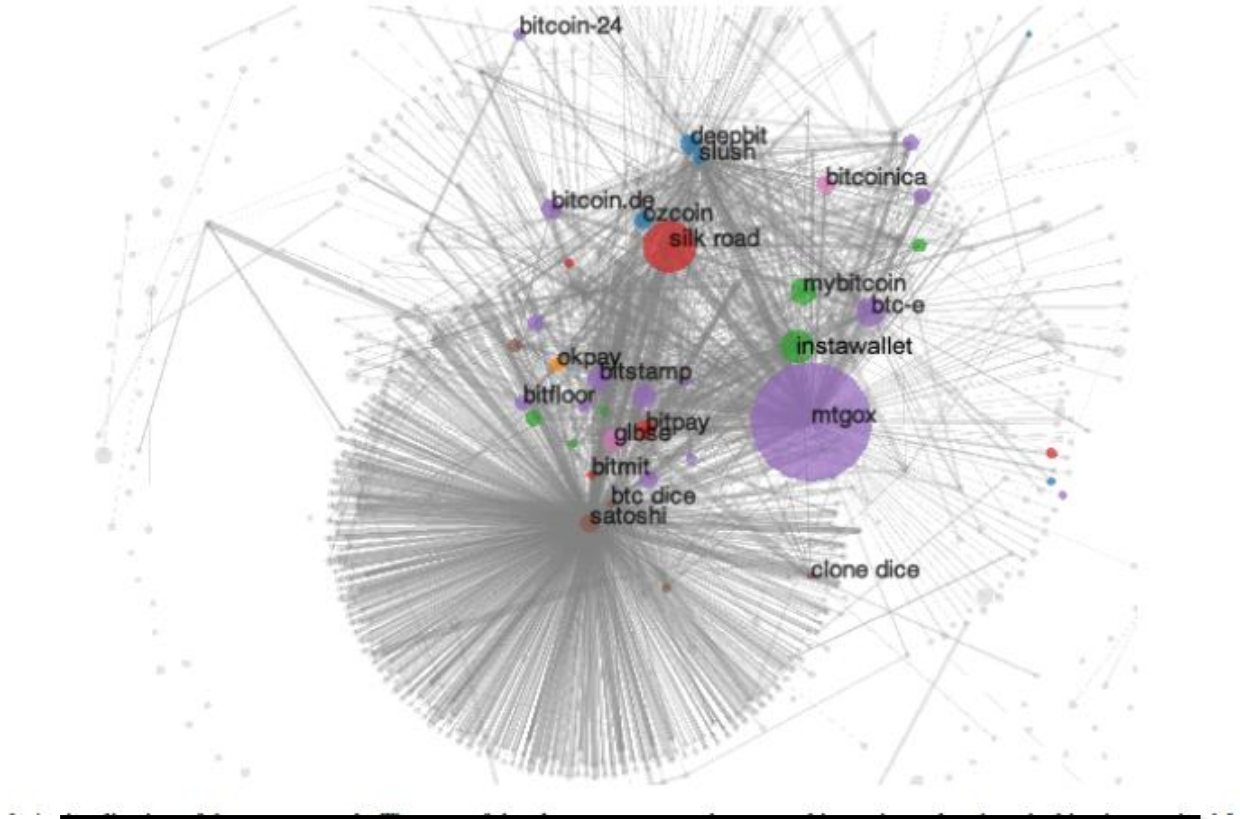
Merchant address seen before

Re-Identifation Attack

How to connect addresses with institutions?

- Buy stuff
- Join mining pool
- Engage wallet services
- Join gambling sites

Research Work from Meiklejohn et al.



Clusters, color-coded by industry

For an edge to appear between two nodes there must have been at least 200 transactions between them

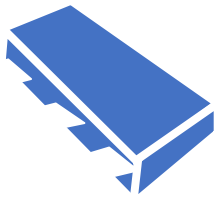
Blue:	mining pools
Orange:	fixed-rate exchanges
Green:	wallets
Red:	vendors
Purple:	(bank) exchanges
Brown:	gambling
Pink:	investment schemes
Grey:	uncategorized

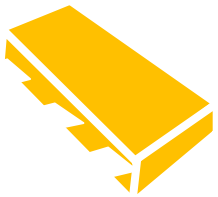
Linking to Real-World People

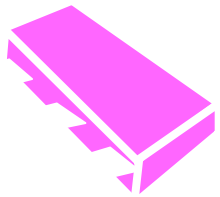
- Direct transactions
  - Carelessness: *e.g.*, request donations to static address
- "Dusting Attack"
- AML/KYC compliance

Dusting Attack

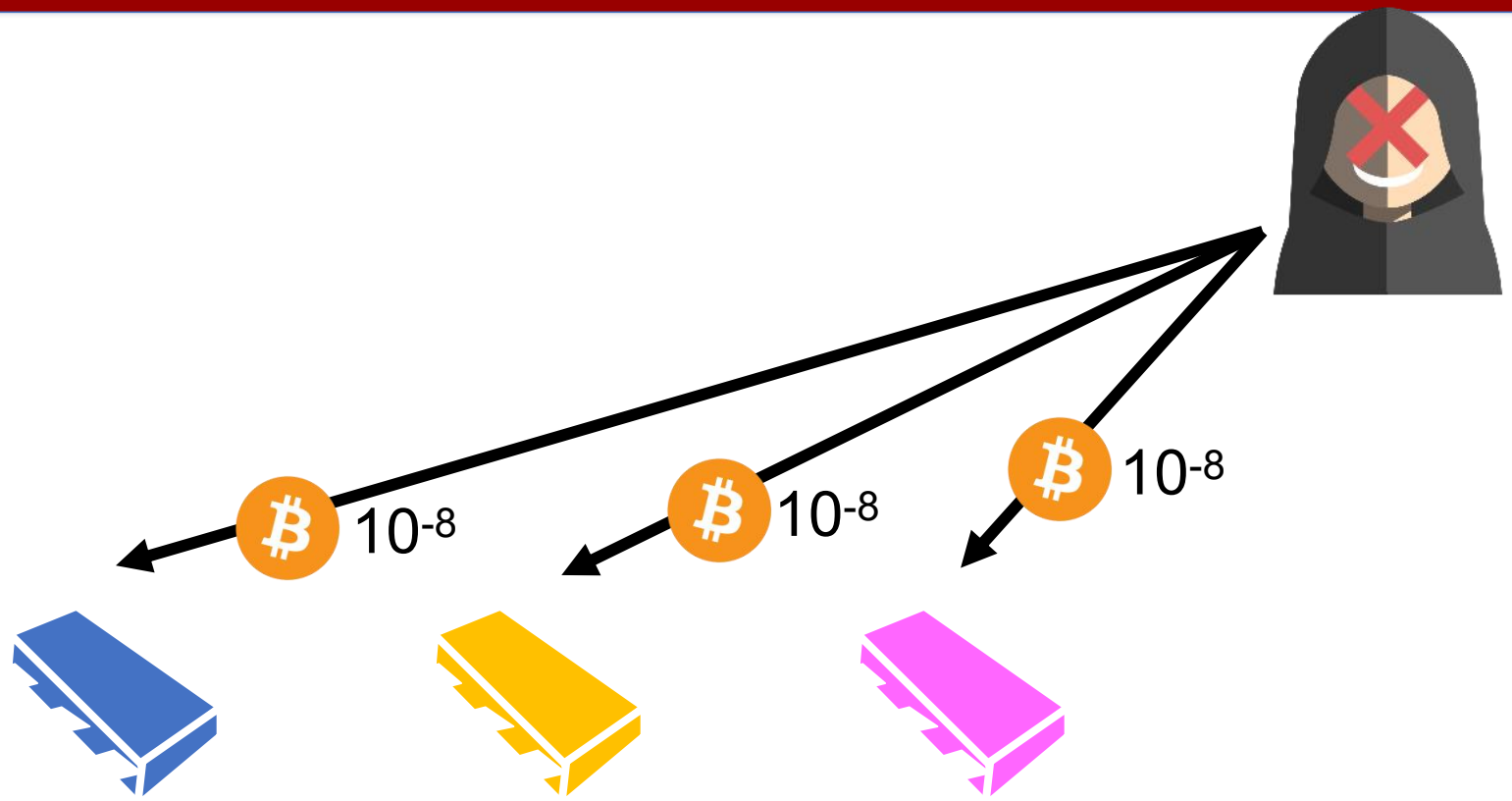








Dusting Attack

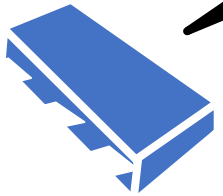


Give each address a tiny sum

Dusting Attack



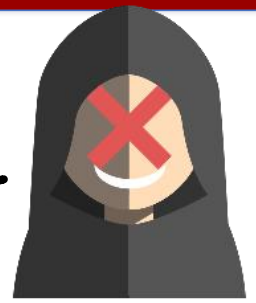
It’s so annoying to have all these UTXOs …



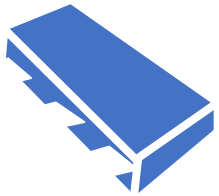
1000

10^{-8}

Dusting Attack



I have you now, my pretty!
I have just linked your address with you!



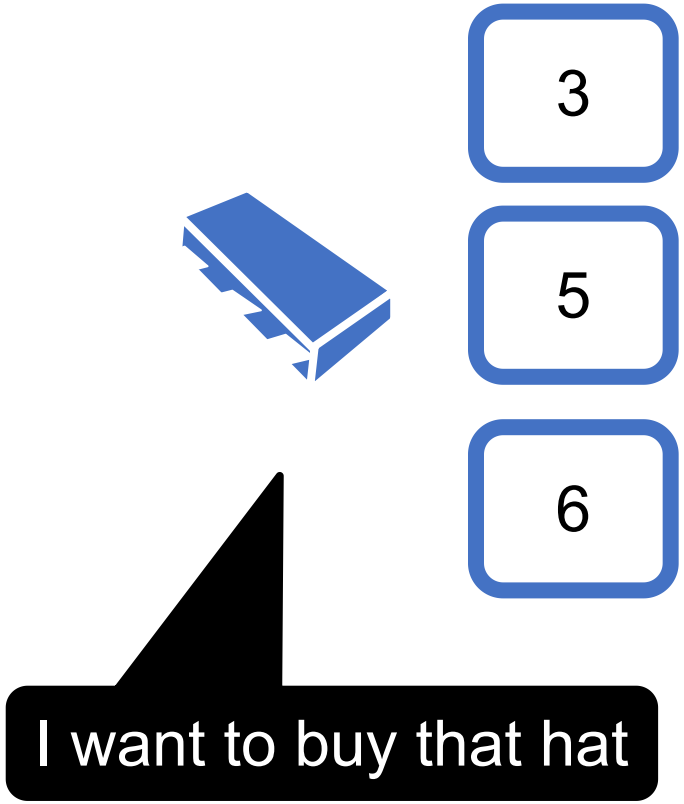
1000

10^{-8}

txn

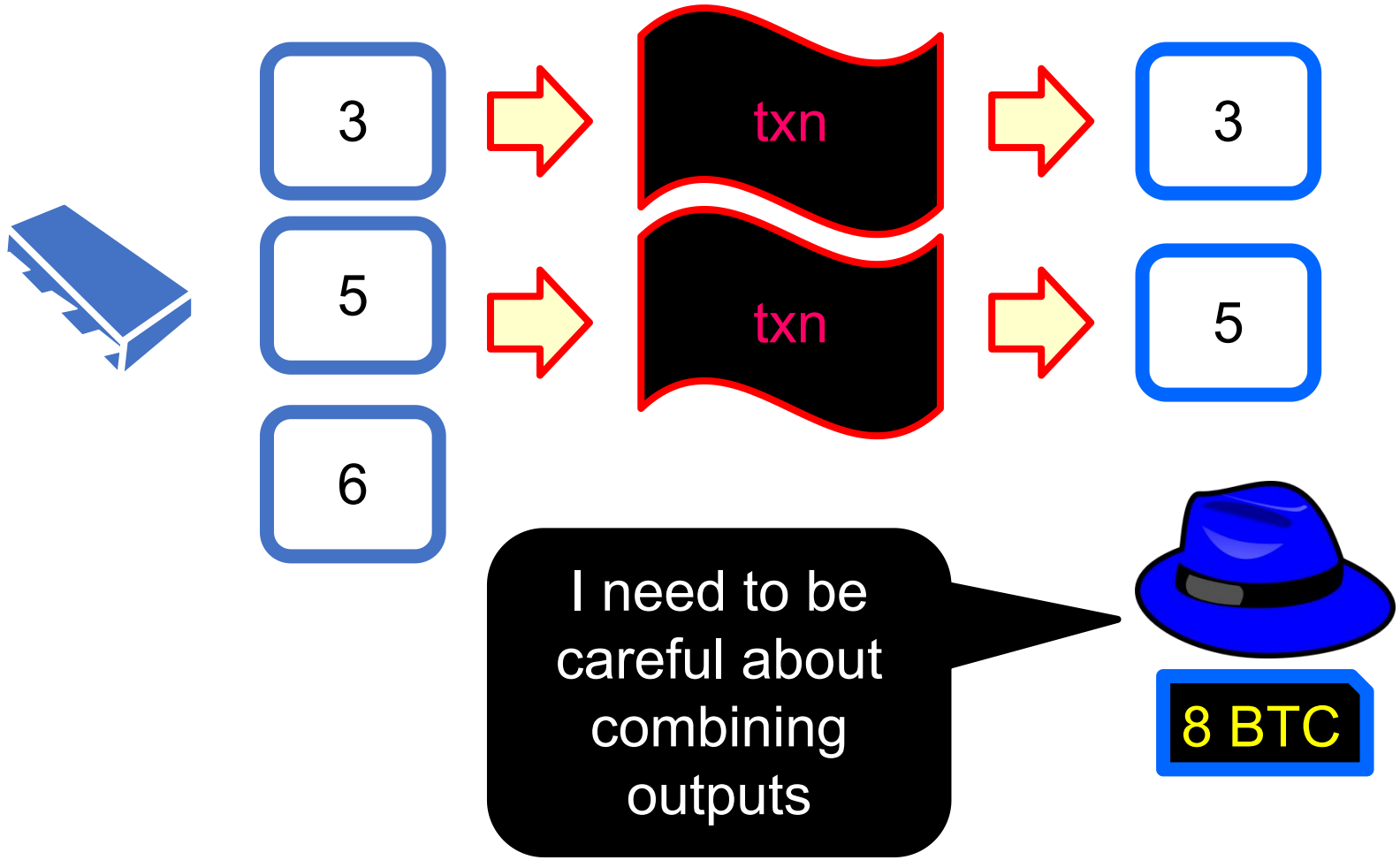
1000+

Merge Avoidance





Merge Avoidance



Anti-Money Laundering (AML) / KYC



Financial institutions are required to find out who you are

Trade BTC all you like, but when you convert to fiat, your identity becomes known

Posted by u/A_lump_of_Carbon 7 months ago

1

Apparently coinbase is now implementing KYC for all withdrawals... Does this effect UK people too?

I use coinbase to cheaply send my bitcoins mined from nicehash.

Network-Level Attacks

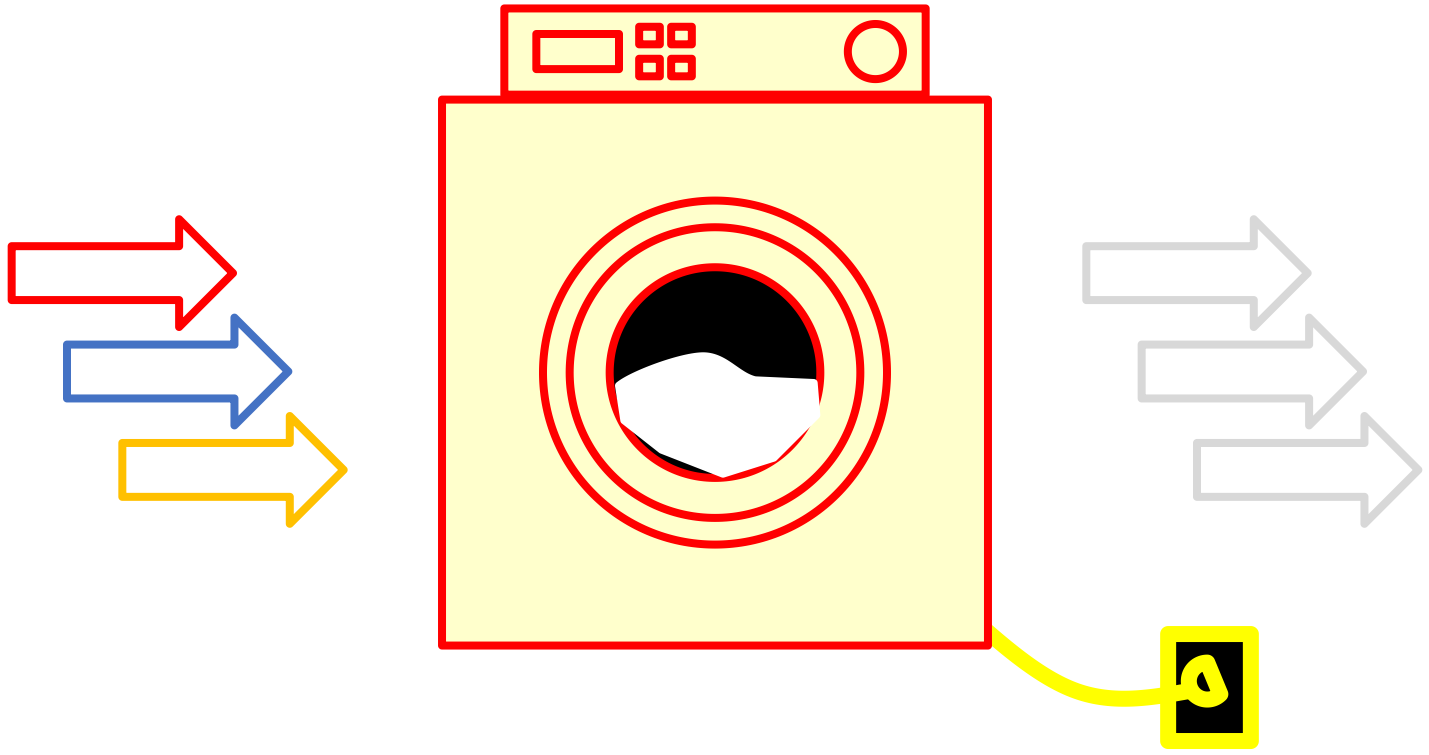
- Transactions flow through P2P network
- Nodes can collude to identify parties
- First node to announce transaction is probable source
- Identify origin IP address
- Partial solution: Tor network hides IP address, not ideal

Unlinkability

It should be hard to link …

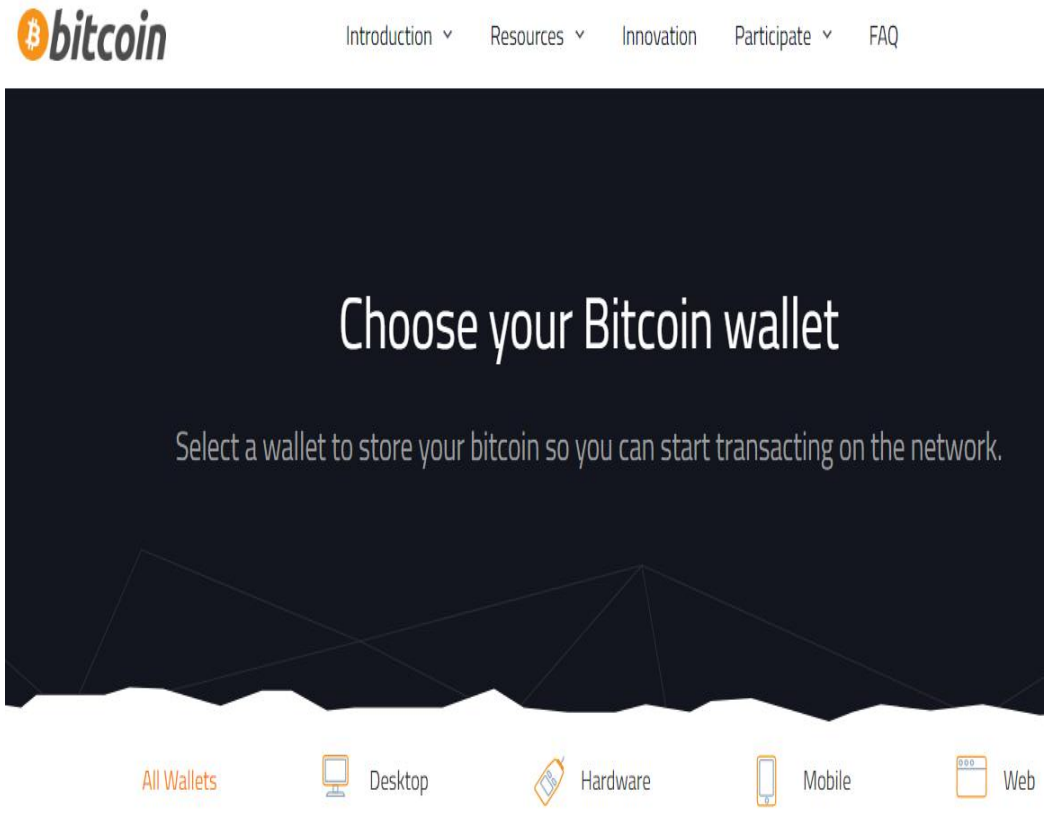
- Different addresses of same party
- Different transactions of same party
- Payment sender to (ultimate) recipient

Let’s Re-Anonymize Bitcoin



Use intermediary to obfuscate ownership

On-Line Wallets



Deposit one address …

Get back another.

But they keep KYC/AML records

No better than traditional banks

Much worse, if hacked

Mixing Services

On-line services

Send them your bitcoins

Give the mixer a destination address

Hopefully, mixer sends your coins to that address

Simultaneously with others in anonymity set

Anonymity Set

A group of entities with the same attributes

making them indistinguishable from each other

Set of transactions that …

the adversary cannot distinguish from yours…

Mixers


English
BESTMIXER.IO
Need help?
Introduction
Fees
FAQ
API
Resources
Contacts
TOP RATED SECURE BITCOIN MIXER
MIX YOUR BITCOINS TO STAY SAFE AND PROTECTED
SELECT THE ASSET AND START COIN MIXING

You need to trust the mixers

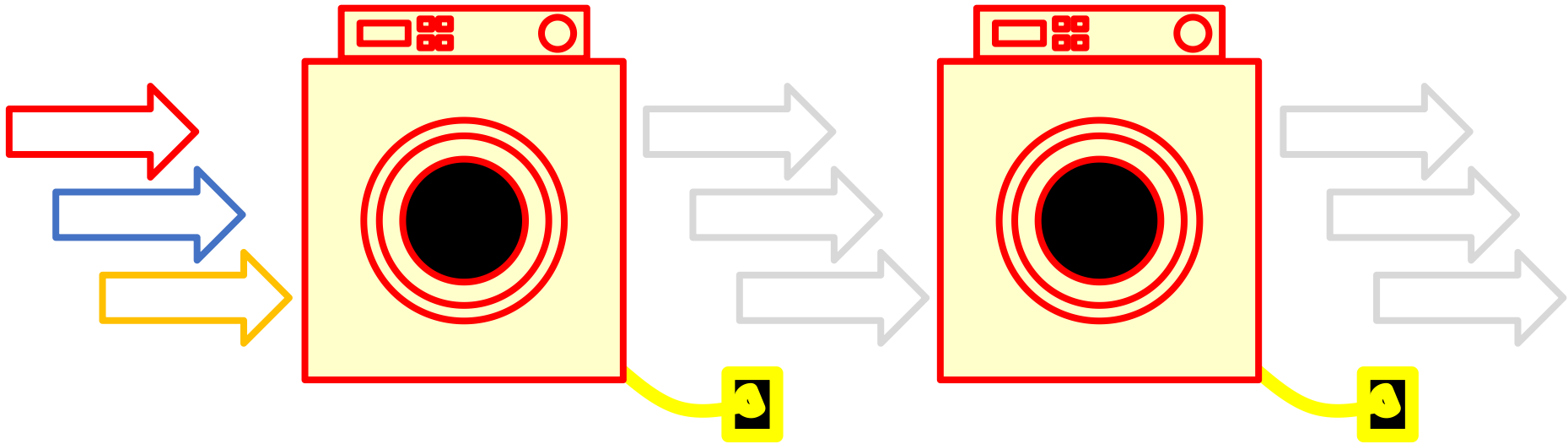
They don’t keep records

They don’t keep your coins

They don’t give you “tainted” coins

Anonymity set not too small

Practice Tips

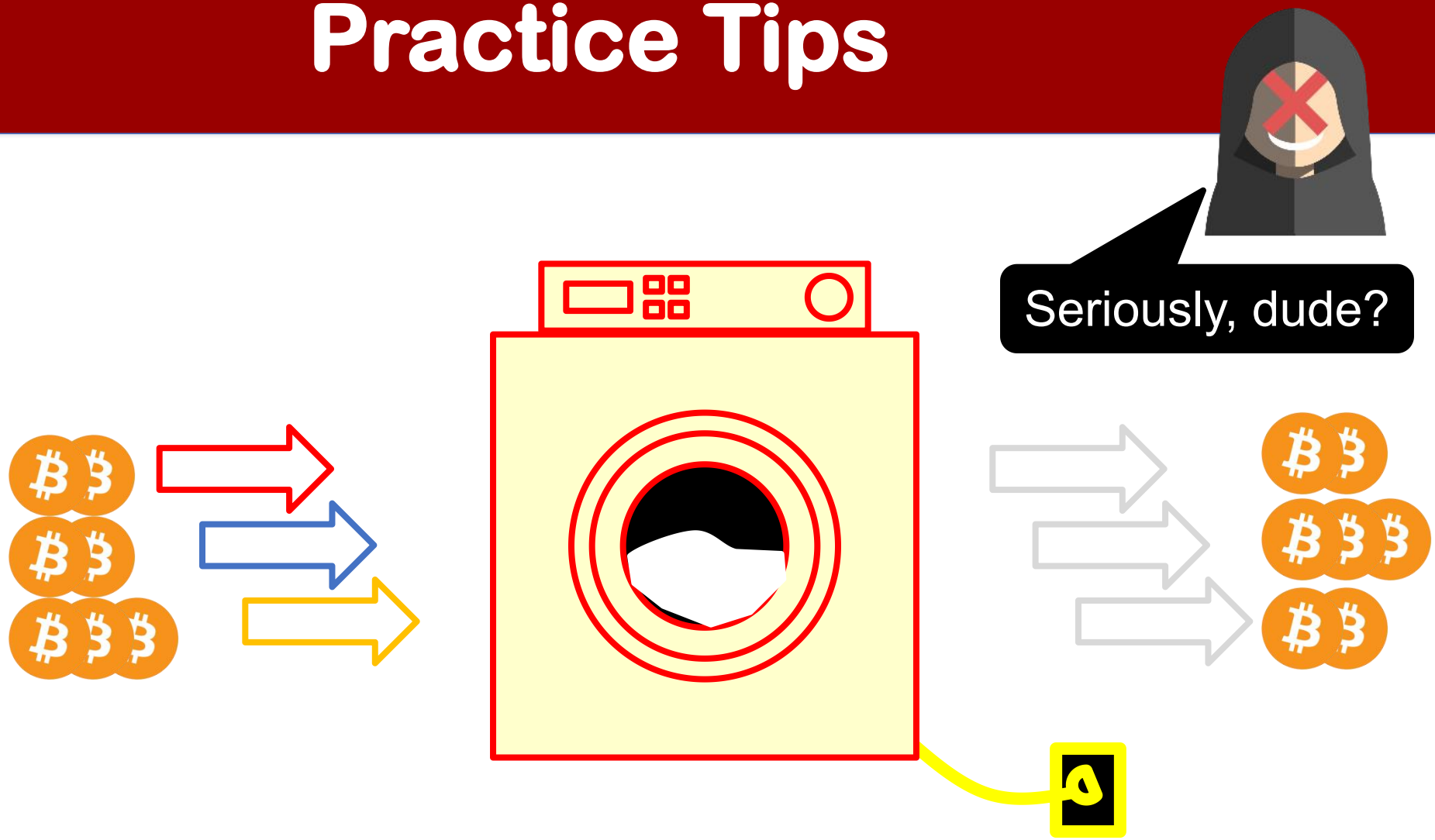


Use a sequence of mixers

You are OK if at least one discards records

Beware of Fees

Practice Tips

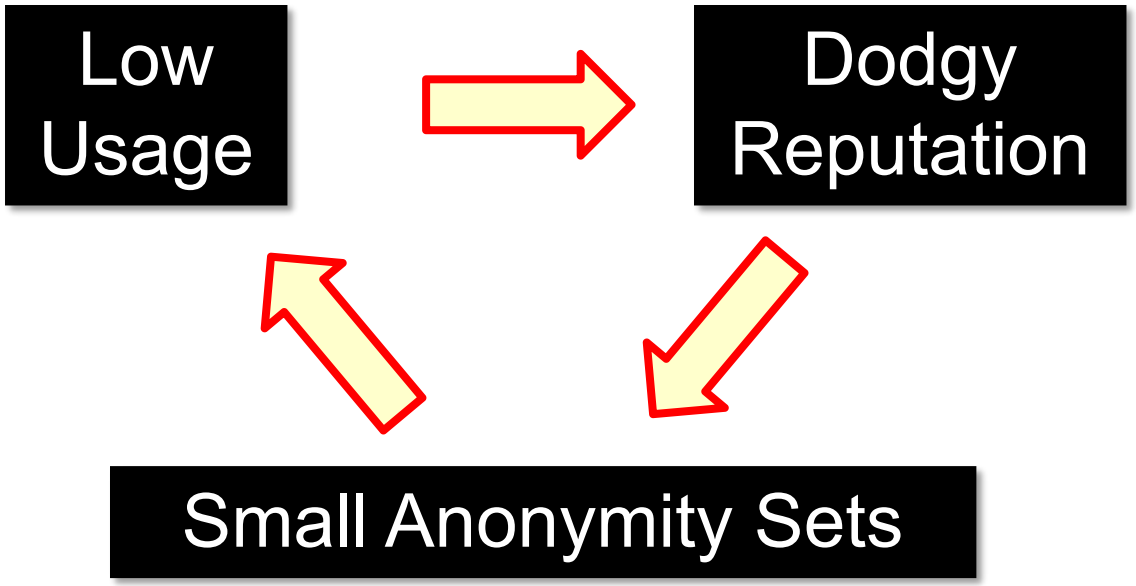


Use standard “chunk” amount

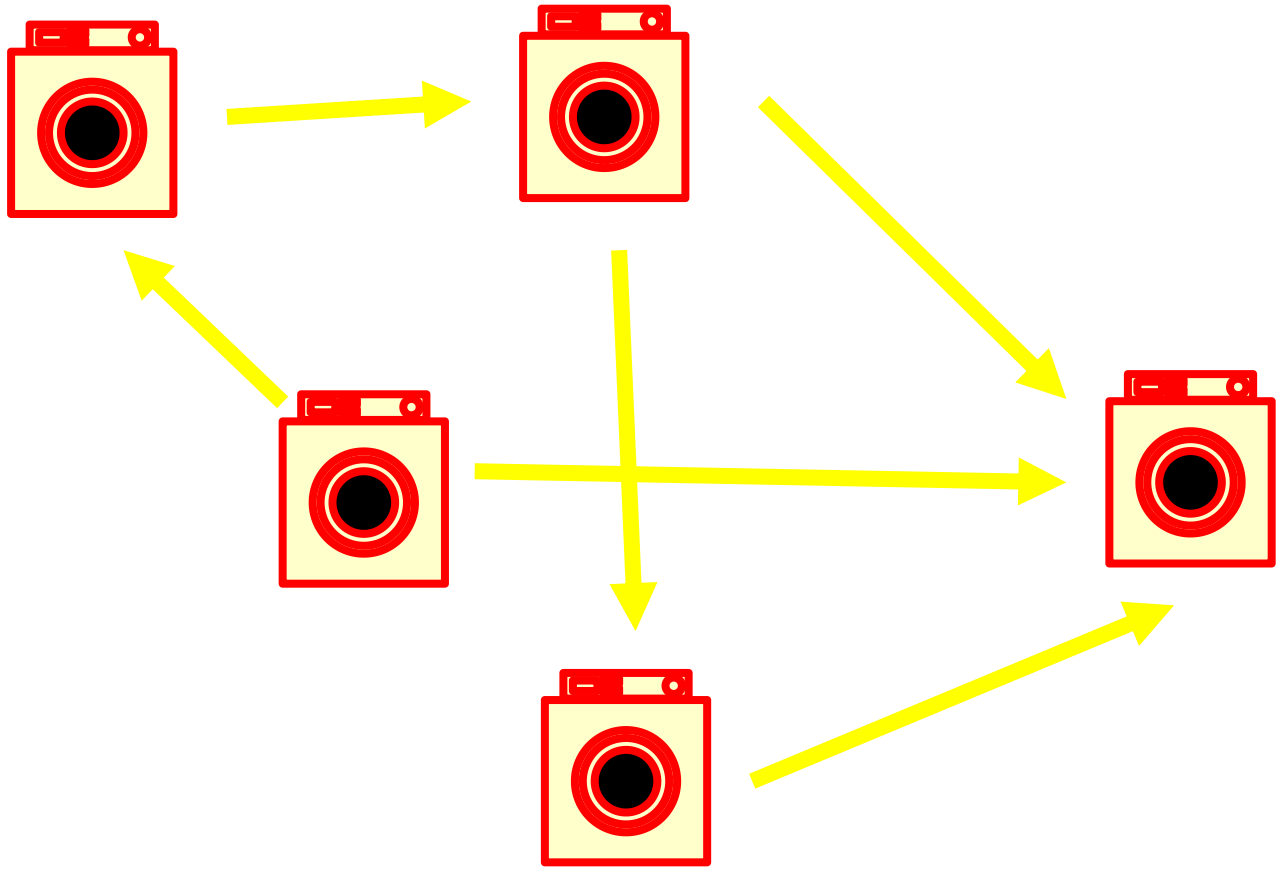
Mixing Services Today



Mixing Services Today



Decentralized Mixers



Decentralized Mixers

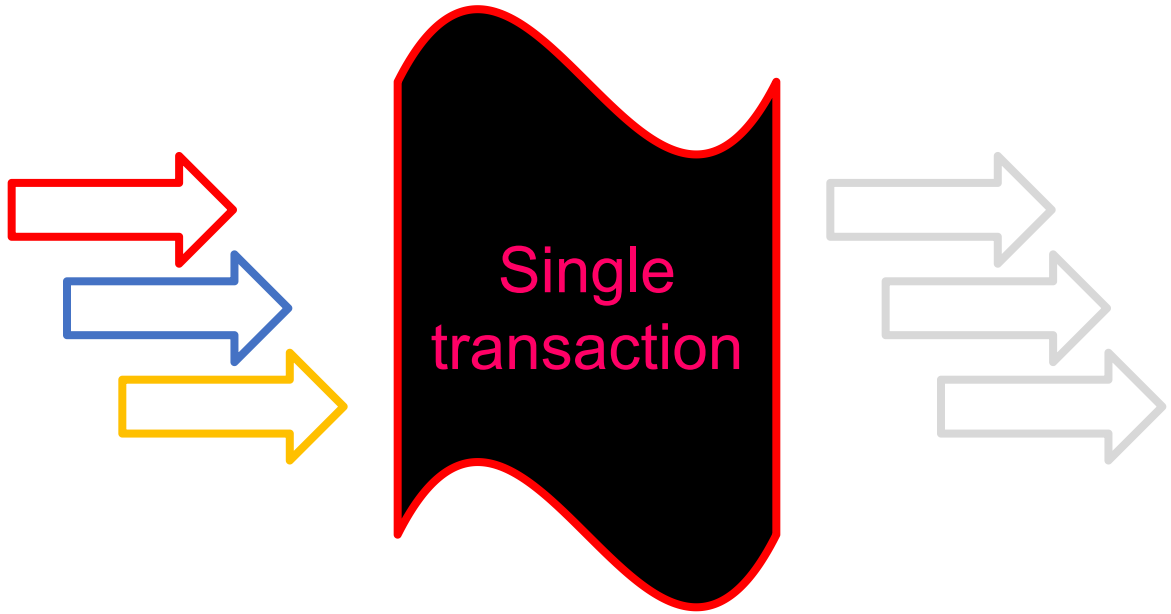
Peer-to-Peer

No Theft

Better anonymity maybe

More aligned with BTC philosophy?

CoinJoin



CoinJoin Protocol

Find Peers who want to mix

Exchange input, output addresses

Construct transaction

Send it around, collect signatures, validate

Boadcast the transaction

Find Peers who Want to Mix

- Services that match up interested parties
- Unlike mixer services, cannot steal your funds
- Cannot compromise identity
- Sometimes charge fees

Find Peers who Want to Mix

JoinMarket Orderbook

20 orders found by 20 counterparties

Toggle non-segwit

Check for timed-out counterparties

BTC ▾ % ▾

Type	Counterparty	Order ID	Fee	Miner Fee Contribution / BTC	Minimum Size / BTC	Maximum Size / BTC
SW Absolute Fee	J5CVNPgz5qtoc3Vw	0	0.00000000	0.00000000	0.01020917	14.19022848
SW Absolute Fee	J5F3j1y5GrShNqsR	0	0.00000273	0.00000000	0.00047978	0.15942784
SW Absolute Fee	J52ryGy9BV8aiaBm	0	0.00000300	0.00000100	0.00100000	32.81462841
SW Absolute Fee	J56px2x5sLmpmPAu	0	0.00000300	0.00000100	0.00100000	51.19093479
SW Absolute Fee	J58FRF3yVvemZxWE	0	0.00000300	0.00000100	0.00100000	59.57317321
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

Exchange Input, Output Addresses

Tricky …

No one should know input-to-output mapping

Maybe use Tor

Or special-purpose anonymous routing

Denial of Service

Peer might not sign transaction

Peer might spend coin before transaction broadcast

Fix: fees, bonds, blame protocols?

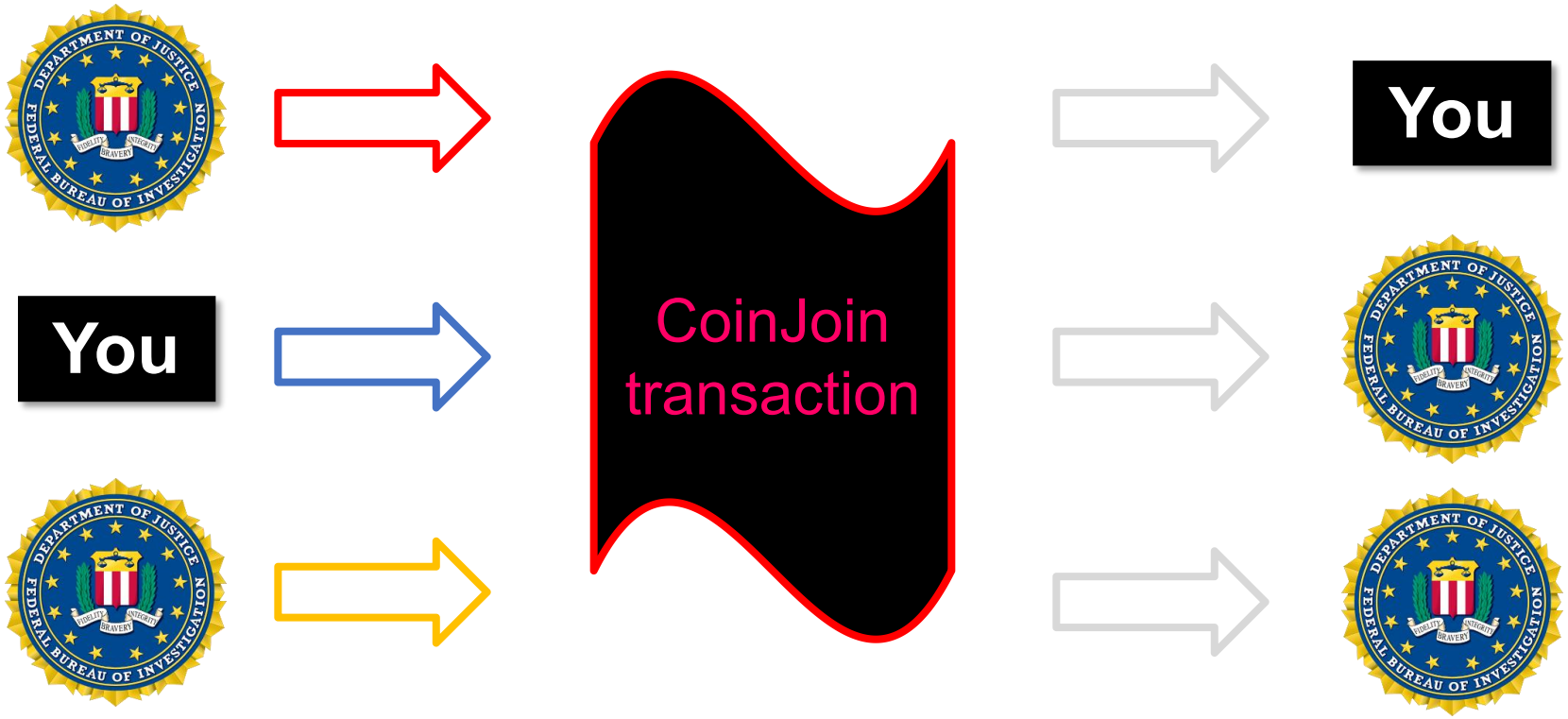
Practice Tips Still Apply to CoinJoin!

Mix multiple times

Use fixed-size chunk amounts

Beware of fees reducing that chunk

Paranoia Strikes Deep



Tornado Cash

How Tornado Cash works



Deposit

A user generates a random key (note) and deposits Ether or an ERC20, along with submitting a hash of the note to the Tornado Cash smart contract.

Wait

After depositing, users should wait some amount of time before withdrawing to improve their privacy.

Withdraw

A user submits a proof of having the valid key to one of the notes deposited and the contract transfers Ether or the ERC20 to a specified recipient.



How Tornado Cash achieves privacy

Tornado Cash improves transaction privacy by breaking the on-chain link between source and destination addresses. It uses a smart contract that accepts ETH deposits that can be withdrawn by a different address. To preserve privacy a relayer can be used to withdraw to an address with no ETH balance. Whenever ETH is withdrawn by the new address, there is no way to link the withdrawal to the deposit, ensuring complete privacy.

Tornado Cash Blacklisted by US Treasury

Policy

Crypto-Mixing Service Tornado Cash Blacklisted by US Treasury

The department barred its use by U.S. persons as a matter of national security because North Korean hackers allegedly use the mixer to launder stolen crypto funds.

By Nikhilesh De Aug 8, 2022 at 10:13 p.m. Updated Aug 9, 2022 at 11:44 p.m.



GENERAL CRYPTO · 1 minute read

Arrested Tornado Cash Developer is Alexey Pertsev, His Wife Confirms

Crypto Trends
@CryptoTrends

BREAKING: Tornado Cash developer has been arrested in the Netherlands.

6:53 PM · Aug 12, 2022 · Twitter for iPhone

RYAN SΞAN ADAMS - rsa.eth @RyanSAdams · 19h

They **arrested** the developer of **tornado cash.** 🚨

I repeat: a man was **arrested** for writing code that served as a public good for people to maintain their privacy online.

They put a man in jail because bad people used his open source code.

This cannot stand in any free society.

808 6,328 21.1K

Another Problem

2.7k

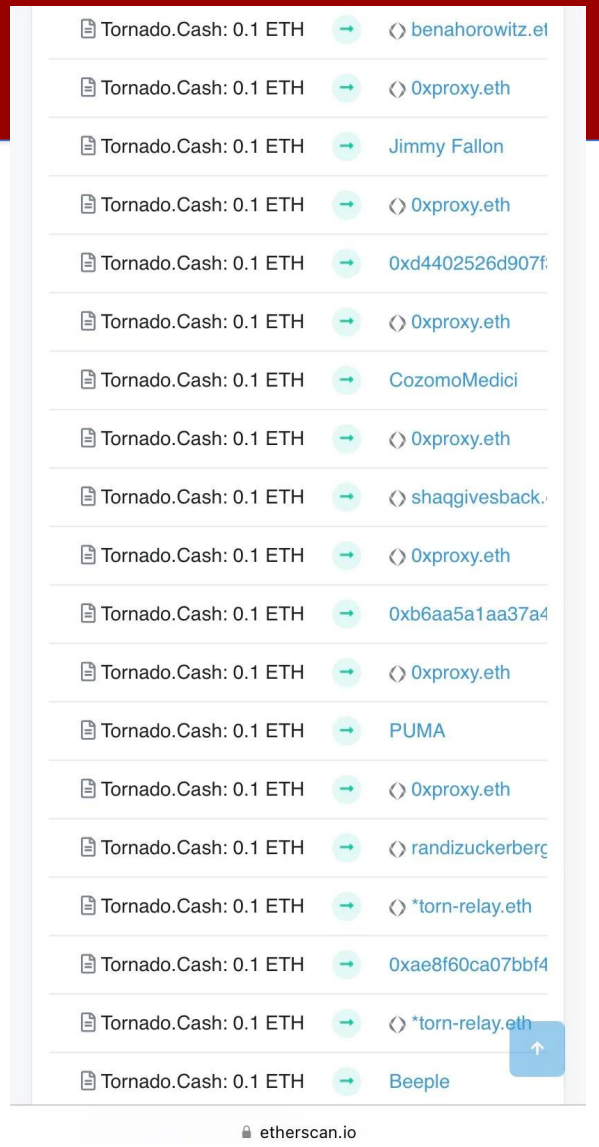
Posted by u/002timmy Platinum | QC: CC 396 | ADA 21 13.9k · 2 months ago

10 9 9

Someone is using Tornado Cash to send ETH to Doxxed Wallets

GENERAL-NEWS

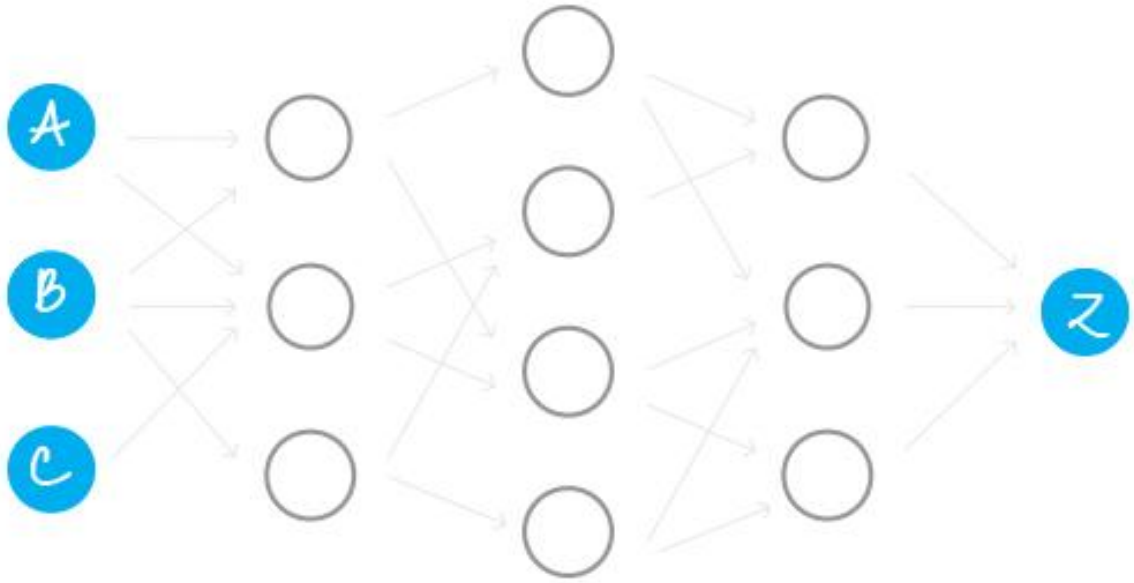
It appears there's still some chaotic good left in the world. Earlier today, someone or a group of individuals began using tornado cash to send ETH to known wallets of celebrities. Recipients included Donald Trump, Jimmy Fallon, Beeple, Shaq, Brian Armstrong, Randi Zuckerberg, BitBoy and many more.



Monero ($XMR)

MONERO

Focusing on Privacy



Ring signatures create ambiguity in blockchain analysis

MONERO



Obfuscates sender, receiver and amount



Monero Explained

Workshop:

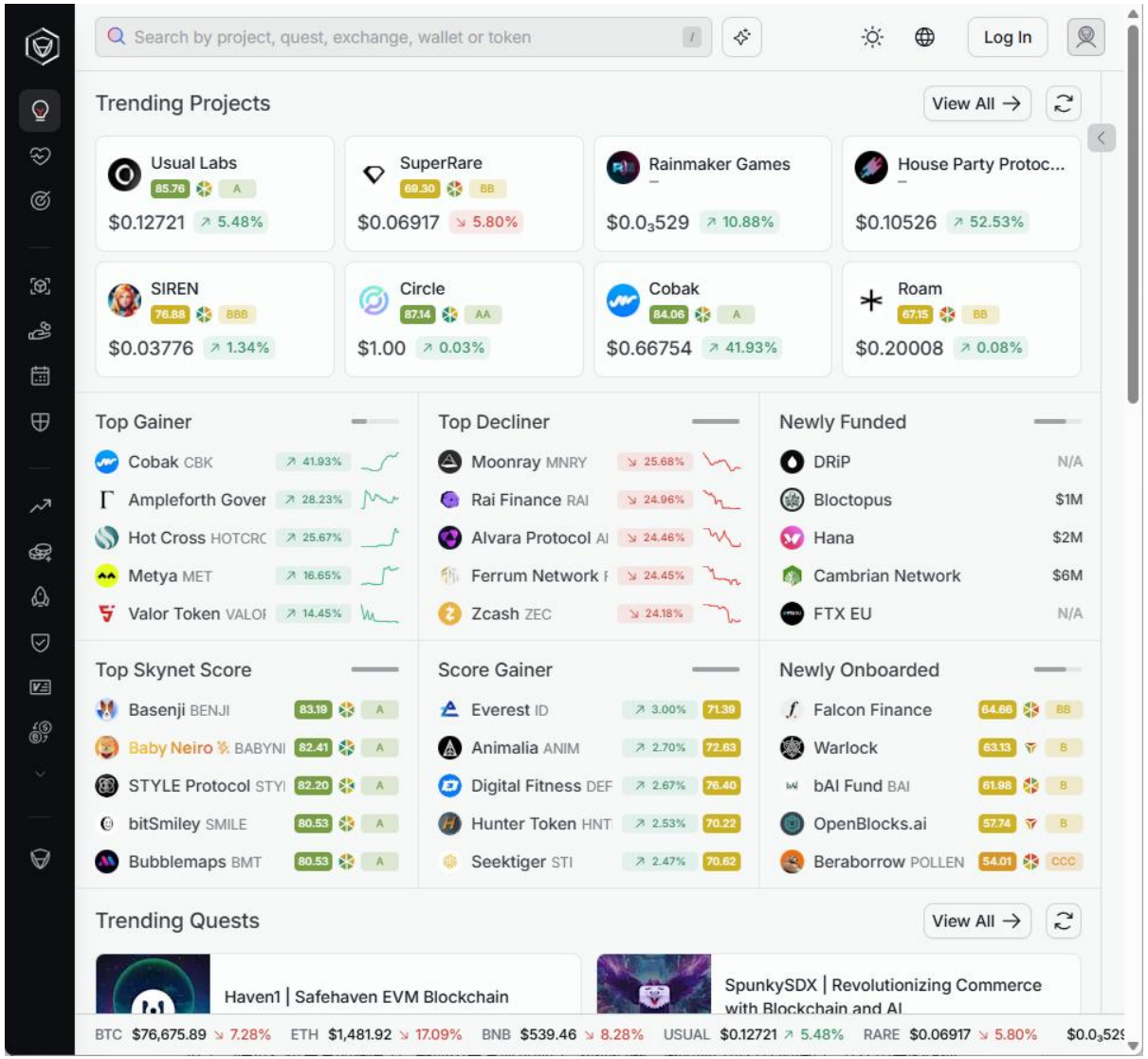
Security Tool in Web3

Security Tool in Web3

1. Dapps Security Auditing Report Ranking ——Skynet for Community
2. Security Incidents Database —— Rekt.database and X (CertiKAlert)
3. Security Chrome Plugin —— GoPlus
4. Blockchain Browser —— Etherscan
5. AML Tool —— Oklink and Skyinsights

Dapps Security Auditing Report Ranking

Website: https://skynet.certik.com/zh-CN



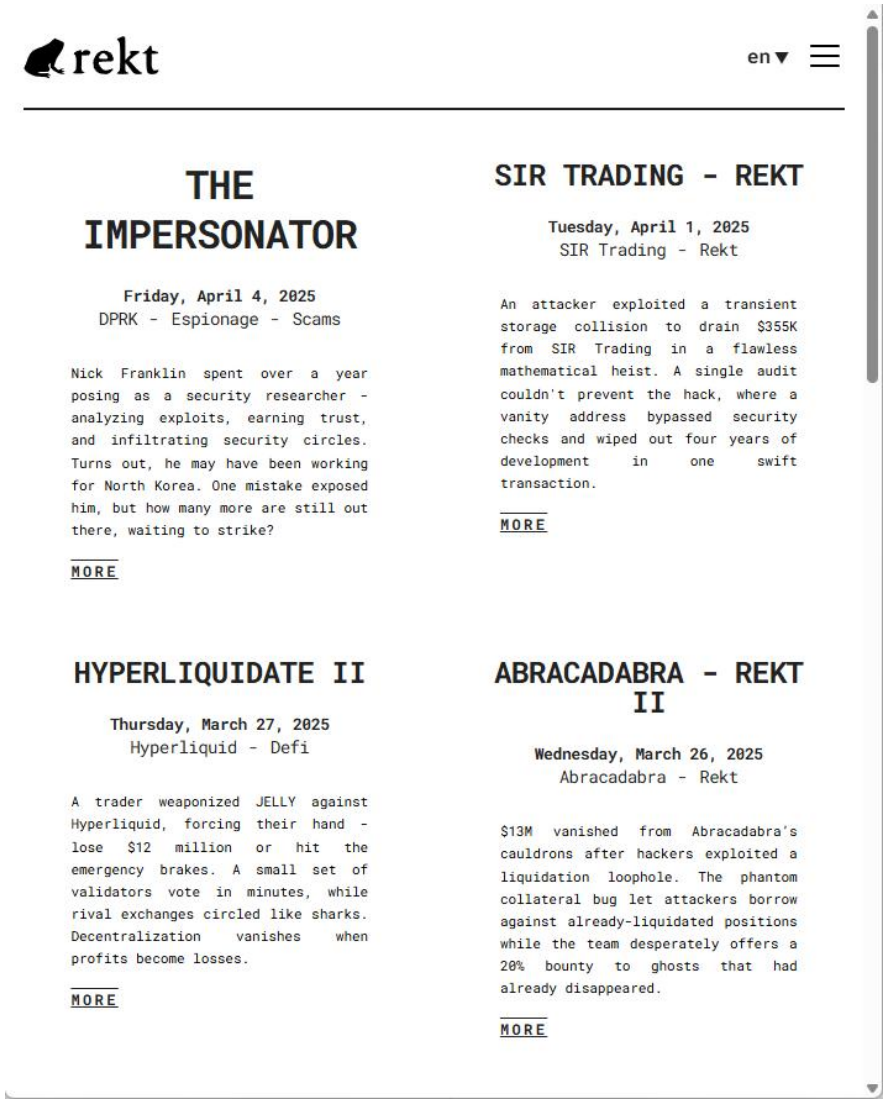
Security auditing refers to specialized security firms that conduct external audits on Web3 applications' smart contracts. They work to identify and fix any vulnerabilities that hackers might exploit, making the applications safer overall.

And usually, these audits finish with the findings being publicly disclosed.

- **Track Project Performance:** See trending DApps with high security and market performance.

Security Incidents Database

Website: https://rekt.news/



The rekt database is crucial in Web3 for helping users and developers learn from past security incidents, improve risk management, and enhance the overall security of decentralized applications and protocols.

It provide details about significant security incident.

Like showing up at the bank with someone else's ID and walking out with their life savings.

Game. Set. Match.

The Vault never stood a chance. The attacker kept spamming uniswapV3SwapCallback from their malicious contract, siphoning funds with every call.

Then, like any seasoned pro, they washed the loot through Railgun - one of crypto's favorite laundromat for post-exploit dry cleaning.

Attacker's Address: 0x27defcfa6498f957918f407ed8a58eba2884768c

Vulnerable Contract Address: 0xb91ae2c8365fd45030aba84a4666c4db074e53e7

Attack Transaction: 0xa05f047ddfdad9126624c4496b5d4a59f961ee7c091e7b4e38cee86f1335736f

THE AFTERMATH

The universe has a twisted sense of humor.

Just a day before the exploit, SIR Trading's founder, Xatarrer, posted a bizarre story on Twitter - one that, in hindsight, makes the timing of the attack feel a little too coincidental.

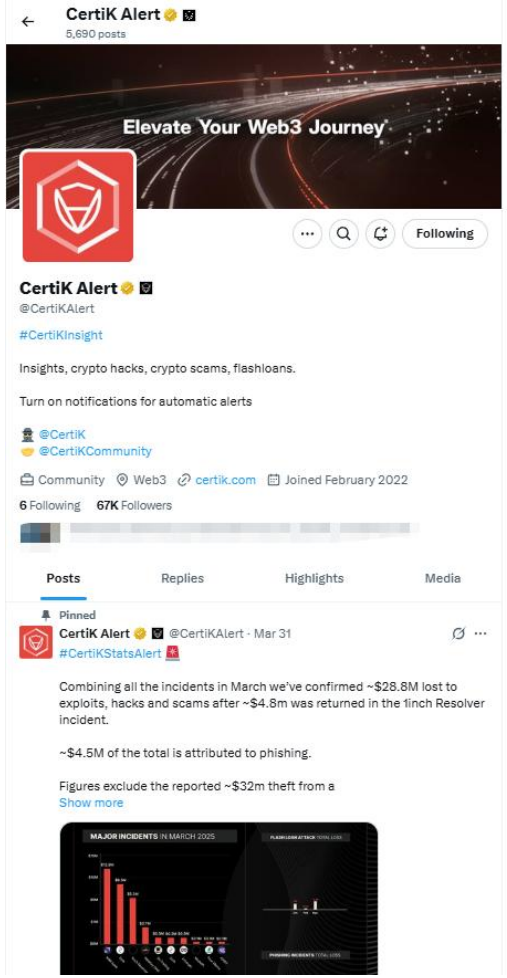
He claimed that a prospective dev had reached out to work for SIR. A promising candidate, well-connected, with top-tier references.

But before hiring, Xatarrer decided to subject him to the Kim Jong Un test - his personal method of weeding out North Korean operatives.

The "test" was simple, say something along the line of:

Security Incidents Database

Real-time Security Alert System: https://x.com/certikalert



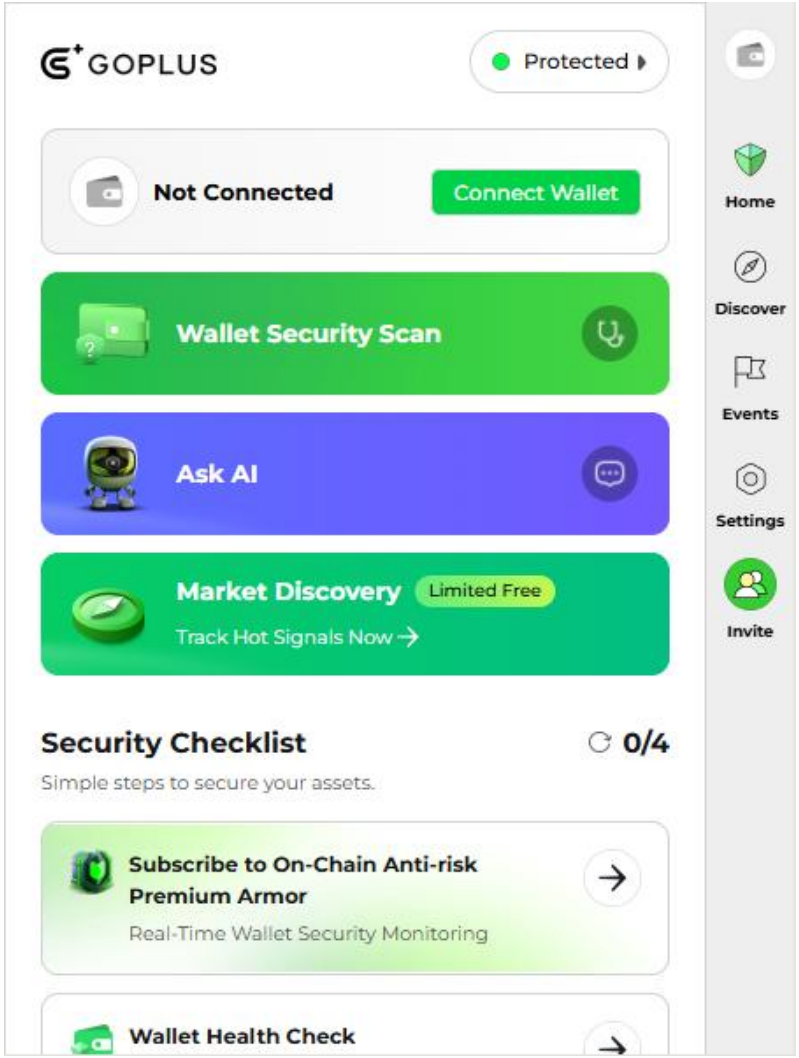
Chines Regular Security Reports：慢雾科技 Certik

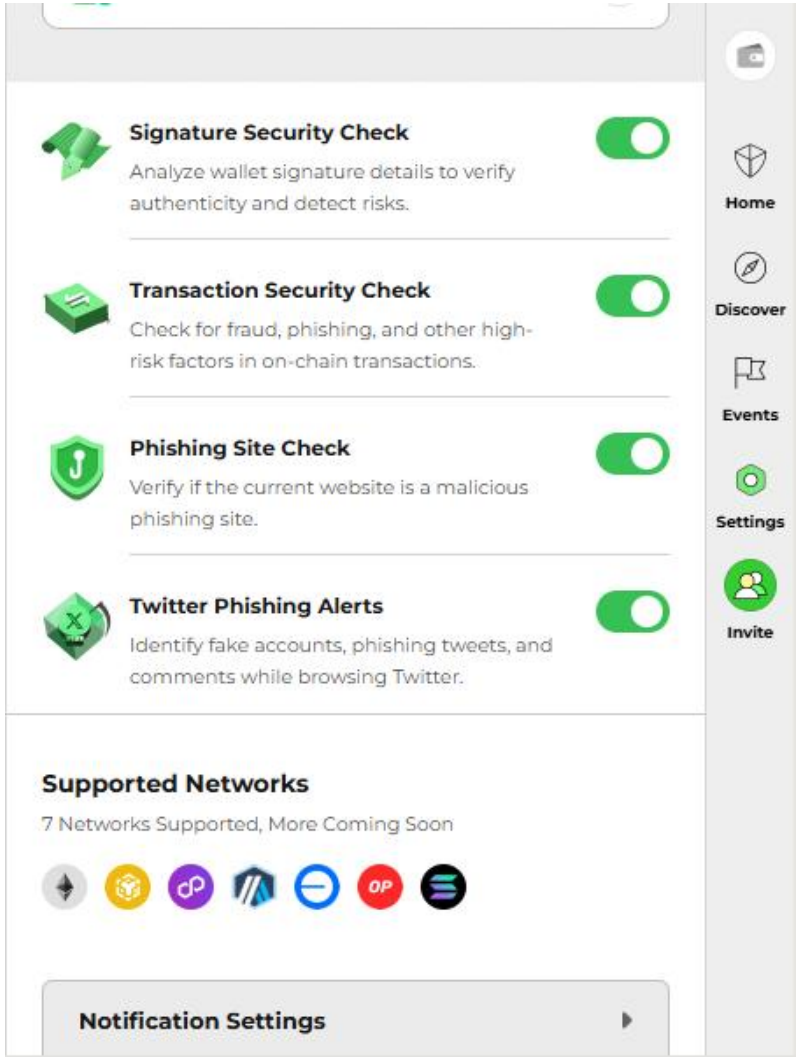




Security Chrome Plugin: Go Plus

Website: https://chromewebstore.google.com/detail/goplus/nfmppnghnhlfnfnhfnahilhfaiahmhfj





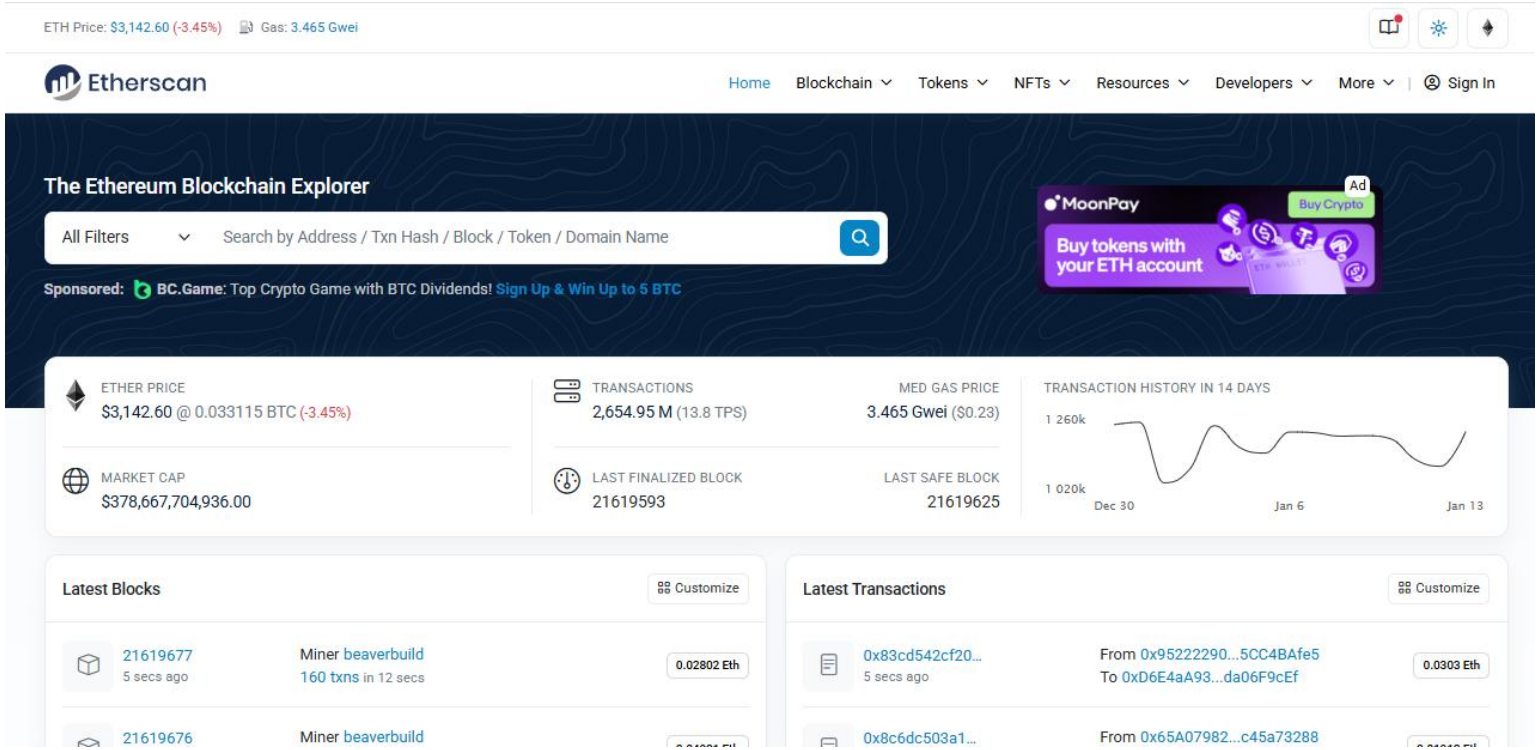


Blockchain Browser

Website: https://etherscan.io/

Etherscan is a blockchain explorer for Ethereum, allowing users to track transactions, view addresses, explore smart contracts, and monitor real-time network activity.

- Each blockchain has its own explorer, e.g., Blockchain.com for Bitcoin, BscScan for BNB Chain, Polygonscan for Polygon, and Basescan for Base.



Blockchain Browser

Case Study(Preventing address poisoning) —— A $68 million Loss

0x1E227979f0b5BC691a70DEAed2e0F39a6F538FD5

Q1: Find Txn 0.05 ETH

Q1: Find the transaction matching the image shown below.



Q2: Understanding Intent

Q2: What is the address 0x1E22 attempting to do here?

Hint: Before transferring large sums to an address, what do people usually do prior as precaution?

Q3: Potential Attacks

Q3: Identify potential attack attempts. Link the transaction.

Hint: There are 2 attack attempts that happen closely after the test transfer. Check the transaction page & token transfer page

Q4: Loss Transaction

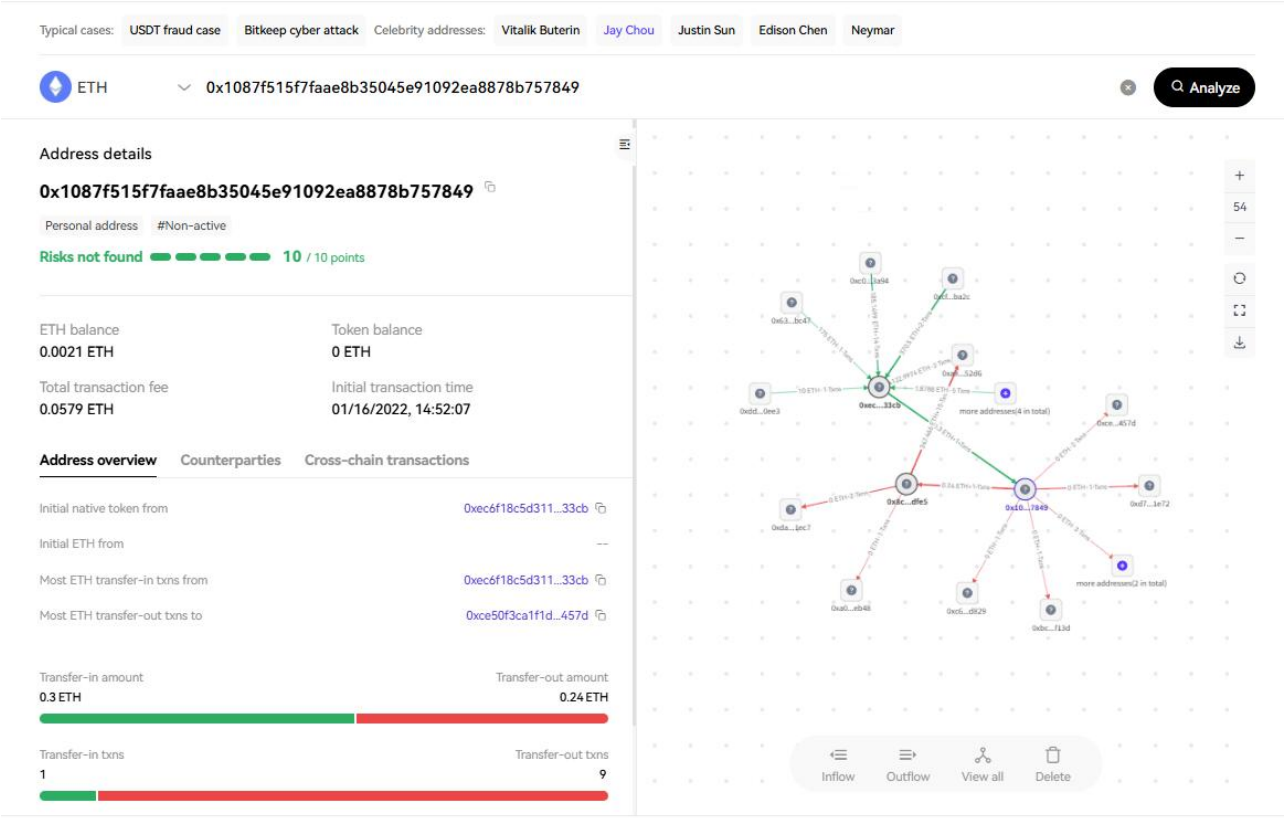
Q4: Identify key transaction leading to the loss of funds. What token was lost? How much?

Hint: It happens shortly after the poisoning attack.

AML Tool —— Oklink and Skyinsights

OKLINK Website:

https://www.oklink.com/chaintelligence/compliance-tools



Skyinsights Website:

https://skyinsights.certik.com/wallet-screening/landing

